

CPE/ME/SSW 424-X2 Senior Design Report

Group C9: Artificial Intelligence for Robust Navigation Solutions

March 31, 2026

Danica Chakroborty

Danny Jutras

Dennis Ren

Caleb Stephen

Andrew Wilson

Dritan Xhelilaj

I pledge my honor that I have abided by the Stevens Honor System.

Table of Contents

Section 1: Project Definition and Plan.....	5
1. Mission Statement (to be completed in IDE 401).....	5
2. Background.....	5
3. Stakeholder List (IDE 401).....	8
4. Analysis of Stakeholder Needs (IDE 401).....	8
5. Project Scope and Resources	8
6. Project Plan (IDE 401).....	9
Section 2: Design, Evaluation, and Optimization.....	10
7. Requirements	10
7a. Product Perspective/Intent	10
7b. Primary Functions.....	11
7c. Operating and User Environments	11
7d. System Requirements.....	11
7e. Hardware Requirements.....	13
7f. Software Requirements.....	16
8. Constraints and Assumptions.....	18
8a. Design and Implementation Constraints	18
8b. Assumptions and Dependencies.....	18
9. Applicable Codes/Standards/Regulations	19
10. Concept Development and Selection.....	19
10a. Software Concepts	19
10b. Hardware Concepts.....	21
10c. Testing Concepts	26
11. Preliminary Design	28
11a. System Architecture	28
11b. System Design.....	35
11c. Mechanical Design.....	36
11d. Electronics Design	42
11e. Software Design	47

12. Design Evaluation Methods	49
12a. Analysis Cases and Results	49
12b. Impact on System Design	55
12c. Testing Considerations	55
13. Alpha Prototyping: Plan and Budget.....	56
13a. Hardware Prototyping Plan	56
13b. Software Prototyping Plan	63
13c. Prototyping Budget	68
14. Testing and Evaluation: Plan and Results	69
14a. Hardware Testing Plan and Results.....	69
14b. Software Testing Plan and Results.....	75
15. Beta Prototyping	80
15a. Mechanical Design.....	80
15b. Electronics Design	83
15c. Software Design	83
15d. System-Level Design.....	92
16. Fabrication, Assembly, and Manufacturing Processes.....	93
17. High-Performance Testing and Evaluation	93
18. Final Design Specification.....	100
Section 3: Entrepreneurship and Business Development	101
19. Business Objectives and Risks.....	101
20. Competitive Intelligence: Market Analysis	101
21. Lean Canvas Business Model	101
22. Financial Analysis	101
23. Intellectual Property	101
24. Innovation Expo (Pitch).....	101
Section 4: Appendices	101
A. Team organization chart	101
B. Project Gantt chart.....	101
C. Prototyping and Testing Budget	101
D. Design Supplementary Materials	101

D.1. Technical Analysis MATLAB Scripts	101
D.2. Jetson Nano Beta Prototype Scripts and Programs	108
E. References and Bibliography	114

Section 1: Project Definition and Plan

1. Mission Statement (to be completed in IDE 401)

2. Background

In mobile robots, localization is crucial to performing many tasks such as autonomous navigation, industrial agriculture and rescue operations. Mobile robots commonly use a combination of Global Navigation Satellite System (GNSS) and other perceptual (or exteroceptive) sensors such as cameras, Light Detector and Ranging (LiDAR) sensors, and proximity sensors to determine their position [1]. However, in environments where there are no known reference points (such as GPS denial), using these sensors is challenging [1], [2]. Additionally, cameras significantly increase the level of computation required to decode the signal and interpret it.

Inertial Measurement Units (IMUs) are a combination of sensors that can be used to identify the position and pose of a mobile robot. They typically consist of an accelerometer used for translational motion tracking, a gyroscope used for rotational motion tracking, and a magnetometer used to identify magnetic fields for more accurate pose identification [3]. These units are crucial in many fields, including mobile robots. For budget-sensitive vehicles specifically, micro electrical-mechanical systems (MEMS) IMUs provide a cost- and size-effective way to obtain inertial measurements. Currently, IMU integration performed by companies such as Kearfott Guidance & Navigation is aided by lengthy, in-house built algorithms to improve accuracy over extended usage. However, all IMUs suffer from drift over extended usage, necessitating periodic calibration to ensure the desired accuracy. The positional drift of MEMS IMUs is typically more significant than other types of IMUs, such as fiber optic or ring laser gyroscopic IMUs, coming as a tradeoff for their smaller size and lower price [3]. MEMS IMUs are also prone to resonance of external vibrations, which influences noise in readings as well as causing measurement errors. Despite this, MEMS technology captures a high market share in commercially available IMUs, with 62% of gyroscopes and 85% of accelerometers utilizing the technology [3].

Most IMUs are currently implemented as strapdown INS's (Inertial Navigation Systems), meaning they are rigidly mounted to the vehicle's body [3], [4]. Consequently, the IMU gyroscope axes move as the vehicle moves. The cost to implement strapdown IMU systems are much less than gimbaled IMU systems, where there are three orthogonal gyroscopes on an actuated platform [5]. Then, inertial navigation using dead reckoning is used to identify the position and orientation of the vehicle. However, the values obtained through this method are prone to error, as the sensor data must be integrated twice to provide position information. IMU-aided localization is popular in part due to its high update rate, providing information at upwards of 100 Hz. Additionally, the data used is entirely provided on board the vehicle, theoretically eliminating the need for other data sources such as GNSS. However, in practice, free INS (or inertial navigation without additional aids) is typically only accurate for short-term navigation purposes with low-cost IMUs such as MEMS IMUs [3].

GNSS interference causes significant issues with regards to robot localization when mobile robots employ them as the main source of location [6]. This can take the form of GNSS jamming (where signal is denied), spoofing (where an incorrect location signal is received), and in urban areas where GNSS receivers cannot contact satellites. Figure 2.1 shows an example of real-world GNSS interference [7]. As seen, there are areas where mobile robots could be deployed where they could experience significant GNSS jamming or spoofing. The need for accurate positional knowledge is still required, and in the absence of reliable GNSS information, an alternative solution must be developed.

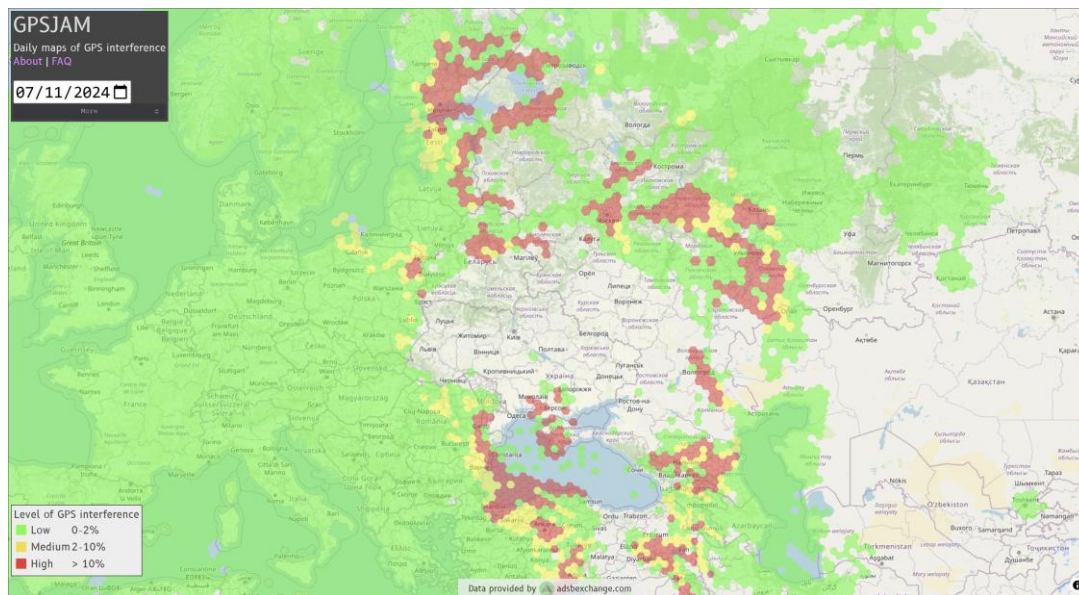


Figure 2.1: Example of GNSS interference map [7]

One potential solution to obtain more confident position measurements in GNSS-denied environments is through Artificial Intelligence (AI) [8]. Determining the performance characteristics of a certain IMU and correcting its potential drift using AI can provide a more confident positional measurement after prolonged usage in a mobile robot application. One recent framework for this is *Advanced Low-grade Inertial Measurement Unit (ALiMU)* [9]. The proposed framework demonstrates promising potential by training a machine learning (ML) model on the differences between low-grade and high-grade IMUs, then deploying only the low-grade IMU in a previously unknown environment. The framework showed up to 33% improvements in dynamic motion conditions with a low response time of 1.438 ms per sample. This makes the framework useful for post-processing and interpreting where a robot has traveled while deployed but leaves room for improvement for use where low latency is key, such as controlling the motion of the robot in real-time.

Figure 2.2 shows the architecture of the ALiMU framework. In the framework, the IMU sensor readings are passed through an attention mechanism, highlighting the important parts while attempting to reduce the effect of noise. Then, the signals are fed into a neural network, where the relationships between the low-grade and high-grade IMU data are found. Finally, the network

outputs the corrected low-grade sensor data that is closer to that of which a high-grade IMU would produce.

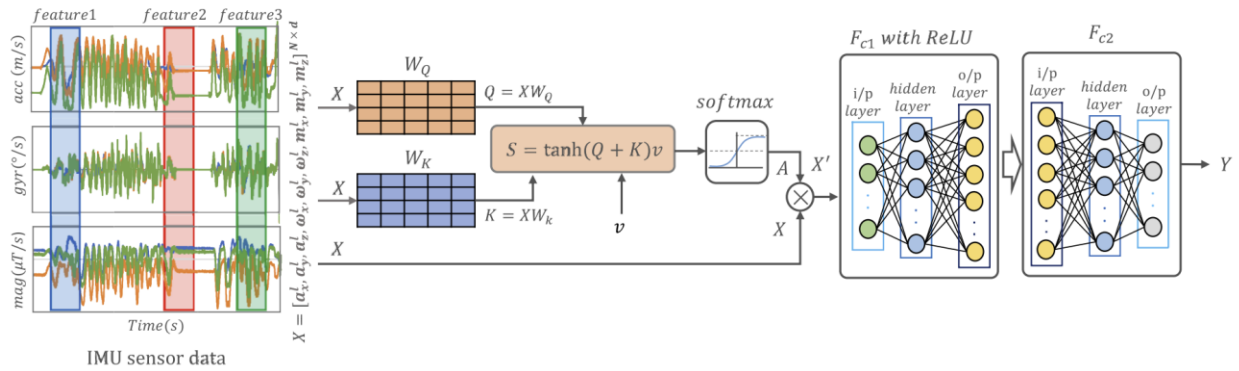


Figure 2.2: ALiMU Framework Architecture [9]

Furthermore, a team of researchers at Military Technical College imposed an ML approach to improve the performance of an Inertial Navigation System (INS) [10]. The team aimed to reduce errors in the form of bias, scale factors, and non-linear noise within traditional IMUs, which contribute to drift. A high-end IMU (IMU-CPT) was used to train a low-cost MEMS-grade IMU (XBOW IMU300CC) using an Adaptive Neuro-Fuzzy Interference System (ANFIS) algorithm. The ANFIS algorithm fuses the learning capabilities of neural networks, and a rule-based reasoning called fuzzy logic to reap the advantages of both techniques and compensate for each one's disadvantages. The algorithm transformed the low-cost IMU into a comparable solution to the high-end one. Specifically, a 70% improvement in position accuracy was observed in the trained IMU in comparison to its original performance.

Figure 2.3 illustrates the ANFIS algorithm structure, consisting of five layers. The first layer has several nodes that assign crisp inputs with a membership function (MF), converting the inputs into datasets. The output suggests the similarity between the input X , to node i , and the linguistic label A_i . The next layer multiplies the input at each node by a weighting to yield each rule's firing strength. The third layer serves to normalize the weights of each rule using the ratio between the weight of each rule to the sum of all the weights. Finally, the fourth and fifth layers multiply the normalized weights by the output of the second layer and sum all the incoming signals to determine the overall output.

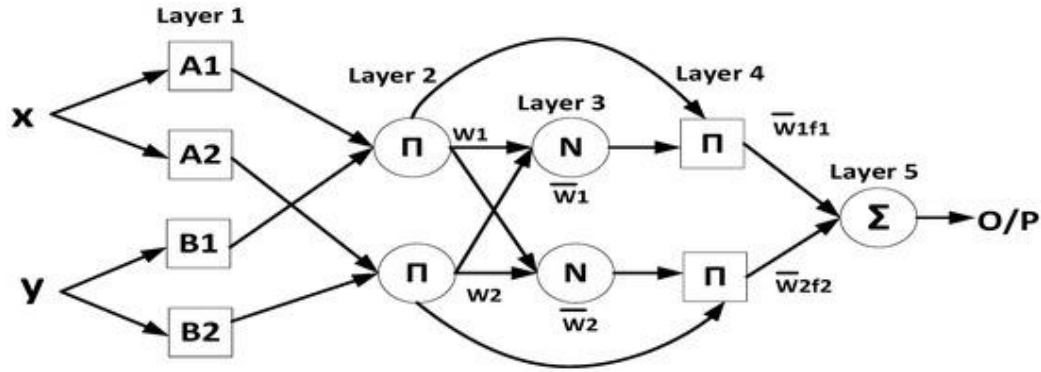


Figure 2.3: ANFIS Structure [10]

The team's model development and handling approach is useful and can be referenced to provide insight into the training and testing phases of our project. This team exercises a common practice for training an ML model, which is to use 50% of the data for training and the other half for testing. While the split does not need to remain in half, this practice can be adopted in our model development phase to ensure it generalizes well to new data, as this is the basis for the problem. Adapting from their research, one possible area for investigation is implementing an Extended Kalman Filter (EKF) to coordinate with a machine learning technique to mitigate the Global Navigation Satellite System outages and denials.

This research demonstrates that machine learning can be used to improve an inertial navigation system by correcting the accumulation of errors in position from integrating sensor measurements over time. Current research focuses on models with combinations of exteroceptive sensors as well as IMUs and focuses on short-term navigation with deployment lengths measured in minutes. Our project will focus on applying AI-driven position correction based only on IMU data to ensure versatility. The model will be deployed on embedded hardware and will be able to provide real-time positional information to the mobile robot. Finally, our project will focus on long-deployment time scenarios, up to hours.

3. Stakeholder List (IDE 401)

4. Analysis of Stakeholder Needs (IDE 401)

5. Project Scope and Resources

The world has become highly dependent on the availability of high-quality GPS for navigation. However, GPS can be disrupted through various methods of interruption including denial of service and spoofing (providing false navigation information). Inertial navigation needs to be improved to overcome the issue of GPS denial [11].

The project scope consists of three main goals:

1. Study methods and techniques to overcome loss of GPS signal
2. Investigate approaches for AI based solutions

3. Consider AI topologies, data structures, neural networks, training models, and adaptive interfaces or AI-based predictive filters

At the end of the project, a demonstration will be performed to showcase functionality. This includes platform development, which will include a platform designed to be mounted on either the Kearfott van or drone for testing. This consists of the navigation system, inclusive of inertial and visual sensors, a GPS receiver, an onboard computer with any required software, and power accommodations. The model created will be deployed onto the platform to obtain results and demonstrate the performance through the deviation from the true position captured through GPS.

The resources for the project include two faculty advisors from Stevens: Prof. Englot and Prof. Vallabh. From Kearfott, there are four engineers we will report our progress to: Kanishke Gamagedara, Bhumit Patel, Matthew Raghunandan, and Sydney Simon. In addition, the following contacts are management-level at Kearfott who will be overseeing our progress: Michael Hlavaty, Joseph Volpe, and Barry Schneider. Additionally, Kearfott will provide a budget of \$10,000 to use for resources required to develop and demonstrate a working prototype.

6. Project Plan (IDE 401)

Section 2: Design, Evaluation, and Optimization

7. Requirements

Identifying system, hardware, and software requirements is a crucial part of the design phase of our project. This will allow us to consider the performance metrics that our project will be evaluated on, as well as the non-performance requirements that are “nice to have” in the prototype phases.

7a. Product Perspective/Intent

This project is centered around the need for reliable robot localization in situations where GPS signals are unavailable. Although GPS signals are commonly the primary source of mobile robot localization, these systems are uniquely vulnerable to GPS denial and spoofing (see Figure 2.1). In this scenario, mobile robots must rely on other methods to accurately identify their position. While cameras and other perceptual sensors are commonly used as a solution, they fall short in many environments where there are no known reference points. Utilizing MEMS IMUs with AI-enhanced interpretation is a potential solution due to their small form factor and affordability compared to other IMUs, but with this comes drawbacks as well. With all IMUs, drift can occur over time and can cause the IMU readings to become unreliable if left uncorrected.

The intent of this project is to incorporate and integrate an AI model to increase the reliability of MEMS IMUs deployed in mobile robots. By developing and training an AI model that can apply corrections to the errors due to drift, the system can deliver navigation data that is closer in quality to what high-grade IMUs provide. In this application, the product is meant to act as an enhancer for the sensors. Overall, the goal of this project is to demonstrate how AI can help bridge the gap between low-grade and high-grade IMUs, making it possible for robots and autonomous vehicles to maintain accurate navigation even in GPS-denied environments while remaining cost-effective and durable.

In this project, the intended use case is for any mobile robot or vehicle (ground or aerial) that needs accurate positional information that primarily uses GPS in typical conditions. Our system is not designed to entirely replace GPS in mobile robots, but rather to complement it when GPS signals are unavailable, providing a robust solution for many use cases. The key use of our system is to begin from a known point (where the GPS information is last known) and interpret the subsequent vehicle movements using inertial sensors enhanced by an AI model. This aims to provide positional data that is more accurate compared to using a standalone IMU. While current solutions focus on using IMU data as a standalone solution, our project will place an emphasis on adaptability, being able to switch seamlessly to and from GPS-denied environments. Additionally, our project will focus on the physical implementation of the system on a vehicle. This will play an important role in testing the feasibility of our system. Finally, our system will not utilize perceptual sensors to increase its usability in situations where there are no known reference points.

7b. Primary Functions

The primary function of the system is broken up into smaller categories, each with requirements to go along with it. To begin, the system must be able to collect data through sensors such as IMUs. The system must then be able to process the data with the use of an AI model and output the vehicle's position, orientation, and velocity. The system must also be able to switch between GPS-active and GPS-denied operation seamlessly, not losing track of its position while doing so. To do this, the system must be able to differentiate between GPS-active and GPS-denied operations, effectively deciding which localization method to employ.

Finally, the system must be able to perform the necessary computation with low latency and processing time. The system must be able to be implemented into a mobile platform such as a ground vehicle or aerial vehicle for testing and have a free-standing power supply. It must also be able to store data for post-processing after deployments are complete.

7c. Operating and User Environments

The system will be deployed in a ground or aerial vehicle and is expected to be housed internally, such that the use of perceptual sensors is not possible. The vehicle is expected to travel in both GPS-compatible and -incompatible environments, demanding an INS to complement the GPS. The system will be equipped with an internal rechargeable battery and have AC power supply compatibility to be functionally resourceful, adapting to the designated vehicle. Externally, it will be convenient to have access to an AC power supply for continuous use, if the vehicle permits. Aside from the operation of the vehicle, the system will not require any special operator training or intervention. With the coordination of the GPS and INS, the system will output the vehicle's position, orientation, and velocity to a monitored system, relaying the information to the vehicle operator in a helpful manner.

7d. System Requirements

7d.i. Functional Requirements Table

The following table states and ranks our functional system requirements from 3 to 1, with 3 being the highest priority and 1 being the lowest priority.

Table 7.1: Functional System Requirements

Requirement	Importance	Notes
System must be fully autonomous and not require special operator training or intervention	3	Operating Environment sections specifies this system is fully autonomous
System must provide real time drift correction aided by a machine learning model with a desired latency less than 50 ms per sample	3	Software Requirements specify the need for the model to respond with correct values in real time
System must store collected data for later post-processing and evaluation	3	Primary Functions section directly specifies this need
System must switch seamlessly between GPS available, and GPS denied modes without loss of position	3	Explicitly noted in the Primary Functions section
System must be deployable on both ground and aerial test platforms	2	Scope specifies testing on Kearfott Van and drone
System must complement GPS navigation but not replace it	2	Product Perspective clarifies the intent in to enhance GPS in denial environments rather than replace it entirely
System must integrate IMU and GPS data through a Kalman Filter (or equivalent)	2	Background mentions a Kalman Filter as a potential fusion method
System must log all raw and corrected sensor data in either a MAT or CSV format for use	2	Software Requirements specify this necessity for training datasets
System must operate on an internal rechargeable battery and support AC power supply	2	Operating Environment specify both onboard battery and AC compatibility

7d.ii. Functional Requirements Description

The highest priority requirements are those which are directly tied to the project's success in achieving our goals, which include maintaining accurate navigation in GPS denied conditions, seamless GPS-IMU fusion, and real-time drift correction. Data from the IMU and GPS will be combined through filtering methods such as a Kalman Filter, while a machine learning model will provide timely drift correction to keep the navigation output responsive. To support evaluation and development for future improvements, raw and corrected sensor data will be recorded in standard formats and stored for later analysis. This design must also be adaptable, operating on the Kearfott van or a drone, and power flexibility through an onboard battery or AC connection. Overall, rather than replace a GPS, the system is to enhance it during outages offering support when signals are unreliable. The system also must remain fully autonomous when providing accurate navigation in GPS denied condition, therefore not requiring special operator training or intervention.

7d.iii. Non-Functional Requirements Table

Table 7.2: Non-Functional System Requirements

Requirement	Importance	Notes
System must remain within platform weight and environment limits such as ingress protection ratings of IP54 or IP52, which indicate moderate dust and water resistance for the system	3	Hardware Non-Functional Requirements define weight limit and IP rating
System must support modular Simulink block design and MATLAB functions for maintainability	2	Software Non-Functional Requirements section specifies this necessity
System must comply with relevant safety and navigation standards	1	Necessary for operation
System must provide visualization tools to make performance metrics transparent to developers and stakeholders	1	Software Requirements mention the necessity for increased ease of use and understanding

7d.iv. Non-Functional Requirements Description

The system must be designed in a modular fashion with Simulink blocks and MATLAB functions structured to support maintainability and collaborative development. Compliance with relevant safety and navigation standards is expected, ensuring the prototype aligns with operational expectations through the planning and prototype development processes. To enhance usability, visualization tools should be incorporated so that performance can be clearly interpreted by both developers and stakeholders without the need of in-depth technical specifications clouding the process. The system must also adhere to weight restrictions and environmental restrictions defined below at the hardware level, which guarantees that the overall system remains suitable for deployment on vehicles.

7e. Hardware Requirements

7e.i. Functional Requirements Table

The functional requirements of our design are considered at this stage to achieve high performance for mobile robot localization. As such, each component's attributes are rigorously evaluated to determine an acceptable specification. These specifications came from researching commercially available products and identifying research where these products are implemented [12], [3]. These requirements are below in

. The importance of each requirement is considered on a 1-3 scale, with 1 representing minimal importance, and 3 representing crucial importance.

Table 7.3: Functional Hardware Requirements

Requirement	Target Spec	Importance
Angular Random Walk	$0.1 \text{ deg}/\sqrt{hr}$	3
Battery Runtime	6 hours	3
Accelerometer in-run bias instability	$100 \mu g$	3
Gyroscope in-run bias instability	2 deg/hr	3
Accelerometer range	$\pm 16g$	2
Gyroscope range	$\pm 800 \text{ deg/s}$	2
GNSS Constellations	GPS, Galileo	2
Hardware functional temperature range	-20°C to $+60^\circ\text{C}$	2
IMU Sampling rate	$\geq 100 \text{ Hz}$	2
IMU DoF	6 or 9	1
GNSS Bands	L1, L2, L5, E1, E5	1
Velocity Random Walk	$0.1 \text{ m/s}/\sqrt{hr}$	1

7e.ii. Functional Requirements Description

One of the most important aspects of the hardware requirements are the sensor specifications. For consumer- and industrial-grade IMUs, most of the errors in long-term positional accuracy is due to gyroscope bias, while accelerometer bias and angular random walk also contribute to overall drift [3]. As such, these are the most important factors to consider when looking at IMUs to use in our system. This is demonstrated in Figure 7.1, showing the estimation error and contribution in overall error due to each component in industrial-grade IMUs. It is shown that for short-term navigation, the accelerometer bias is significant during the first portion of navigation but is overtaken by the gyroscope bias. Based on the information shown, the velocity random walk is considered as the least important specification among the four analyzed.

Table 7.3 summarizes the IMU hardware requirements while also outlining the target specifications. The accelerometer range defines the measurable acceleration range while the gyroscope range defined how much angular velocity can be detected. A six-hour runtime is ideally required so the system can run for long periods of testing and deployment without having to worry about power issues. The IMU must also have 6 or 9 degrees of freedom, meaning it includes both an accelerometer and gyroscope (3 DoF each), with an optional magnetometer to achieve 9 degrees of freedom. A sampling rate of at least 100 Hz is also required to capture real-time motion data. This requirement can also be increased to achieve higher sampling rates for real-time data.

For accurate position, the IMU must support and sync with GNSS constellations GPS and Galileo. These constellations should also be compatible with L1, L2, L5 for GPS while E1 and E5 should be compatible with Galileo.

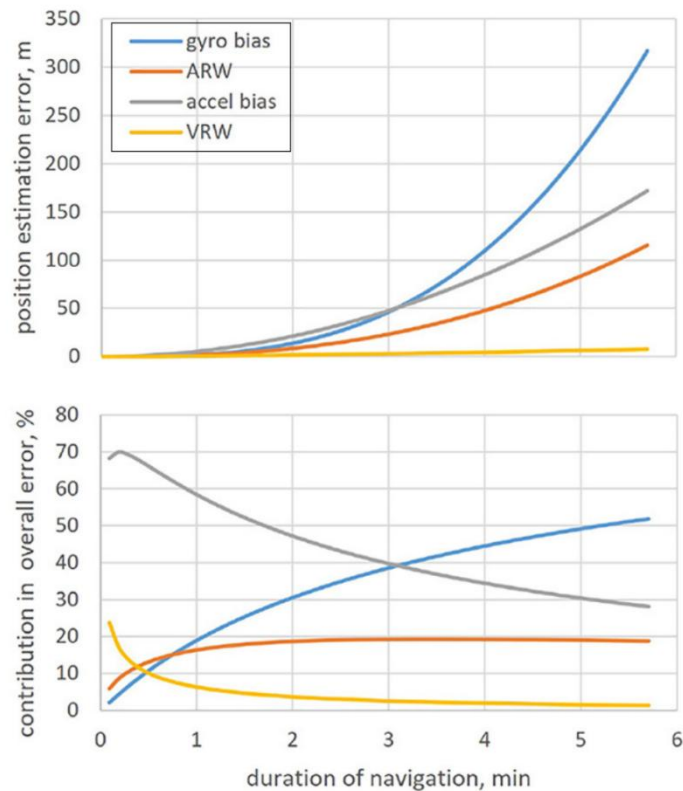


Figure 7.1: Industrial-Grade IMU Error and Contribution versus Time [3]

In addition to the IMU specifications, the GNSS receiver must be chosen to certain specifications/compatibility. Specifically, the receiver must be compatible with both the GPS and Galileo constellations, as these are approved in the United States for civilian use [13]. The receiver must also be able to receive signals from the L1, L2, and L5 satellites. These are the primary signals GPS uses to communicate with receivers, and having L5 compatibility will increase the accuracy of the point that dead reckoning inertial navigation begins. Finally, the battery must have enough capacity to run the system for upwards of 6 hours continuously. On systems with backup batteries, this is not a concern, but on mobile vehicles that will be deployed for many hours at a time, this is crucial to ensure an accurate localization for the vehicle is available for the length of deployment.

7e.iii. Non-Functional Requirements Table

Table 7.4: Non-Functional Hardware Requirements

Requirement	Target Spec	Importance
Package weight	17 lb.	3
Ingress protection	IP54 or IP52 rating	2
Temperature stability	Ability to regulate temperature	2
Ease of maintenance	Easy method of access	2

7e.iv. Non-Functional Requirements Description

The non-functional requirements that our team established are important for the physical and environmental effects on the system. The total package weight of 17lbs is a constraint set by the maximum payload of the Kearfott Drone. Additionally, the team decided that the system must have ingress protection ratings of IP54 or IP52, which indicate moderate dust and water resistance for the system. Meeting these requirements would enable the system to be tested in harsh environments while also ensuring that the hardware remains durable and functional. Additionally, the system must be able to maintain a range of temperature and easily maintain the system through battery replacement, storage replacement, and other common maintenance requirements.

7f. Software Requirements

7f.i. Functional Requirements Table

Table 7.5: Functional Software Requirements

Requirement	Target Spec	Importance
Data collection frequency	>20 Hz data processing rate (50 Hz optimal)	3
Processing time	< 50 ms inference latency per sample	3
Sensor fusion	Timestamp synchronization	3
Sensor Noise Reduction	Kalman Filter Integration	3
Data logging and storage format	Support MAT, CSV	2
Deployment data storage quantity	Must support onboard storage of several hours of IMU/GNSS data and AI model parameters	2
Programming interface	Deployment via MATLAB/Simulink	2

7f.ii. Functional Requirements Description

The functional requirements ensure the system's ability to collect, process and correct IMU data in real time, simultaneously providing reliable navigation output under both GPS available and GPS denied conditions. The real-time IMU drift correction will involve a deep learning model with a Kalman filter, which must infer with latency low enough to keep up with the IMU sampling rate which is greater than or equal to 100 Hz. The software also must integrate sensor fusion algorithms so that transitions between GPS-active and GPS-denied states occur without discontinuity in position. For data logging, comprehensive logging in standard formats such as MAT or CSV is

required to enable supervised learning, retraining and benchmarking of models, and for embedded feasibility, code generate by MATLAB/Simulink must run efficiently on embedded processors to enable portability across vehicle platforms. Moreover, accurate alignment of IMUs and GPS timestreams ensures efficient model training and sensor fusion, hence the need for timestamp synchronization. Failover handling is also integral to the system as it should switch between GPS assisted and GPS denied operation modes without intervention.

Finally, the system must be able to support the continuous logging and storage of the output data. It should be stored and able to be retrieved for post-processing, as well as support numerous consecutive vehicle deployments. This necessitates the requirement of dedicated onboard storage such as an SSD. This data will be processed at >20 Hz, providing timely information to the robot for processing.

7f.iii. Non-Functional Requirements Table

Table 7.6: Non-Functional Software Requirements

Requirement	Target Spec	Importance
Maintainability	Modular Simulink block and MATLAB functions	3
Code Documentation	Inline documentation in addition to design reports	3
MATLAB plots	Automatically generate plots showing vehicle's path (position) during deployment	1

7f.iv. Non-Functional Requirements Description

These requirements ensure that as the team is using MATLAB & Simulink to train a model for the first time, the process of programming is understood by every member of the team as well as other stakeholders. A modular approach will allow for clear direction and efficiency with, for example, one block in Simulink being responsible only for one clear function. This allows for editing without deletion. Documentation will aid this process, especially with a modular approach – with each function being responsible for one clear function, and those functions being well-documented will allow for ease in reviewing and navigating the code both for understanding the code, and for debugging the code. This also allows for parallel work as multiple people can work on separate functions that can collaborate in achieving the result. Additionally, generating MATLAB plots of the vehicle's deployment will aid real-time visualization, the debugging process, and increase system transparency for both developers and stakeholders, displaying complex metrics in a digestible manner.

8. Constraints and Assumptions

8a. Design and Implementation Constraints

The project is limited to a \$10,000 budget provided by Kearfott, which restricts hardware selection, data collection resources, and computational hardware options. All AI models must be deployable on embedded platforms compatible with MATLAB/Simulink, meaning they must operate efficiently under limited processing power and memory. This constraint eliminates the use of excessively large models that require GPU clusters for inference.

The system's total weight must not exceed 17 pounds to remain mountable on both the drone and van testbeds, requiring all components, such as the IMU, power supply, and computing hardware, to be compact and energy efficient. Environmental conditions also impose constraints: all components must operate reliably within the IMU's specified temperature range of -20°C to $+60^{\circ}\text{C}$ and must achieve at least IP52 ingress protection to withstand dust and light water exposure.

Additionally, latency requirements demand that the machine learning model perform drift correction in real time, with a response time under 50 ms per sample. This requirement significantly influences the selection of neural network architectures and training strategies. Testing will be conducted using the Kearfott van and drone platforms, which provide controlled and reliable environments for validation. However, real-world testing opportunities may still be limited by logistical and regulatory factors. For example, scheduling for the Kearfott van to travel to the testing site and conduct driving trials can be challenging due to limited availability on both sides and the fact that the van can only be operated by a Kearfott engineer. Similarly, drone testing is constrained by FAA flight restrictions, including latitude and altitude limits, the need for a licensed Kearfott operator to perform the flights, and the fact that only a few designated areas in Hoboken and New Jersey permit drone operations.

8b. Assumptions and Dependencies

For our project, it is assumed that the GPS will be available occasionally, and the system needs to operate seamlessly between GPS active and GPS denied states. It is also assumed that sufficient training data is essential for developing and testing AI models effectively, and the system will always start from a known reference position. The testing of mobile robot or drone domains will approximate expected real world use case of GNSS-denied navigation. Toolboxes such as MATLAB and Simulink, which includes support for Deep Learning, ANFIS, and EKF, should be available and usable for embedded deployment. When it comes to the AI model, it can be trained locally using Stevens hardware without significant computational or scheduling constraints. It is assumed that the GNSS reference data that is realistic and reliable will be accessible for use when necessary. Testing will take place in Hoboken and will occur in constrained scenarios that approximate real-world dynamics, including activities such as the Kearfott van drives and biking to simulate realistic movement patterns. The project schedule will allow ample lead time, shipping

allowances, and ordering and as needed, last-minute support or system replacement from Kearfott. All scenarios for deployment will remain realistic and controlled on several levels.

When it comes to dependencies, this project relies upon critical availability of project components, ongoing mentorship and technical monitoring from Kearfott engineers and Stevens advisement, and uninterrupted access to GPS capabilities to assess accuracy of the system. Sensor compatibility with MATLAB/Simulink and Jetson Nano are also required for integration and testing, as well as institutional backing for Stevens computing resources for training AI models, access to necessary MATLAB/Simulink toolboxes, for example Deep Learning, ANFIS, EKF, for embedded deployment, and local facilities in Hoboken to perform experimental validation.

9. Applicable Codes/Standards/Regulations

In this project, there are limited codes, standards, and regulations that are applicable due to the research emphasis on this project. The project is focused on a proof-of-concept prototype that demonstrates AI's ability to correct consumer-, industrial-, or tactical-grade IMU data to become more reliable over extended deployment periods. However, for prototyping purposes, there are some considerations that restrict the ability of our testing. For example, the available GNSS constellations in the United States are GPS and Galileo, leaving out GLONASS and BeiDou. While our system might be able to receive these signals, we are prohibited from using them [13]. Additionally, there are flight restrictions on the Kearfott drone that must be considered while testing our design, such as altitude limits and air traffic control clearance before taking off.

Additionally, some IMUs are subject to ITAR regulations. We must be cautious to ensure that our system is compliant with ITAR. The result of this is that we must select a non-ITAR controlled IMU to proceed with our prototype, as we are not cleared to use ITAR-controlled devices.

All other standards, regulations, or applicable codes, we will defer to Kearfott on what are necessary to uphold for this research.

10. Concept Development and Selection

10a. Software Concepts

Several candidate software approaches are being considered for drift prediction. In recent years, numerous machine learning approaches have been used. Early neural network-based methods that utilize pseudo-positions generated from recent GNSS information, such as radial basis function (RBF) and multilayer perceptron (MLP) networks, have been shown to improve positioning accuracy in small intervals of time. Although these models are effective for brief GNSS outages, they fail to utilize longer sets of data.

In contrast, recurrent neural networks (RNNs), particularly LSTM and GRU architectures, are much more effective in combined navigation systems and can make full use of historical information, making fuller use of temporal data to maintain stability during outages [14]. RNN has loops that allow information from earlier steps in the sequence to influence later steps, commonly using a Kalman filter to estimate the true state of the system when measurements are noisy or uncertain [15].

Among these models, LSTM-based networks have been shown to reduce navigation error in case of GNSS outages by an order of magnitude. Though, in practical applications, these neural networks need to be trained on a large amount of historical data before GNSS outages to work, which puts a high requirement on training efficiency. Gated Recurrent Unit (GRU) based networks, on the other hand, especially when combined with Adaptive Kalman Filters (AKF), outperform LSTM models in both prediction accuracy and training efficiency [14].

Another approach utilizes a set of CNNs, seen below in Figure 10.1, using its convolution layers to scan smaller sections of input. Each layer had multiple filters of kernels and a small matrix of numbers, working to detect things like drift or noise patterns in IMU signals [16]. Each convolution filter was trained to detect specific features. By stacking layers, the network builds up from low level features like simple signals to higher level features like drift signature. The CNNs in this study were trained to recognize both deterministic errors like bias and scale factor, as well as stochastic errors such as drift and random noise. By dividing IMU data into small segments to train the networks to classify and correct each individual segment, the authors were able to improve error performance by over 30% in both straight-line and oval motion experiments [16]. They implemented their models in a way that also focused on hardware efficiency, which proved that this type of learning-based error correction could be deployed on embedded systems outside of a purely theoretical tool - which directly correlates with our project.

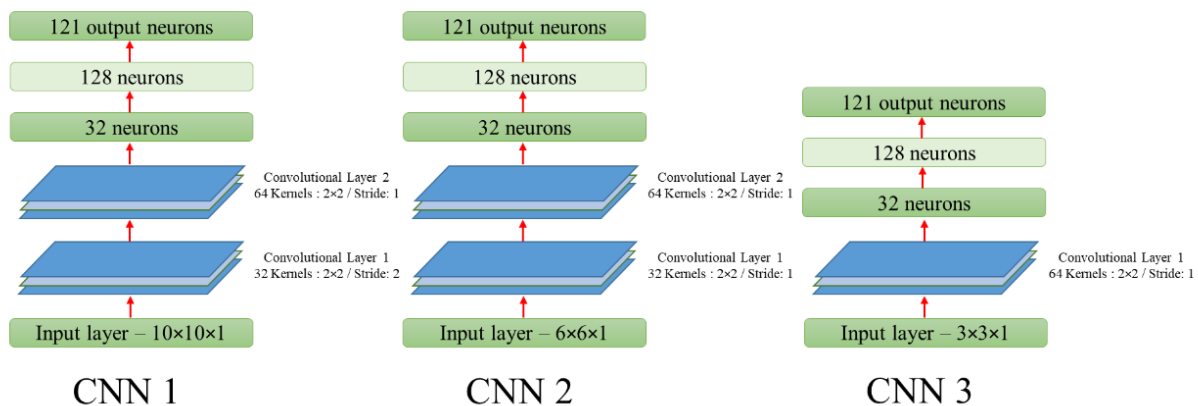


Figure 10.1: CNN Layer Breakdown [16]

Another hybrid model of LSTM and CNN, convolutional neural network-long short-term memory (CNN-LSTM) has been shown to improve training efficiency as well as the speed of feature extraction using CNNs. However, most of the current articles use offline simulations, assuming that the GNSS failure time is known and do not consider the time that is required to train the model online [14].

For our project, a learning model which feeds corrections or constraints into an existing estimator could provide a robust solution without having to reach for a large or computationally intensive network. For this reason, we will be exploring a GRU-AKF or type of CNN approach as these seem the most promising.

10b. Hardware Concepts

10b.i. Hardware Concept Generation

Hardware concepts reflect the physical system implementation, and cover sensor selection, compute unit selection, and a mounting/enclosure platform. To generate hardware concepts, we took inspiration from testing methods described in research papers as well as combining subsystems together. This resulted in three distinct concepts that were evaluated at this point in the project. At this phase, we have not selected any hardware components definitively, such as specific IMUs, GNSS receivers, and compute units. However, they are included in concepts as ideas to build from as the design progresses. For example, the Jetson Nano represents a state-of-the-art compute unit solution that the team is still considering for use.

Concept 1: “Box Enclosure with Antenna” IMU and GNSS with Jetson Nano Enclosure

Our first concept consists of a box-style enclosure, with all the units contained inside. This allows for water and dust-tightness, but reduces airflow throughout the system. There is a hinged door that will provide access to insert/remove the battery and all components. There is a section for the GNSS antenna to stick out, and we envision a clear portion of material to be able to transmit GNSS signal through to the receiver. The focus of this concept is adaptability while remaining simplistic, and there will be a mounting setup on the bottom to ensure multiple testing platforms are compatible with this design. This design has a relatively large cube footprint due to the antenna sticking out, but provides simple manufacturing and setup. This concept is shown below in Figure 10.2.

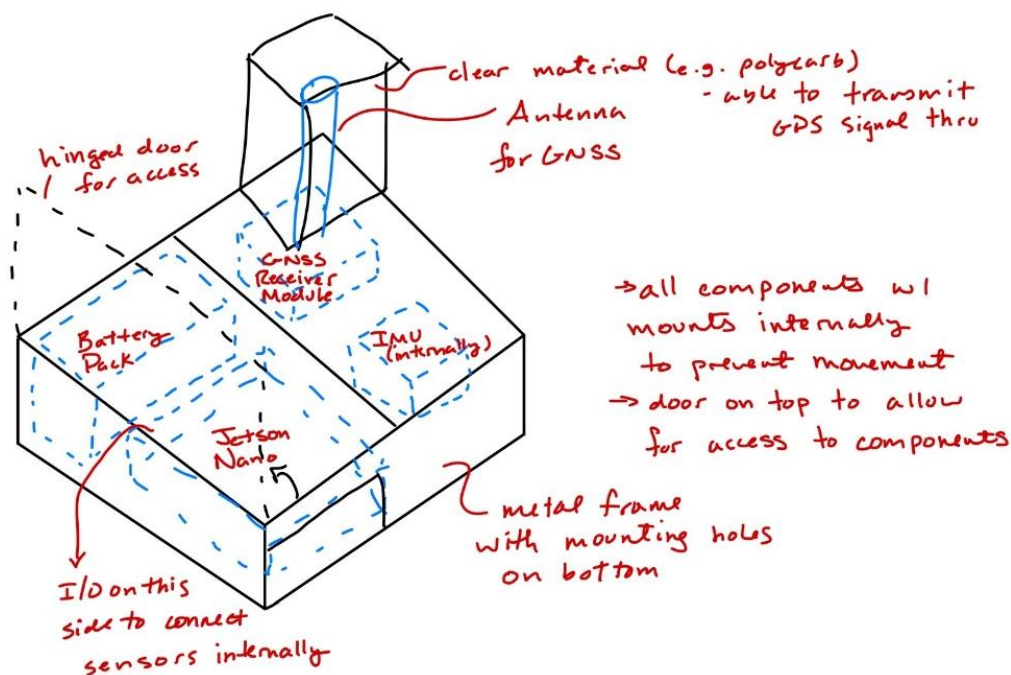


Figure 10.2: Concept 1 Sketch

Concept 2: “Miniature Briefcase” - Multi-IMU and GNSS with Jetson Nano Enclosure

The second concept developed has a briefcase-like enclosure, using hinges to allow for easy access to the internal components. The design is unique due to the integration of a multi-IMU system. This will allow the team to collect more data and identify sensor failures and tendencies, which may contribute to a more robust machine learning model and improved navigation. The system has a GPS receiver with an antenna that protrudes through an air-tight slot in the side of the case for transmission. There are also supply and exhaust fans embedded in the case to improve air flow and temperature management. The focus of this concept is providing an accessible enclosure with increased data collection and temperature control. This concept is shown below in Figure 10.3.

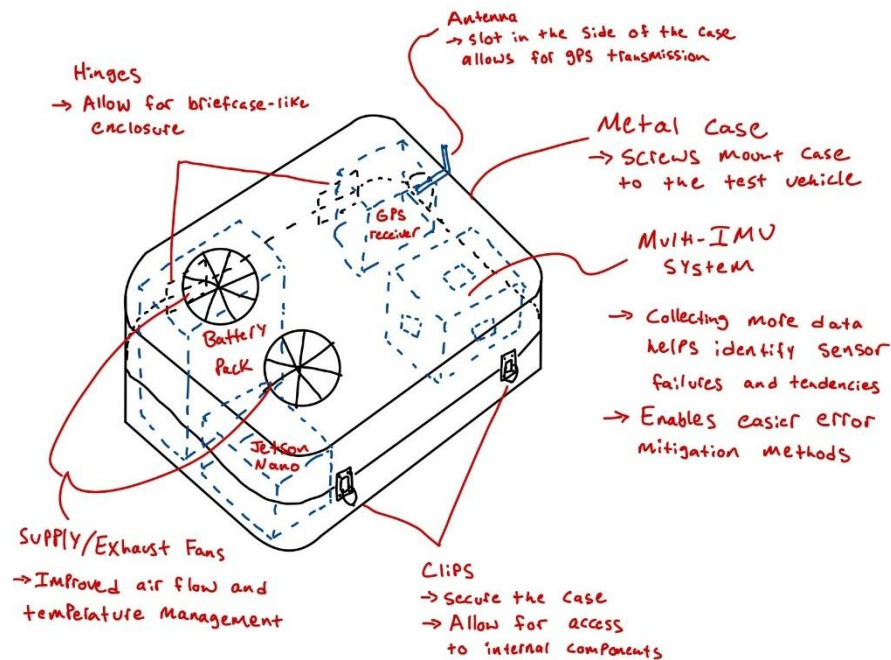


Figure 10.3: Concept 2 Sketch

Concept 3: “Sliding Lid Enclosure” – IMU with Built in GNSS integrated with Jetson Nano Enclosure

Our third and last concept also consists of a box-style enclosure, with all components contained inside. There is a sliding door that will provide access to insert/remove all components. There is a section for the GNSS antenna to sit, and we envision a clear portion of material on the slider lid to be able to transmit GNSS signal through to the receiver. There is a vent on the top of the slider lid as well that will enable the fan from the Jetson to dissipate heat to the atmosphere. There is also a cutout in the side of the enclosure to easily access the I/O of the Jetson. The focus of this concept is adaptability while remaining simplistic, and there will be a mounting setup on the bottom to ensure multiple testing platforms are compatible with this design. This concept is shown below in Figure 10.4.

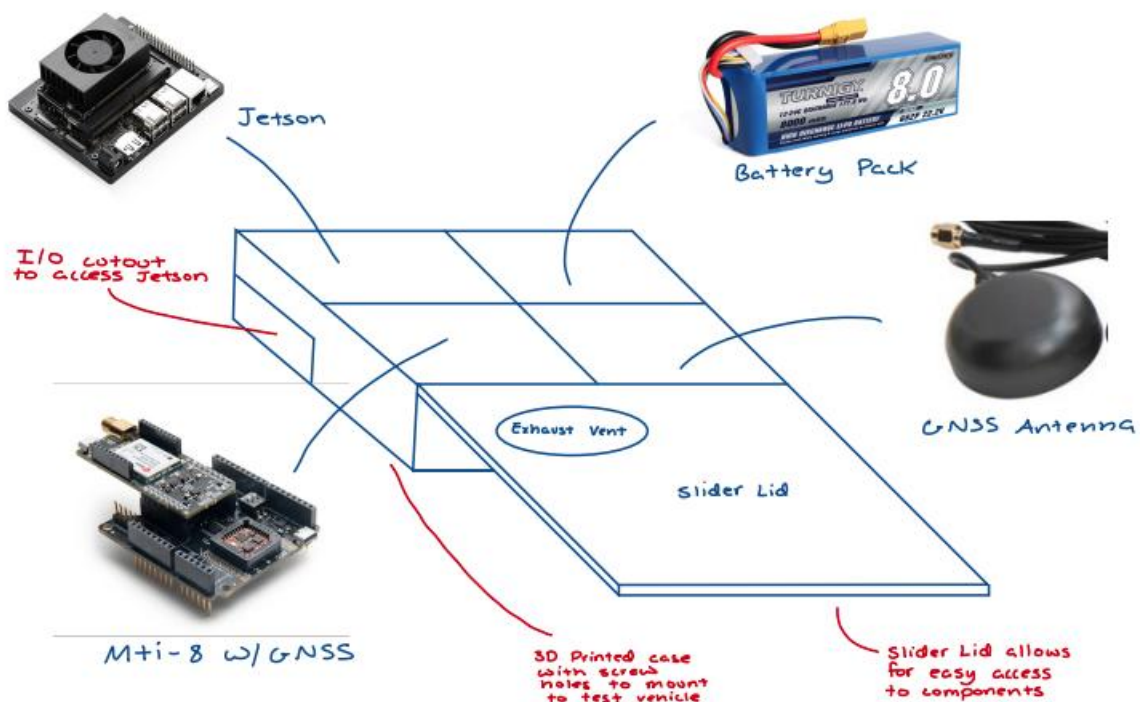


Figure 10.4: Concept 3 Sketch

10b.ii. Hardware Concept Selection

To select a hardware concept, our team first decided what was most important in the hardware portion of the system. Then, each of our concepts were rated based on certain categories and according to certain weights as shown in Table 10.1 below.

Table 10.1: Hardware Concept Selection Metrics and Weights

Number	Selection Criteria	Relative Weight	Description
1	Physical Robustness	5	The system should be able to withstand environments with vibrations, high accelerations, and repeated movements without failure.
2	Ease of Integration	4	The hardware should be able to be integrated easily with test platforms and consider interfaces and physical design.
3	Manufacturability	3	The hardware should be fabricated and assembled with ease.
4	Thermal Considerations	3	The system must be able to regulate its temperature for longer periods of time to ensure sensor accuracy.
5	Size	2	The dimensions and form factors of the system must fit in all test platforms.
6	Power Efficiency	2	The power consumption of the system should be as minimal as possible while achieving desired performance.
7	Ingress Protection	1	The enclosure ideally protects against dust and water entering the system.

With the evaluation metrics selected, our previous concepts will be evaluated. Table 10.2 shows the concept evaluation for our designs based on the criteria in Table 10.1. In Table 10.2, the scores for each design are shown in the first row of each design, and the weighted score is shown on the second row for each design. The scores for each category range from 1-5, with 1 being least capable and 5 being most capable. Then the score is multiplied by the relative weight to get the weighted score for that category. The sum of weighted scores is shown in the furthest right column.

Table 10.2: Hardware Concept Selection Scoring Matrix

		Design 1		Design 2		Design 3	
Selection Criteria	Weight	Rating	Weighted Score	Rating	Weighted Score	Rating	Weighted Score
Physical Robustness	5	3	15	4	20	4	20
Ease of Integration	4	3	12	4	16	4	16
Manufacturability	3	4	12	2	6	5	15
Thermal Considerations	3	1	3	5	15	4	12
Size	2	2	4	4	8	4	8
Power Efficiency	2	3	6	3	6	3	6
Ingress Protection	1	4	4	3	3	3	2
Total Score (max 100)		56		74		79	

As a result of the scoring matrix, Design 3 is selected as the most promising design and will be further developed. However, since Design 2 had a very similar score to Design 3, with only a 5-point difference, the team plans to integrate some of the features from Design 2 into Design 3, specifically the thermal considerations. The fans implemented into Design 2 help improve airflow within the system, resulting in lower temperatures within the enclosure. This ensures that the components remain cool and reduce the risk of overheating.

10c. Testing Concepts

At this stage, the team aims to identify potential testing concepts for our prototype. The focus of testing is to demonstrate the functionality of our system (see the System Requirements section). Accordingly, the testing concepts will be narrowed down through a rigorous scoring process, identifying the most qualified and useful methods to pursue. Shown below, Table 10.3 includes the testing concepts considered by the team and brief descriptions of each.

Table 10.3: Prototype Testing Concepts

Number	Testing Concept	Description
1	Path Train	Evaluates system performance when transitioning between GPS-enabled and GPS-denied environments. The test begins with GPS connectivity, then the prototype enters the PATH train where GPS signals are lost and finally exits back into an area where GPS reconnects.
2	Dynamic Hand Motion	Evaluates the system performance to test the robustness of the AI model by moving the system in a non-predictable motion.
3	Kearfott Van	Evaluates systems performance while traversing through highways and roads with occasional GPS denial when entering tunnels.
4	Bus	Examines system robustness under conditions of increased vibration, noise, and motion typically experienced in public transit.

5	Scooter	Measures functionality and data collection accuracy when subjected to moderate vibration and lower speeds.
6	RC Car	Tests system accuracy and stability when mounted on a small vehicle exposed to high-speed turns and quick directional changes.
7	Running	Evaluates the system's ability to maintain accurate tracking during fast and irregular human motion.
8	Parking Garage (car or walking)	Tests system performance in GPS-denied environments and evaluates how quickly the system requires signal after exiting.
9	Bike on Cobblestone (or rough) Surface	Assesses robustness and data stability when the system experiences continuous vibration or uneven terrain.
10	Kearfott Drone	Evaluates system performance while traversing aurally.

With the testing concepts identified and defined, a list of selection criteria can be created to evaluate each of them. The selection criteria represent the most important areas for successful testing. They can be assigned a relative weighting to better represent the value each contributes to this success. A higher relative weight indicates higher importance. Table 10.4 identifies the selection criteria, their relative weight, and provides a brief description of each.

Table 10.4: Testing Concepts Scoring Metrics

Number	Selection Criteria	Relative Weight	Description
1	Real world Applicability	5	The testing scenario should provide valuable test scenario that emulates deployment conditions, such as intermittent GNSS availability.
2	Ground Truth Reliability	5	The testing scenario should have an accurate reference point to accurately determine the error performance of the system.
3	Hardware Safety	4	The testing scenario should have minimal risk to permanent hardware damage.
4	Repeatability	4	The testing scenario should be able to be repeated to evaluate updates to the system.
5	Cost Efficiency	2	The test should be mindful of any financial costs incurred to run the test.

Now, the selection criteria specified above can be used to rank the testing concepts. Note that a weighted score is calculated for each testing concept with respect to each selection criteria. This score is achieved by multiplying the selection criteria weight by a rating from one to five. The total score for each concept is the sum of its weighted scores for each selection criteria. The concept scoring is depicted in Table 10.5 below.

Table 10.5: Testing Concept Selection Scoring Matrix

	Concept	1	2	3	4	5	6	7	8	9	10
Selection Criteria	Weight	Wt'd. Score	Wt'd. Score	Wt'd. Score	Wt'd. Score	Wt'd. Score	Wt'd. Score	Wt'd. Score	Wt'd. Score	Wt'd. Score	Wt'd. Score
Real-world applicability	5	25	5	25	25	20	15	20	25	20	25
Ground Truth Reliability	5	15	10	25	20	20	15	15	20	20	15
Hardware Safety	4	20	4	20	20	8	8	8	16	12	12
Repeatability	4	16	20	16	20	20	20	12	16	16	12
Cost Efficiency	2	6	10	4	4	8	4	8	10	8	4
Total score (max 100)		57	49	90	89	76	62	63	87	76	68

Based on the scoring matrix, we will proceed with the top three concepts: Kearfott van, bus, and parking garage for our initial system testing. The remaining testing concepts will remain open to consideration for future testing. The high-scoring concepts will provide adequate variety and scenarios to start with for our designed system.

11. Preliminary Design

11a. System Architecture

The architecture of our system is crucial to determining the preliminary design. The core of the system is shown below in Figure 11.1. Here, the main elements of the system are shown, with the IMU and GNSS acting as the main sensors to sense the vehicle's motion in its environment. The battery will supply to all elements, and the compute unit will be utilized to provide AI-enhanced processing of the raw data from the IMU and GNSS sensors. Then, the vehicle's position, orientation, and velocity will be provided as the end result. This demonstrates the system's broad performance and the inputs/outputs of the system as a whole.

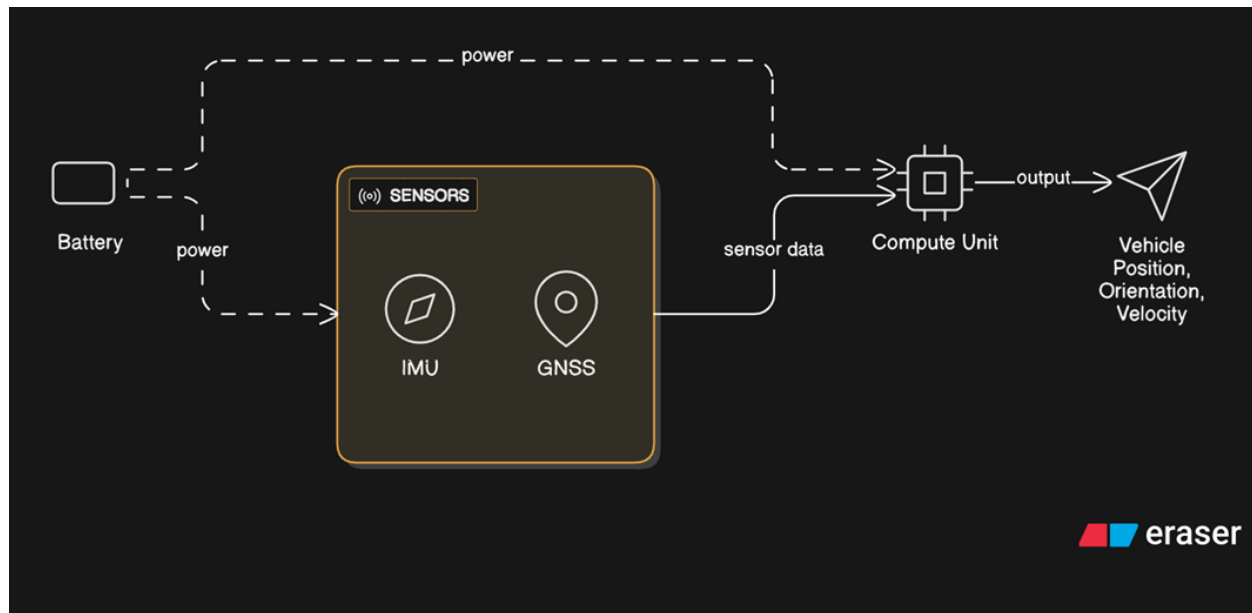


Figure 11.1: System Architecture Overview

11a.i. Hardware Architecture

The hardware architecture represents the foundation of the AI-enhanced navigation system. It integrates sensing, computation, and power components to enable data acquisition, processing, and evaluation of navigation accuracy in GNSS-compatible and GNSS-incompatible environments. Moreover, it illustrates how the relationships between components yield an output. The system consists of six hardware components: a battery, an IMU, a GNSS, and an embedded compute unit. The battery supplies power to each component. The IMU and GNSS collect positional data that can be utilized by the compute unit. The compute unit will apply our drift-correction algorithm to output the vehicle's position, orientation, and velocity in real-time. Shown below in Figure 11.2 is the hardware architecture diagram that helps visualize the system.

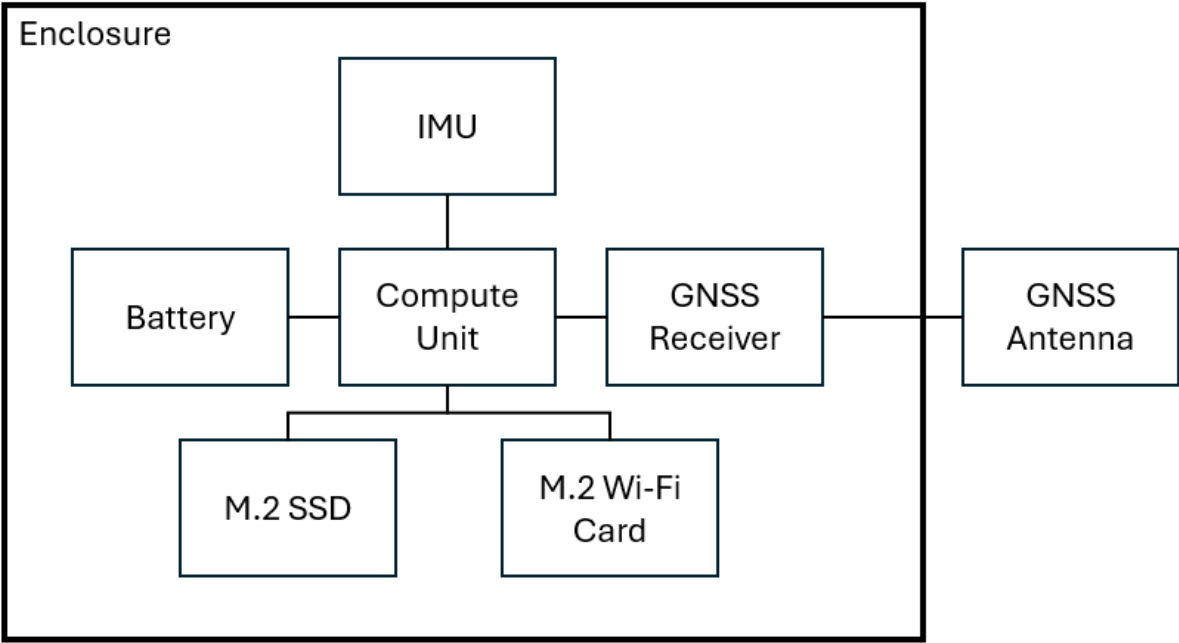


Figure 11.2: Hardware Architecture Diagram

Furthermore, each hardware component has a function that contributes to system operation. Identifying these functions and noting their role in the system more clearly defines the hardware architecture. Shown below, Table 11.1 describes the hardware components, their functionality, and their role in the team’s system.

Table 11.1: Hardware Component Functionality

Component	Description	Function	Role
IMU	3-axis accelerometer and gyroscope	Measure linear acceleration and angular velocity	Provide inertial data to understand motion and pose in GNSS-incompatible environments
Battery	Rechargeable, high-capacity, and high-power battery	Provide power to all components	Enables stable operation during testing with a consistent power supply
GNSS Receiver	Multi-band, centimeter-level accuracy receiver	Provide accurate position	Use as ground truth, a drift-correction reference for the IMU. Primary navigation method for GNSS-compatible environments
Compute Unit	Compact embedded AI micro-computer	Execute sensor fusion algorithms, AI model inference, and data logging	Process GNSS and IMU data for real-time inference. Integrates SSD and Wi-Fi card

M.2 SSD	Solid-state drive storage device with M.2 interface	Provide data storage	Provide large dataset storage for IMU, GNSS, and model predictions. Use for offline training
M.2 Wi-Fi Card	Wireless communication module with M.2 interface	Enable wireless connection	Provide GNSS-correction data

11a.ii. Software Architecture

The software architecture is the main component of the AI-enhanced navigation system, orchestrating data ingestion, synchronization, estimation, learning-based drift correction, and logging. It is to be designed as a set of modular functions running on the embedded compute unit, with clear interfaces to the IMU and GNSS sensors to meet real-time constraints. The architecture supports two operating modes —GPS active and GPS denied— using a management function that transitions between them without discontinuities in the state estimate. The software architecture diagram is shown below in Figure 11.3.

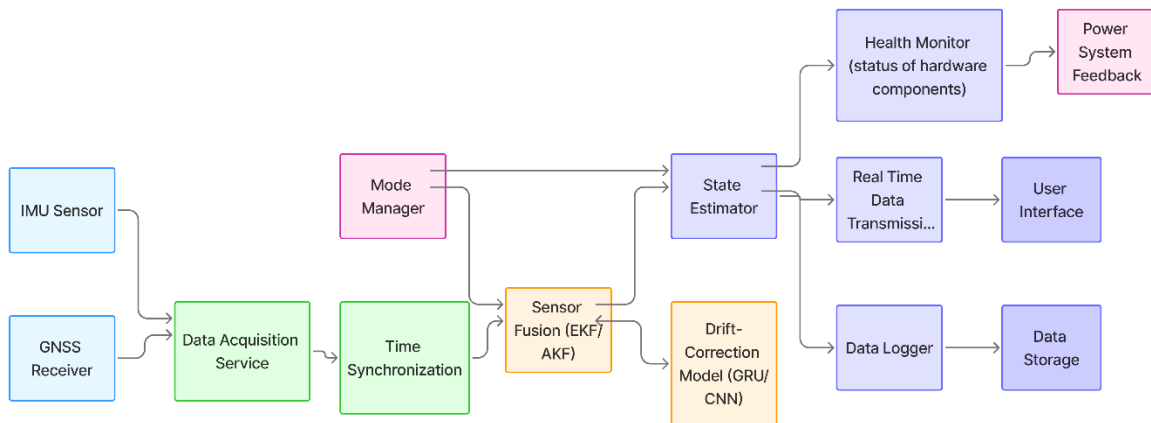


Figure 11.3: Software Architecture Diagram

As depicted, the software architecture is categorized by various colors. Each color has a specific function in the system. The category and role for each color in the software architecture diagram are defined below in Table 11.2.

Table 11.2: Software Component Functionality

Color	Category	Role in System
Light Blue	Sensor Inputs and External Data Sources	Represents the hardware sensors feeding data into the system, specifically the IMU Sensor and GNSS Receiver, which are the physical devices providing raw motion and position data.
Light Green	Data Handling and Pre-processing Layer	Cover modules that collect, timestamp, align raw sensor data, and both components ensure consistent and reliable data flow before computation.
Light Pink	Supervisory Control and System Management	The mode manager oversees GPS-active and GPS-denied transitions, while the Power System Feedback provides safe operational control signals back to the hardware.
Light Orange	Core Computational and AI Processing Layer	Denotes the estimation and correction algorithms that form computational components, such as the sensor fusion and drift-correction modules which combine filtering and AI-driven correction to produce accurate navigational estimates.
Dark Blue	System Outputs, Monitoring, and Interfaces	Encompasses all the modules that output data, monitor system health, and interact with user, which includes the state estimator, health monitor, real-time data transmission, user interface, data logger, and data storage. These elements handle system reporting, diagnostics, telemetry, and persistent storage.

The navigation control logic consists of nine states that govern the software behavior from power on to shutdown, which are detailed below and clarified in Figure 11.4.

1. Init: The system boot state – the Jetson initializes the hardware interfaces, load configuration files, and launches essential background services, and if initialization fails, the system transitions directly to Fault.
2. Sensor Check: Validates IMU and GNSS connectivity by polling sensor drivers and confirming signal availability, and successful verification advances to Calibrate, failure or timeout routes to Fault
3. Calibrate: Performs initial bias estimation and alignment using stationary IMU data, and once calibration converges, the system proceeds to Normal Operation. If calibration fails or exceeds allowable error, the software moves to Fault.
4. Normal Operation: The steady state mode in which all navigation computations occur, and this superstate contains several sub models detailed below:
 - a. GPS Active: Indicates that a reliable GNSS fix is present, and the EKF/AFK performs full measurement updates, integrating GNSS and IMU data for precise state estimation

- b. GPS Denied: Triggered when GNSS signals are lost or degraded, and the navigation system relies solely on inertial data aided by the machine-learning drift correction model. The system continues propagating inertial states while searching for a satellite fix
 - c. Degraded: A protective sub mode invoked when thermal, voltage, or CPU thresholds are exceeded, so computational load is reduced, logging is throttled, and the health monitor attempts recovery, which routes back to GPS Active or GPS Denied depending on GNSS availability.
5. Fault: A global fail-safe state entered from any operational mode when a critical event such as hardware fault, software assert, or sensor dropout occurs, and the software logs diagnostic data and assesses whether recovery is possible.
 6. Retry Re-Init: A transient recovery path used when a fault is recoverable; the system restarts essential services and returns to Init for re-initialization.
 7. Safe Shutdown: The terminal state reached after a non-recoverable fault - all processes close gracefully, power is disengaged, and data logs are finalized to prevent corruption.
 8. Shutdown: The normal termination path commanded by the operator – logging and telemetry stop cleanly, and the system returns to idle power state.

Transitions between states are driven by sensor state, GNSS quality metrics, temperature and voltage monitors, and user commands. This state architecture ensures deterministic startup, adaptive navigation under varying GNSS conditions, autonomous safety recovery, and shutdown for both operator-initiated and fault-triggered scenarios.

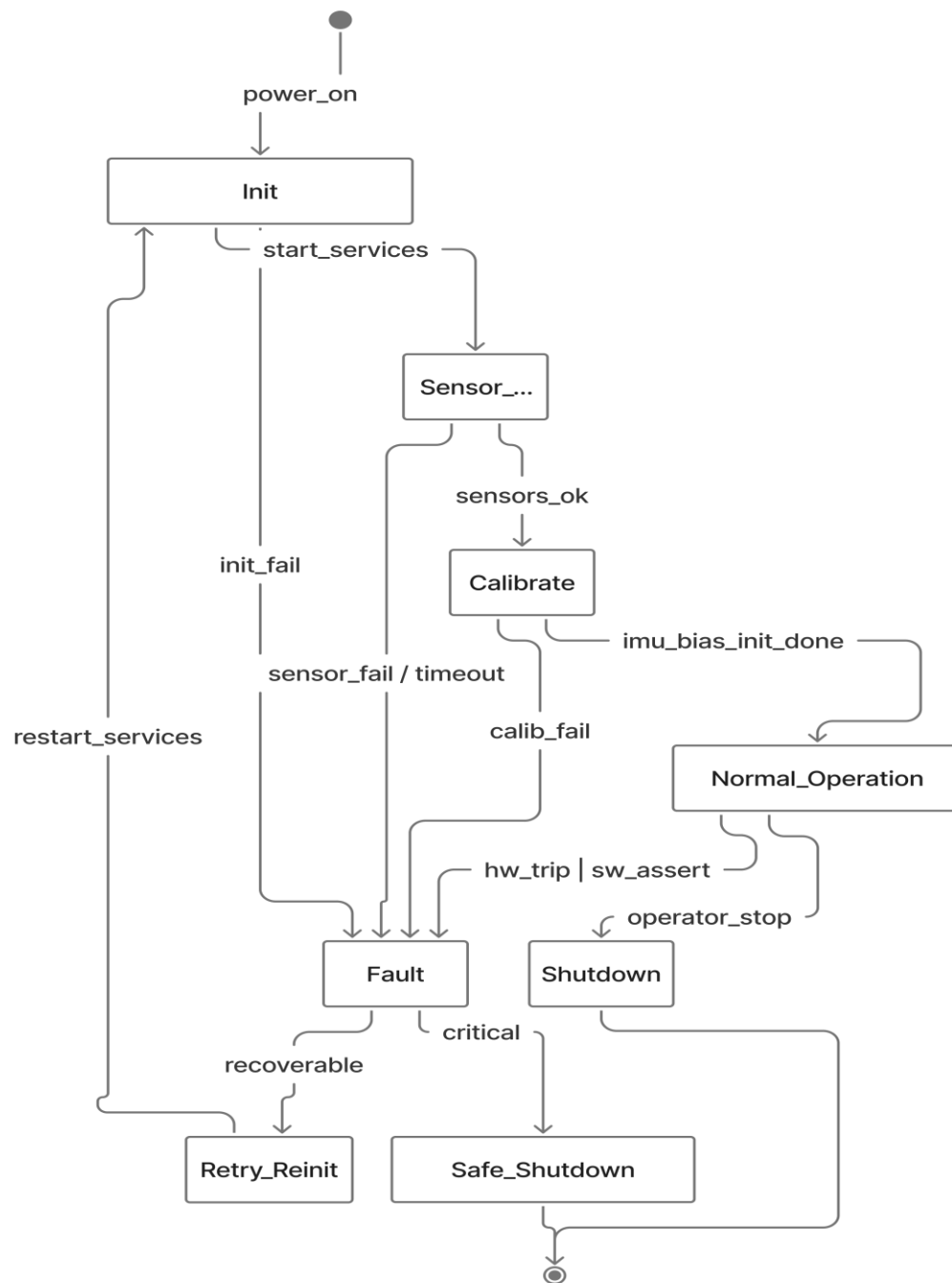


Figure 11.4: Navigation Control Logic Diagram

11b. System Design

11b.i. System Behavior Description

The system will continuously collect IMU and GNSS data during operation. The IMU will provide linear acceleration and angular rate measurements at high frequency. On the other hand, the GNSS module will supply position and velocity when a signal is available. Subsequently, all the incoming data will be timestamped and synchronized before being sent to the GRU-AKF algorithm. The GRU network will predict the position during GNSS outages. The adaptive Kalman filter will fuse this prediction with inertial data to correct drift and estimate position in real time. The system will operate in training mode to update the GRU model and calibrate fusion parameters when GNSS signals are available. When GNSS signals are lost, it will switch to prediction mode, where the GRU will generate pseudo-GNSS measurements for the Kalman filter. Corrected position, velocity, and RMSE values will continuously be outputted to the terminal and logged to onboard storage. After each test run, the data will be retrieved for post-processing for analysis, retraining, and performance evaluation.

11b.ii. Data Storage

The system keeps in storage the raw IMU readings, GNSS measurements, GRU outputs, and AKF corrected data for future assessment. The logging is performed in MATLAB compatible formats. Timestamped acceleration, velocity, angular velocity, attitude, and predicted position variation values as defined in the GRU-AKF framework are included in each dataset. These will be stored on a storage device attached to our compute unit (such as a M.2 SSD) for future use.

In our research, we have come across datasets that contain IMU data, such as *ALiMU* [9]. In this dataset, real IMU data was collected from two 9-DoF IMUs. When collected at 40 Hz, 14990 samples takes up ~2.5 MB of space. This represents approximately 6 minutes 15 seconds of real-time data for their setup. Based on this, to store deployment information for 4-6 hours, there should be at least 150 MB of free space to store data. Because our system will only contain one IMU, this can be approximated as half of this, or 75 MB free space required. The GNSS data will take up storage, but less than the IMU as it only collects data at maximum 25 Hz, and only outputs four measurements compared to nine of the IMU. Time synchronization will also take some storage to store timestamps, as well as storing the AI model on the local edge hardware. Based on this, a M.2 SSD that is readily available with a capacity of 500 GB or greater will have sufficient storage for a significant amount of deployed operation.

11b.iii. Data Conversion

The data from the IMU and GNSS receiver will be time-synced for processing. This will be done by matching the time of the GNSS receiver to data received by the IMU at that instant. The IMU selected must support time synchronization protocols to achieve this. This will be aided by the selection of an IMU that is supported extensively with libraries, technical support, and

community forums, as discussed in the System Requirements. Because the IMU will have a higher sampling rate than the GNSS receiver, the GNSS receiver data can be used as a trigger to pull the most recent IMU data for sampling. This method will essentially allow us to collect time-synced data.

11b.iv. Sub-System Interfaces

Our project functions as a single system and doesn't have any sub-system interfaces.

11b.v. User Interface Design

There will only be a user interface when testing the machine learning model on a computer that is not the Jetson Nano. The user interface will display real-time output metrics through terminal, such as the predicted position and root mean square error, or RMSE, which are generated from the GRU-AKF algorithm. During deployment on the Jetson Nano, the metrics will instead be continuously recorded and saved to a csv file, which can be extracted for examination. To visualize these metrics, a basic dashboard will be developed for post processing and performance evaluation. This will allow users to view RMSE trends, drift correction effectiveness, and trajectory accuracy after each test run.

11c. Mechanical Design

11c.i. Mechanical Components and Bill of Materials

The mechanical design of the system uses a mix of structural hardware and mounting components to secure all components inside the 3D-printed enclosure. Each part was chosen to provide good strength, light weight, and easy assembly during testing and maintenance. Stainless steel and aluminum were mainly used because they resist corrosion and can handle repeated tightening without damage to the enclosure.

The assembly uses 18-8 stainless-steel helical inserts to reinforce threads in the 3D-printed plastic housing, allowing screws to be removed and reinstalled without wearing out the threads. Aluminum socket head screws and washers are used for fastening, providing both durability and a clean appearance. Barrel plug cables are used to connect the power system, while the SMALL-RIG V-mount VB99 battery supplies portable power for field testing. Additionally, the IMU will be provided by the team's project sponsor, Kearfott Corporation. Together, these mechanical components ensure the enclosure is sturdy, modular, and ready for integration with the navigation and computing hardware. These parts are labeled below in Table 11.3. Note: The selection of any new hardware components listed here is discussed further in 11d.ii and 11d.iii.

Table 11.3: Preliminary Design Bill of Materials

Product Name / Description	Quantity	Unit Price	Price
GNSS Receiver Board: u-blox EVAL BOARD FOR ZED-X20P	1	\$ 299.99	\$ 299.99
NVIDIA Jetson Orin Nano Super Developer Kit	1	\$ 249.00	\$ 249.00
Crucial P310 1TB SSD, PCIe Gen4 NVMe M.2 2280	1	\$ 72.28	\$ 72.28
Intel Dual Band M.2 Wireless Card	1	\$ 16.88	\$ 16.88
Battery: SMALLRIG V Mount VB99 Battery	1	\$ 199.99	\$ 199.99
Barrel Plug Cable	2	\$ 3.89	\$ 7.78
18-8 Stainless Steel Helical Insert, 1/4"-20 Right-Hand Thread, 3/4" Long	1	\$ 5.15	\$ 5.15
Aluminum Washers	1	\$ 12.60	\$ 12.60
Aluminum Socket Head Screw	4	\$ 3.91	\$ 15.64
IMU (Movella MTi-3-5A)	1	\$ 469	\$ 469
TOTAL			\$ 879.31

11c.ii. CAD Models and Drawings

Two CAD models were developed for the project enclosure. One utilized a sliding lid mechanism, and the other featured a screw-on lid design. Both models were created in SolidWorks to evaluate assembly practicality and ease of access to components for performing maintenance. The GNSS receiver is mounted on the lid in both designs, so it has a clear signal and doesn't get interference from the parts inside the box.

The first enclosure design uses a sliding lid that fits into a guided track along the sides of the enclosure wall. This allows the top to slide open and give easy access to the internal components. This makes the assembly process and wiring much easier during early testing. The enclosure measures 10.75 in x 7.75 in x 2.875 in, giving plenty of space for components and wire management. While this design will be simple and easy to use, it will not seal as well and will not be as structurally strong. The sliding mechanism will leave the interior more open to dust and vibration, which could cause issues when the system is being transported and tested in different environments. Figure 11.5 and Figure 11.6 illustrate an isometric view of the design 1 CAD model with the enclosure closed and opened, respectively.

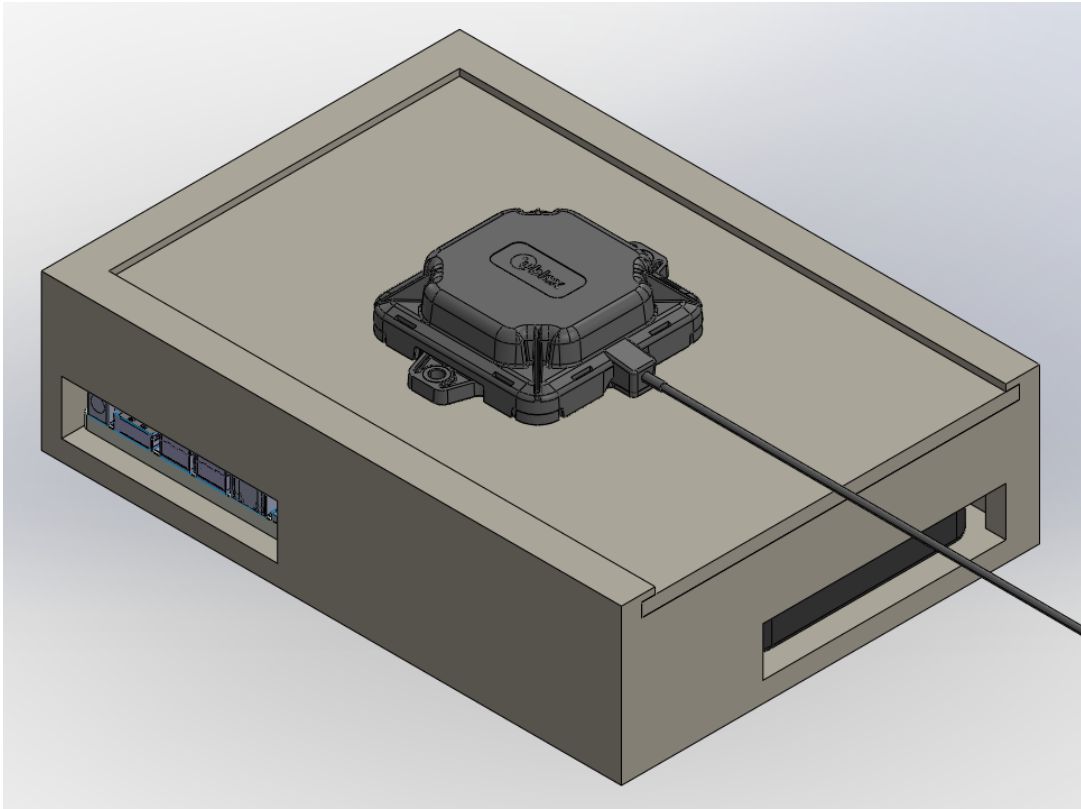


Figure 11.5: Design 1 Sliding Lid Enclosure (Closed)

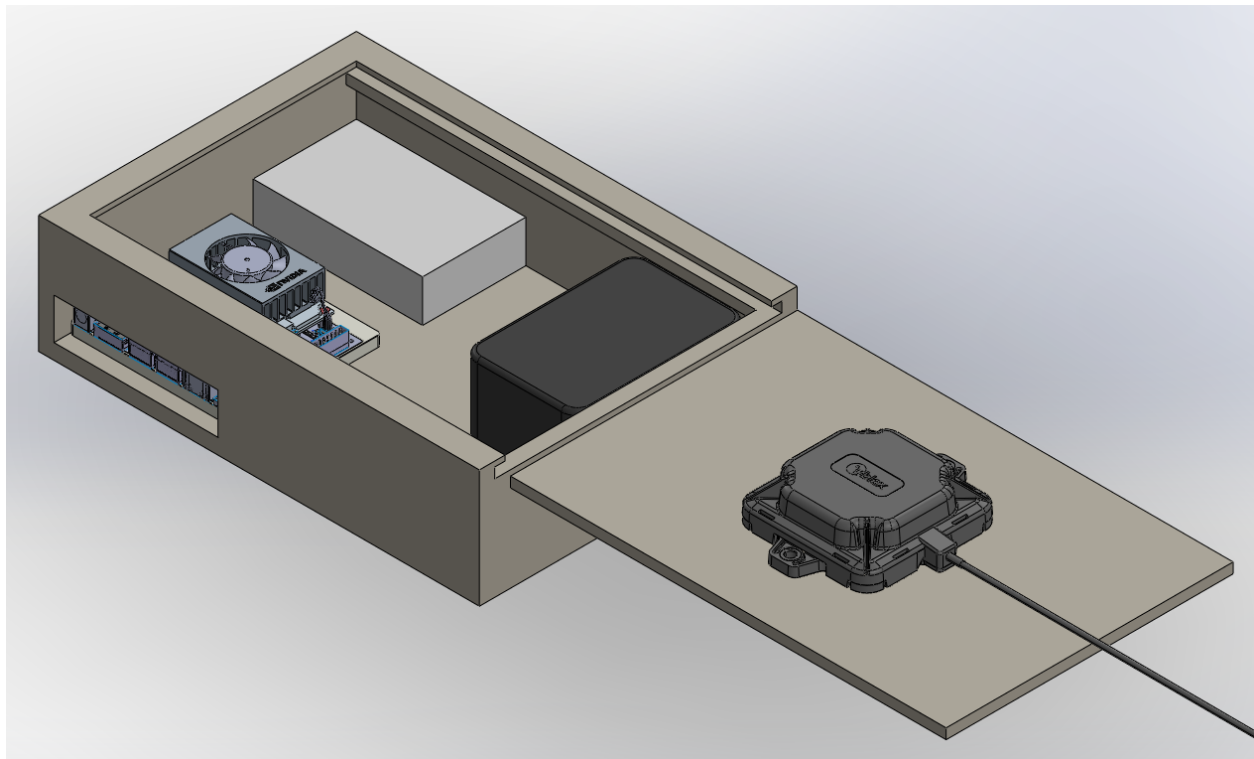


Figure 11.6: Design 1 Sliding Lid Enclosure (Open)

The second design utilizes a screw-on lid fastened at each corner with four screws. This design will create a tighter seal and make the enclosure stronger and more stable during transportation and testing. The screw on lid enclosure measures 9.5 in x 9.5 in x 3.75 in, making it slightly taller but smaller in overall footprint. Although this design will take a bit longer to open, it will offer better protection and durability. The team will move forward with this design because it is more secure, reliable, and suitable for long term use especially when exposed to vibrations or rough handling. Figure 11.7 and Figure 11.8 illustrate an isometric view of the design 2 CAD models with the enclosure closed and opened, respectively.

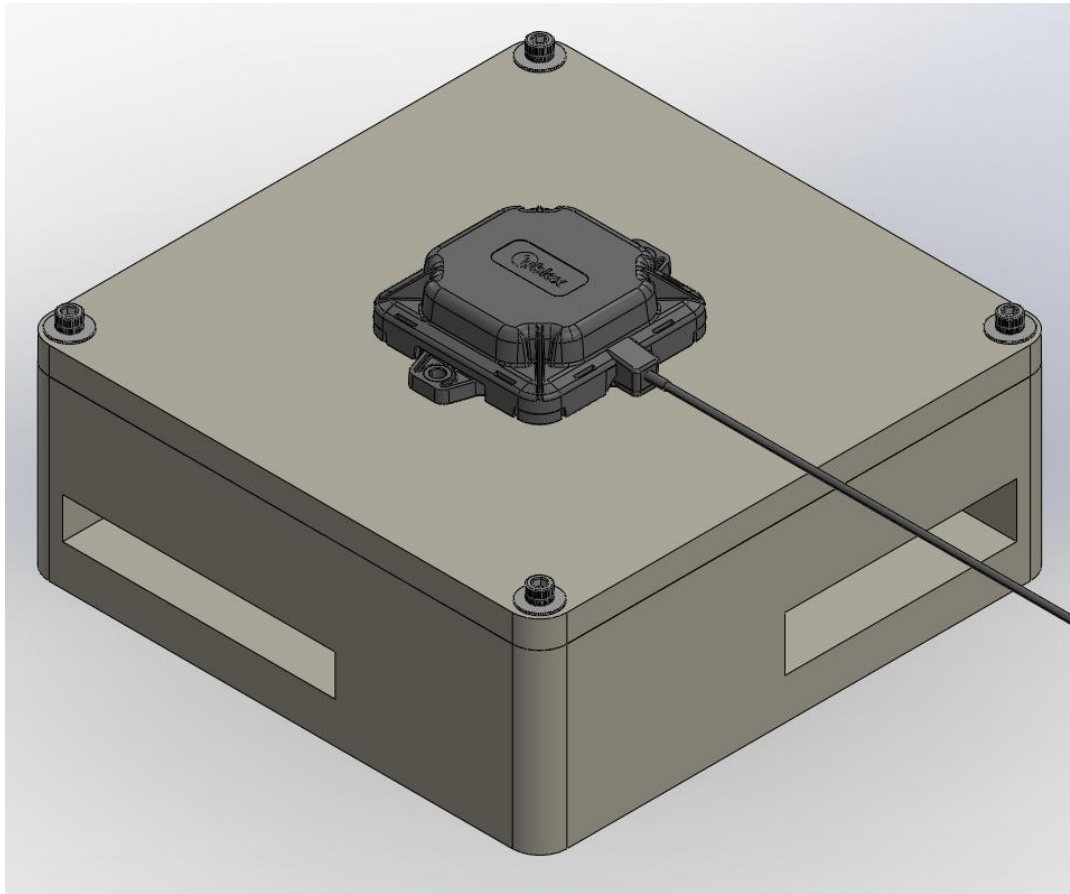


Figure 11.7: Design 2 Screwing Lid Enclosure (Closed)

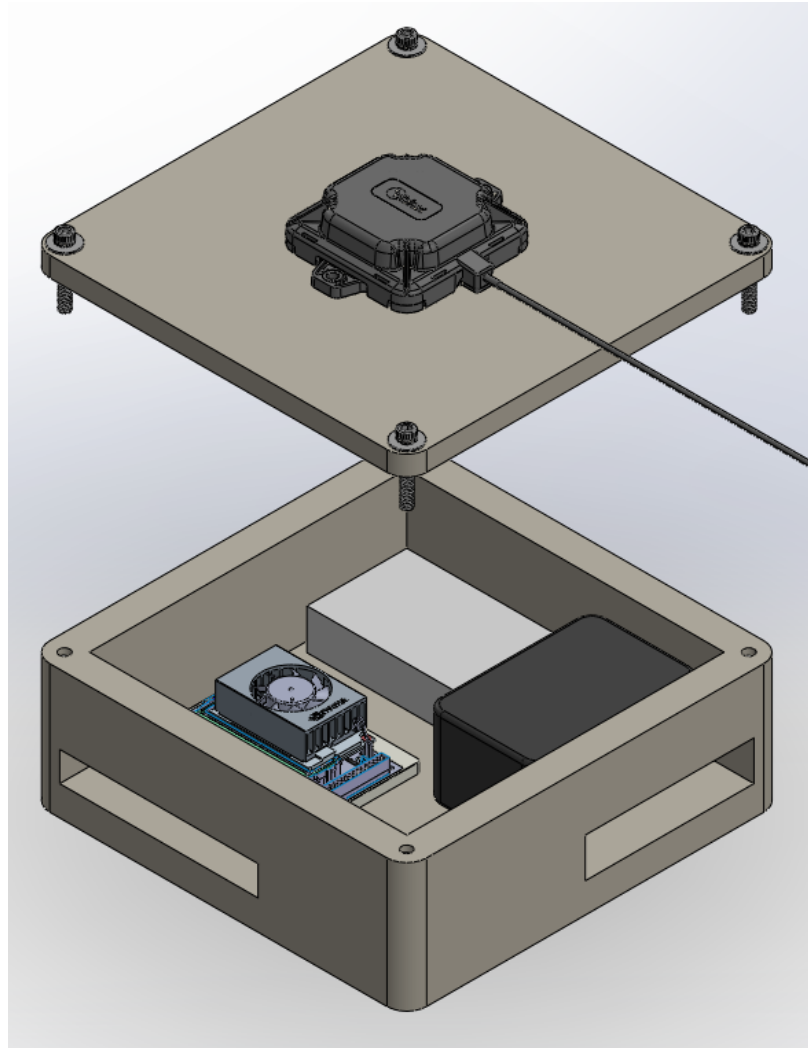


Figure 11.8: Design 2 Screwing Lid Enclosure (Open)

11c.iii. Materials

For our enclosure, we compared three common 3D printing materials: PLA (Polylactic Acid), ABS (Acrylonitrile Butadiene Styrene), and PETG (Polyethylene Terephthalate Glycol). These materials were chosen because they are widely used for mechanical parts and casings. The team compared these materials by conducting a trade study. The study comparison examined important factors such as printability, dimensional accuracy, impact resistance, cost, and environmental impact. The criteria that mattered most for our project were printability and accuracy, since the enclosure needs to fit other components precisely and be easy to print with consistent quality.

Table 11.4 below shows the trade study used to compare the three materials. Each property was given a weighting based on its importance to the design. The materials were rated from 1 (poor) to 5 (excellent), and the weighted scores were used to find the total score for each material.

Table 11.4: Enclosure Manufacturing Material Trade Study

		PLA		ABS		PETG	
Selection Criteria	Weight	Rating	Weighted Score	Rating	Weighted Score	Rating	Weighted Score
Printability	6	5	30	3	18	4	24
Dimensional Accuracy	5	5	25	3	15	4	20
Impact Resistance	3	2	6	5	15	4	12
Cost & Availability	3	5	15	4	12	4	12
Environmental Impact	3	5	15	1	3	3	9
Total Score		91		63		77	

From the results, PLA received the highest total score of 91, making it the best choice for our project. PLA is very easy to print, gives accurate and clean results, and is both affordable and biodegradable. It also doesn't warp or produce strong fumes like ABS. While ABS and PETG are tougher materials, they are harder to print and less environmentally friendly. Overall, PLA was selected as the material for our enclosure because it provides the best balance between print quality, cost, and ease of manufacturing for our design.

11c.iv. Manufacturing Processes

The enclosure for our system will be fabricated using 3D printing through the Fused Deposition Modeling (FDM) process. This method was chosen because it allowed for fast prototyping, accurate part geometry, and easy design changes compared to traditional subtractive manufacturing. The 3D model will be created in SolidWorks and exported as an STL file before being prepared for printing using Bambu Labs Software. Print settings such as layer height, infill, and support will be optimized to achieve a strong and reliable print. The enclosure will be printed using PLA Filament on a Bambu Labs X1 Carbon printer, with a layer height of .2mm, infill of 25%, and a nozzle temperature of 210 Celsius. A heated bed at 60 Celsius will be used to improve layer adhesion and prevent warping during the printing process.

After printing, the enclosure will be post-processed by removing supports and lightly sanding the surfaces for better fitting of components. The top and bottom halves will be assembled using machine screws and threaded inserts, allowing for easy disassembly during testing and maintenance processes. Overall, 3D printing will serve as an efficient and cost-effective manufacturing method. This will enable the team to quickly manage design iterations and produce a functional prototype tailored to the project's needs.

11d. Electronics Design

11d.i. Electrical Interfaces

The electrical interfaces used in our design are primarily concerned with powering sensors and ensuring data transfer is occurring between the sensors and the NVIDIA Jetson Nano compute unit. To power the IMU and GNSS receiver, power will be supplied via the USB ports on the Jetson. This will supply the required voltage (typically 3.3 or 5 V) for the IMU and GNSS receivers. Power will be supplied to the Jetson using an external battery to the barrel jack on the Jetson, which accepts between 9-20 V. The battery selection and power budget is further discussed in section 11.d.iii. Power System.

To power the Wi-Fi module and SSD used for internet connection and data storage, the M.2 slots on the Jetson Nano will be utilized. The SSD will be placed into the M.2-2280 slot, and the Wi-Fi card will be placed into the M.2-2230 slot. Both of these will provide adequate power to the components for use in our system.

Finally, the GNSS antenna used will be powered through the GNSS receiver, which is provided as an evaluation kit. This will supply the necessary voltage to power the passive antenna used to receive the GNSS signals.

To facilitate data transfer between the GNSS receiver and Jetson compute unit, UART over USB will be used. To facilitate data transfer between the IMU and Jetson, UART over USB will be used. Both of these provide high communication rates for our usage and should be acceptable to meet our requirements. For data transfer between the M.2 SSD, PCIe will be used, and PCIe will be used for the M.2 Wi-Fi card.

The battery chosen for our prototype (see Section 2:11d.iii11d.iii Power System) contains a built-in battery management system (BMS). This reduces the complexity required in our circuitry by automatically implementing protections such as over/under-voltage, current, and charge protections. This will allow the battery to function optimally and ensure the electronics used in the design are not damaged during use.

11d.ii. Electrical Components

The preliminary design focused on selecting electrical components that would provide valuable performance for the system while leaving room to demonstrate the usage of AI for improving inertial navigation. At this stage, the important parts to be selected include an IMU, GNSS receiver, GNSS antenna, and compute unit.

For the IMU selection, Kearfott provided our team with five models to pick from, and the selected one will be provided by them. Our team performed a trade study based on a variety of factors. The most important factors for the IMU selection include the bias instability specs, online documentation availability, and potential AI improvement (defined as the expected drift over time, demonstrating sufficient room for improvement with AI), among other factors. The team's

full IMU trade study is shown below in Table 11.5. As a result of the trade study, the team selected the Movella MTi-3 IMU. Kearfott has confirmed they will provide this IMU for prototyping, as well as the necessary adapter to interface this with an NVIDIA Jetson Nano.

Table 11.5: IMU Trade Study

		IMU1		IMU2		IMU3		IMU4		IMU5	
		Inertial Labs H100		IXSENS Polaris		Inertial Labs Kernel-110		Analog ADIS16465		Movella MTi-3-5A-DK	
Selection Criteria	Wt.	Value	Wt'd Score	Value	Wt'd Score	Value	Wt'd Score	Value	Wt'd Score	Value	Wt'd Score
Potential AI Improvement	8		8		8		24		24		32
Gyro Bias Instability	4	1 deg/hr	20	1 deg/hr	20	2 deg/hr	16	2 deg/hr	16	6 deg/hr	8
Accel Bias Instability	4	1.2 mu g	20	25 mu g	8	10-50 mu g	8	3.6 mu g	16	40 mu g	8
Availability of SDK	3		9		9		9		9		15
Online Documentation	4		12		4		12		16		20
Gyro Range	2	2000 deg/s	10	400 deg/s	6	2000 deg/s	10	125 deg/s	4	2000 deg/s	10
Accel Range	2	40 g	10	10 g	6	8 g	6	8 g	6	16 g	10
Power Consumption	3	3 W	3	2 W	6	0.5 W	12	0.2 W	12	< 0.1 W	15
Sampling Rate	2	2000 Hz	10	1000 Hz	8	2000 Hz	10	2000 Hz	10	1000 Hz	8
Total Score		102		75		107		113		126	

For the GNSS selection, the team looked for units that comprised a representative view of the market. This included looking at high and low-end models from different manufacturers. Because the team is purchasing this GNSS receiver out of the budget, price is a factor in the search for receivers. Additionally, the team prioritized looking for developer or evaluation kits, which have greater support and allow for plug-and-play functionality in our system. This is especially useful for prototyping to avoid custom PCB manufacturing for a GNSS receiver module. The most important factors for the GNSS trade study include online documentation, multi-band and constellation support, as well as ground-truth reliability, as the team must ensure that the training data obtained from the system is accurate to ensure the AI model is trained properly. The GNSS trade study is shown below in Table 11.6. As a result of the trade study, the u-blox EVK-X20P evaluation kit was selected, with a u-blox ZED-X20P receiver module internally. This module provides all-band, all-constellation support in a kit, and includes an antenna to receive the signal. Additionally, at \$299, the receiver provides great affordability for our project. This will be directly used with our prototype and is compatible with the NVIDIA Jetson Nano. Additionally, the update speed of 25 Hz means that the signal can be processed more frequently, increasing the accuracy of our system.

Table 11.6: GNSS Receiver Trade Study

		GNSS1		GNSS2		GNSS3		GNSS4		GNSS5	
		u-blox ZED-X20P (EVK-X20P)		u-blox ZED-F9P (EVK-F9P-01)		Juniper Geode GNS3		Quectel LG290P		mosaic-go X5	
Selection Criteria	Wt.	Value	Wt'd Score	Value	Wt'd Score	Value	Wt'd Score	Value	Wt'd Score	Value	Wt'd Score
Online Documentation	5		20		20		15		15		20
Power Consumption	2	~1 W	8	~1 W	8	~2 W	4	~0.3 W	10	~0.6 W	8
Availability of SDK	4		16		16		12		8		12
Multi-Band Support	4	All	20	Dual-band	8	All	20	Quad-band	16	All	20
Multi-Constellation Support	4	All	20	4	16	All	20	All	20	All	20
Size	1	105x64x26 mm	2	105x64x26 mm	2	110x110x57 mm	1	43.2 mm square	4	74x59x11.5 mm	3
Update/Sampling Rate	3	25 Hz	12	15 Hz	9	20 Hz	9	10-20 Hz	9	100 Hz	15
Interface	2	USB-C	8	USB-C	8	USB-C & Bluetooth	10	USB-C	8	USB-C	8
Ground-Truth Reliability	4	6cm PPP-RTK	20	<6 cm RTK	16	RTK: 1cm RMS	20	0.7 m CEP	8	0.6 cm RTK	20
Price (dev kit when available)	4	<u>\$299.00</u>	12	<u>\$291.38</u>	12	<u>\$1.995.00</u>	4	<u>\$189.85</u>	16	<u>\$748.55</u>	4
Total Score		138		115		115		114		130	

One additional feature of the u-blox family of GNSS receivers is the augmentation services available for increased ground-truth reliability without the purchase of an RTK correction rover-base system. U-blox offers *PointPerfect Flex* [17], a usage-based internet connection service. It provides 40 hours per month of free correction data, which is sufficient for collecting training and testing data for our system. The usage of *PointPerfect Flex* will increase the accuracy of the GNSS data to within 6 cm, increasing the data reliability and subsequently the AI model.

The compute unit selected is a Jetson Orin Nano developer kit. At a retail price of \$249 and a compact size, this provides a cost and size-effective solution for designing and prototyping. The Jetson compute unit provides sufficient computing power for deploying AI models on edge devices. Shown in Figure 11.9, the Jetson developer kit provides USB ports for interfacing with sensors, providing an efficient method to test and prototype with IMUs and GNSS receivers. The USB ports will also be used to power the IMU and GNSS receiver as shown in the hardware architecture previously discussed in Figure 11.2. As mentioned in the Electrical Interfaces section, the Jetson will interface with an M.2 SSD and Wi-Fi card to provide storage and internet connectivity.



Figure 11.9: NVIDIA Jetson Orin Nano Developer Kit [18]

Attached to the Jetson Orin Nano will be a M.2 Wi-Fi card as well as a M.2 NVMe SSD. For this, an Intel AX200 Wi-Fi card will be attached to the M.2-2230 port of the Jetson Nano. A Crucial P310 1TB SSD (or similar) will be attached to the M.2-2280 port of the Jetson Nano. This provides Wi-Fi connectivity and sufficient storage space for the system to record data during testing runs, as well as the required storage to hold the finished AI model onboard.

11d.iii. Power System

The power system of our design is important to ensure all components work properly. To do this, our team analyzed the power requirements of all components in our system and worked to select a battery to power the system while deployed. This is shown below in Table 11.7.

Table 11.7: Power Requirements of Each Component

Component	Power Requirement
NVIDIA Jetson Orin Nano Developer Kit	9-20 V, <25 W
SSD (Crucial P310 1 TB)	~3-4 W typical; 8.5 W max
GNSS Receiver Module (u-blox EVK-X20P)	<1 W
GNSS Antenna (u-blox ANN-MB2)	~0.1 W typical
IMU (Movella MTi-3-5A)	~0.5-1 W
Wi-Fi Card (Intel M.2 AX200)	~1.6 W
Maximum Power Requirement	37.2 W

For the prototype, our team valued ease of integration heavily when selecting a battery. Because everything can be powered from the Jetson Orin Nano Devkit (via USB ports) our team focused on finding a battery capable of powering the Jetson directly. Our team wanted to avoid sensitive batteries such as hobbyist lipo batteries due to their sensitive storage conditions and charging

considerations. These batteries also typically output in either XT90/JST connectors, which requires a conversion to the DC barrel jack accepted on the Jetson. Our team then worked to find batteries with either USB-C Power Delivery (PD) or direct DC ports to power the Jetson and all of its components.

Ultimately, our team selected a V Mount battery to power the system. While these batteries are typically used for photo/videography, they provide barrel jack DC outputs in a compact form factor. They provide rugged performance in all conditions, one of our key considerations throughout other aspects of the project, and are designed to be moving during usage. As such, they work well in the scenario for our prototype. Our team selected the SMALLRIG V Mount 99Wh battery for use in the prototype. This battery will deliver a system runtime that allows for deployment in prolonged scenarios. With a 99Wh capacity, the battery should comfortably provide ~2.5-3 hours of runtime under the maximal load of our system, with increased runtime with less strenuous computing requirements.

Additionally, the V Mount battery provides a USB-C charging interface and does not require users to purchase a sophisticated charging setup. The battery also has an onboard battery management system, with over-discharge, over-temperature, short circuit, over-load, over-voltage, and over-current protections provided out-of-the-box. This allows users to get set up with our product easily and with minimal considerations for battery health and safety. The V Mount battery is shown in Figure 11.10.



Figure 11.10: SMALLRIG VB99 V Mount Battery [19]

11e. Software Design

11e.i. Algorithms

The navigation software implements a hybrid architecture fusing classical estimation with machine learning based drift correction. Its core components operate within a real time control loop at 100 Hz to ensure deterministic response on the embedded platform. The components are detailed as follows.

1. **Extended/Adaptive Kalman Filter:** The Extended Kalman Filter propagates vehicle position, velocity, and attitude using inertial data during the prediction phase and performs measurement updates when GNSS data is available. The Adaptive Kalman Filter, another possible option, adaptively tunes process noise covariance when there is sensor noise or dynamic change.
2. **Machine Learning Drift Correction Model:** A GRU network trained on historical IMU sequences predicts time-varying gyroscope and accelerometer bias. The output bias vector is injected into the EKF prediction step to constrain long-term error growth during GPS denied operation.
3. **Mode Manager Logic:** The mode manager governs transitions among GPS-active and GPS-denied states according to GNSS quality indicators such as fix count, signal to noise ratio, and system health flags. This logic ensures smooth state re-entry without discontinuities in the navigation solution.

11e.ii. Modules, Classes, and Methods

Our software will require modules, classes, and methods as follows.

11.e.ii.1 Modules

To uphold readability as well as maintainability, the code must be organized in a modular way. The classes and functions will be organized as follows:

train.py: trains the model.

imu_dataset.py: builds causal training windows and supplies runtime windows

- **IMUWindowDataset**
- **CausalWindowProvider**

gru_net.py: the Gru that outputs displacement over a short horizon

- **GRUNet**

error_state_akf.py: the Adaptive error-state Kalman filter.

- **ErrorStateEKF**

outage_manager.py: switches measurement source (eg. GNSS to Gru) and aligns horizons.

- **OutageManager**

metrics.py: calculates rmse and max_err

evaluator.py: evaluates the model on a dataset

utils.py: stores helper functions

11.e.ii.2 Classes

NavState: This class will store all the navigation state values.

INSMechanization: This class performs strapdown inertial navigation by utilizing IMU data to propagate the normal navigation state forward in time. In short, it updates all navigation system values.

Methods:

- **propagate:** handles the integration of all IMU data values

ErrorStateAKF: is an adaptive Kalman filter formulated in the error state space. Its state will contain position/velocity errors, small attitude errors, and IMU bias terms. When GNSS is present it uses GNSS position as measurements and during GNSS outages, it instead uses GRU-generated pseudo-measurements that act as a replacement for GNSS values.

- **pred_update:** returns the predicted error state and covariance
- **meas_update:** uses sensor readings to pull the predicted error state back towards reality and shrinks uncertainty.

GRUNet: This class will define the GRU model

- **forward:** takes a window of IMU data and outputs a 3-D displacement vector that allows predicted GNSS values to be calculated.

IMUWindowDataset: This class builds casual training windows.

- **getitem:** builds the input window and the target for the window

CausalWindowProvider: This class slices a casual input window from a growing IMU time series. This is used for inference/deployment.

- **get_window:** returns a tensor containing the samples in a given window size

OutageManager: Switches measurement source and aligns horizons

- **mode:** sets the mode of the GNSS “measurement” (gnss/gru)

- **next_update_time:** aligns the pseudo-measurement's timestamp

11e.iii. Application Programming Interfaces

A separate application programming interface is not required for this system since the software components operate locally on the embedded device.

11e.iv. Coding/Implementation Details

The implementation of the machine learning model pipeline will be done mainly using python on either VS Code or MATLAB. The code will use libraries, such as Pytorch, TensorFlow, Pandas, Scikit-Learn and NumPy to create, train, and test the machine learning model. The model will then be deployed onto the jetson nano and evaluated through the metrics it saves.

12. Design Evaluation Methods

12a. Analysis Cases and Results

Evaluating design methods is a crucial step in any engineering problem, and it begins with analysis. The team performed three analysis cases: drift for a stationary IMU, IMU simulations for a vehicle following a trajectory, and an algorithmic approach to drift reduction based on reference data. The first two analyses demonstrate that unaided inertial navigation with specifications commonly found in MEMS IMUs is unreliable, especially over long periods of time. Alternatively, the third analysis investigates the impact of a solution, examining how a drift-correction algorithm can improve IMU performance. Overall, these analyses provide an approximation of expected behavior and are theoretical as they are not based on measured data from the team's system.

First, the stationary IMU case can be examined over a short term to determine the performance of a MEMS IMU with these specifications, without any motion involved. High performance is represented by minimal position drift. This means the gyroscope yields angular velocities near zero, and the accelerometer yields approximately $9.8m/s^2(1g)$ upward along the axis normal to the Earth's surface due to its gravitational pull, and no acceleration along the remaining axes. For a MEMS IMU, the team expects to have drift. The team developed a MATLAB script to simulate the drift based on IMU parameters that mimic the Movella MTi-3 specs. The parameters are shown below in Table 12.1.

Table 12.1: Case 1 IMU Drift Simulation Parameters

Error Source	Gyroscope	Accelerometer	Source
In-run bias stability	6 deg/hr	40 μg	MTi-3 datasheet
Noise density	0.003deg/s/ $\sqrt{Hz}$	70 $\mu g/\sqrt{Hz}$	MTi-3 datasheet
Bias random walk coefficient	$2.91 \times 10^{-5} rad/s/\sqrt{Hz}$	$3.92 \times 10^{-4} m/s^2/\sqrt{Hz}$	Approximated from in-run bias stability
Initial bias	[0.001, -0.0005, 0.0008] rad/s	[3.9e-4, -2e-4, 3e-4] m/s	Representative (not on datasheet)
Scale factor	0	0	Assumed negligible

With the IMU parameters defined, a program was developed to achieve the IMU measurements with errors over 60 seconds by summing each source with the real value for both the gyroscope and accelerometer. Then, the measurements at each time step were integrated to get the orientation and position. The results were plotted to showcase the position drift over the short period with respect to each coordinate axis. An additional plot was created to visualize the magnitude of this drift. Both plots are shown below in Figure 12.1.

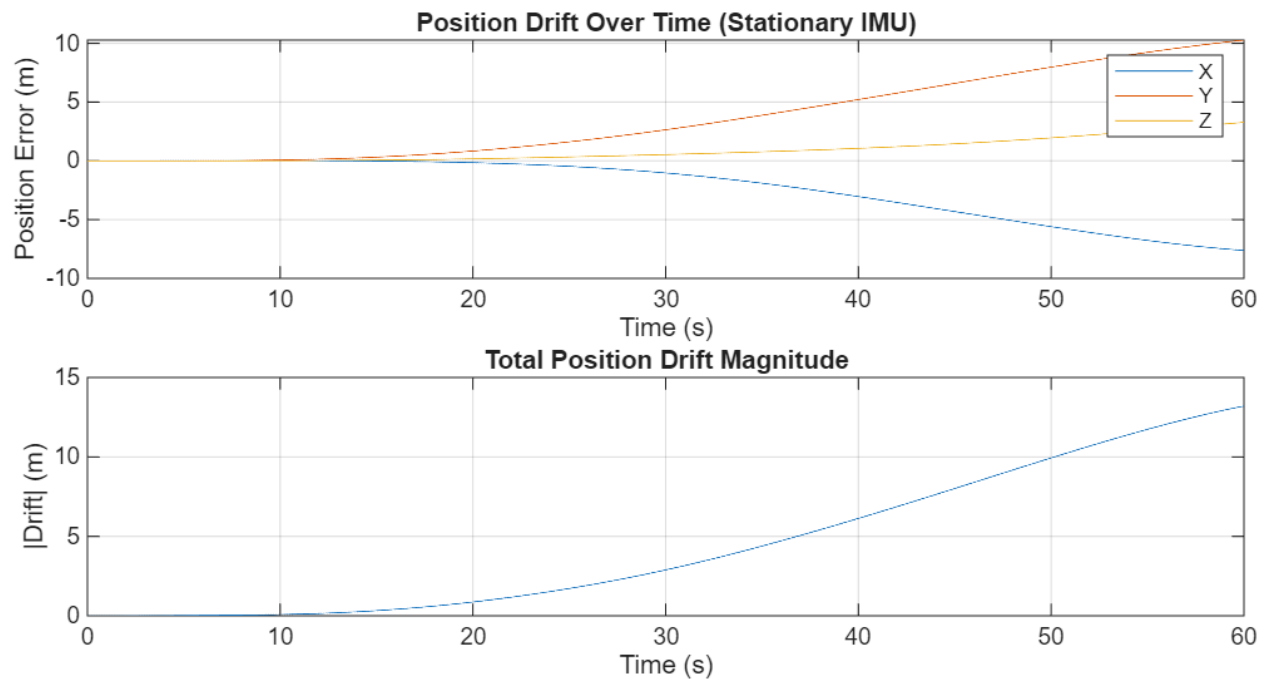


Figure 12.1: Case 1 MATLAB IMU Drift Simulation Results

As shown, over a short 60-second interval, the simulation suggests a magnitude drift of approximately 10-15 meters even when the IMU is stationary. This suggests how the Movella MTi-3 is unreliable when unaided and expresses the need for a drift-correction method.

In analysis case 2, the Movella MTi-3 IMU specifications are used to simulate IMU readings over a test trajectory on Stevens' campus. The main objective is to develop a simulation that can be referenced during prototype testing to determine if our IMU data is acceptable while navigating a similar trajectory as the simulation.

Using MATLAB, a test trajectory and vehicle driving scenario are created using the Automated Driving Toolbox and Navigation Toolbox. To generate this code, MATLAB sample code was modified to fit the needs of our simulation. Specifically, a Mathworks sample code '*Simulate Inertial Sensor Readings from a Driving Scenario*' [20] was used. To modify this code, adjustments were made, such as the waypoints provided as well as the starting GPS location, given in LLA format (latitude, longitude, altitude). The code used can be found in Appendix D. Additionally, the IMU specifications were set to the available specifications of the MTi-3 as mentioned above. Below, Figure 12.2 shows the trajectory taken by the simulated vehicle.

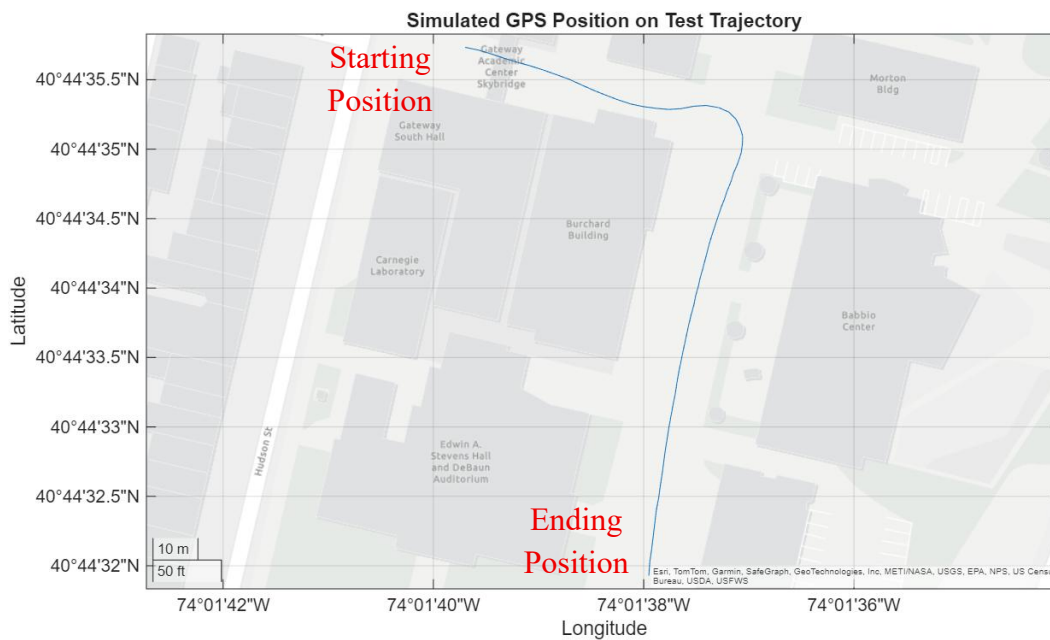


Figure 12.2: Case 2 Simulated Trajectory in MATLAB

Next, the accelerometer and gyroscope data of the IMU were simulated in MATLAB. This was done through the use of the *imu* object, which takes inputs of acceleration, angular velocity, and orientation of the vehicle based on the trajectory. The *imu* object accounts for properties of the IMU, such as bias instability, measurement range, and noise density, and outputs a predicted IMU reading for the accelerometer and gyroscope. Because the simulation is specific to a ground vehicle, the z-direction acceleration is not relevant, and the x- and y-directions of rotation are not relevant. As such, they are excluded from the analysis below. Figure 12.3 and Figure 12.4 show the simulated accelerometer and gyroscope readings, respectively.

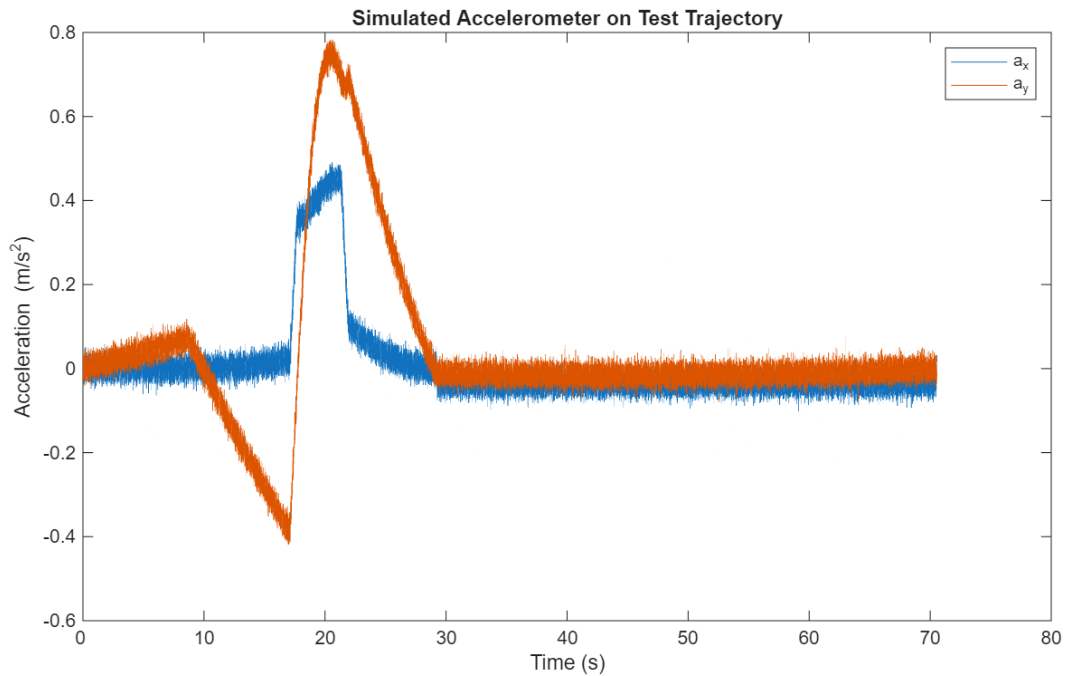


Figure 12.3: Case 2 Simulated Accelerometer Data on Test Trajectory

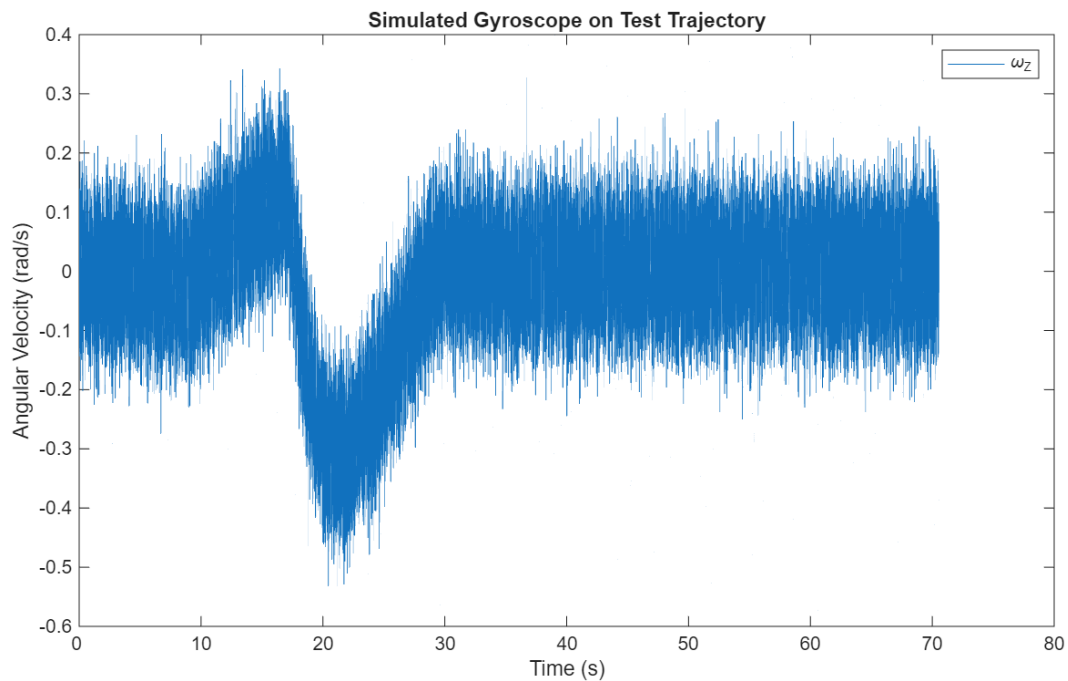


Figure 12.4: Case 2 Simulated Gyroscope Data on Test Trajectory

As shown, there is significant noise in both acceleration and angular velocity data gathered from the simulated IMU. This demonstrates the need for AI-enhanced inertial navigation that will be

performed in our system. Due to the noise, the positional estimates found in dead reckoning algorithms are subject to error, which becomes significant quickly as errors compound.

Finally, the last portion of this analysis case simulates the GPS velocities. Because the GPS updates infrequently compared to the IMU, the data has much less fidelity. This is shown in the graph. Similar to the acceleration of the IMU, the GPS velocities are only relevant in the x- and y-directions, and the z-direction velocity will be omitted from the analysis for simplicity. The simulated GPS velocities (x and y-directions) are shown in Figure 12.5. In the figure, the velocity is consistent with the expected path. The vehicle's trajectory is mostly horizontal at the beginning before turning and shifting to mostly vertical velocity. Because the velocity is downwards on the map (see Figure 12.2), the y-velocity is negative.

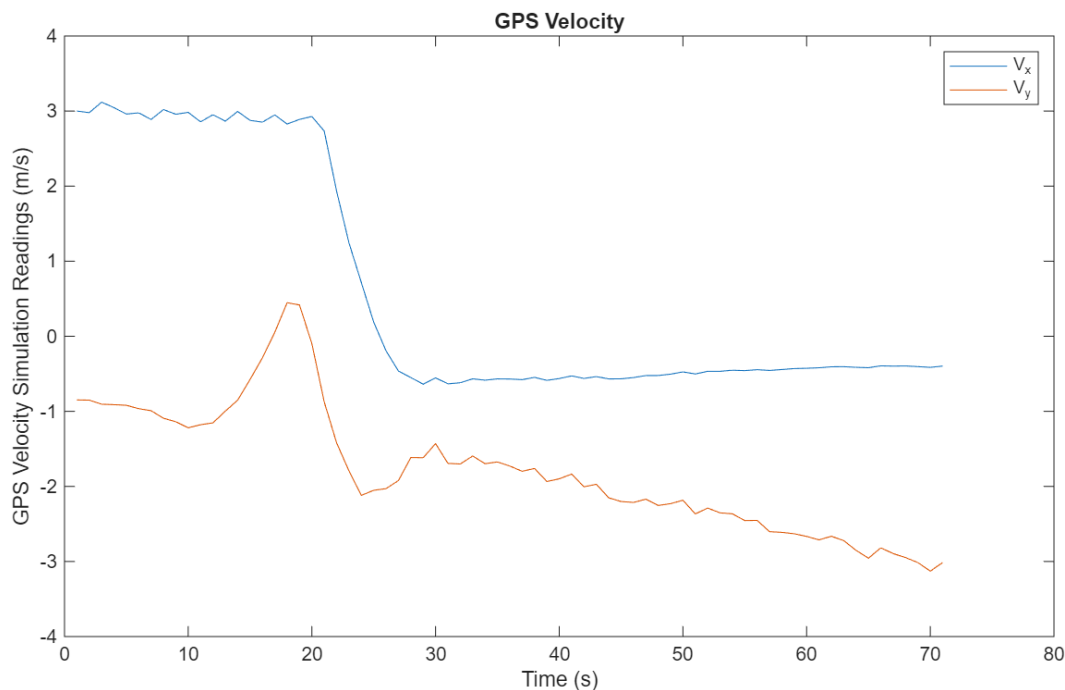


Figure 12.5: Case 2 Simulated GPS Velocities on Test Trajectory

In analysis case 3, the team investigated software methods for drift correction. Because our team doesn't currently have the means to test several fully trained models, we looked to academia for benchmarking instead. Metrics from two different papers were used in our analysis of software design.

Table 12.2: Performance Data of AI-Aided GNSS/INS Integration Models [21]

Metric	Stat	RBF	SINS	LSTM	PSO-LSTM
Easterly vel (m/s)	MAX	3.611	9.932	1.569	1.124
Easterly vel (m/s)	RMSE	1.423	5.218	0.639	0.454
Northbound vel (m/s)	MAX	3.033	9.789	2.184	1.493
Northbound vel (m/s)	RMSE	1.459	5.671	0.822	0.591
Latitude (m)	MAX	6.157	243.828	3.881	2.676
Latitude (m)	RMSE	1.798	115.849	1.491	1.065
Longitude (m)	MAX	6.4	239.291	2.543	1.863
Longitude (m)	RMSE	1.839	103.076	0.905	0.689

The data in Table 11.2 comes from a study that conducted an on-road vehicle test where IMU output was fused with high-precision dual antenna GNSS to provide ground truth [21]. The models were evaluated and summarized with MAX and RMSE of the component errors: latitude error (m), longitude error (m), and North/East velocity error (m/s). The MAX shows us the worst-case scenario, while RMSE shows us average performance. Within this study, PSO-LSTM was shown to outperform the rest of the models in the study, with very low errors, which makes it a good candidate for our software design. However, a drawback is that these metrics were the results of short GNSS outage tests. The models in this study were only tested for 30-60 seconds.

Table 12.3: Performance Data of GRU and AKF Hybrid Algorithm Compared to Other Models [22]

Algorithm	Max North (m)	Max East (m)	Max Horizontal (m)	Horizontal RMS (m)
Not-aided KF	109.639	105.48	152.141	64.258
MLP-aided KF	28.513	30.177	53.285	27.342
LSTM-aided KF	19.99	26.512	30.291	17.294
GRU-aided KF	21.227	26.467	31.532	18.015
GRU-aided AKF	21.331	20.544	26.45	15.813

The data in Table 12.3 comes from a similar study that also conducted an on-road vehicle; however, it explicitly simulates long GNSS blackouts (≥ 120 s) in the test window [22]. The metrics they captured were **Max North** and **Max East** (worst component errors in meters), plus **Max Horizontal** and **Horizontal RMS** (the 2-D norm $\sqrt{(\text{North}^2 + \text{East}^2)}$, reported as worst-case and RMS). In this study, GRU-aided AKF performed the best with the lowest amount of error.

Table 12.4: Combined Performance Data of Both Studies

Model	Horizontal RMS (m)	Max Horizontal (m)
RBF	2.572	8.881
SINS	155.067	341.631
LSTM	1.744	4.64
PSO-LSTM	1.268	3.261
GRU-aided AKF (120s outage)	15.813	26.45

In order to compare the two studies, **Horizontal RMS** and **Max Horizontal** error were calculated where needed. Horizontal metrics matter most for navigation because they capture the **total planar miss distance** rather than a single axis. In table 12.5, while PSO-LSTM seems to perform the best when comparing both horizontal RMS and max horizontal errors, the testing conditions need to be considered. Because GRU-aided AKF was tested in greater than 120s intervals, which is more prevalent to our project and performed the greatest in its study, we've decided to move on with it as our main solution.

12b. Impact on System Design

Through the analyses performed, there are valuable insights that can be taken to improve the team's system design. Case 1 demonstrated that a stationary IMU that is representative of the team's selected Movella MTi-3 is subject to drift up to a magnitude of approximately 10-15 meters. Similarly, case 2 shows how a trajectory IMU simulation based on MTi-3 parameters yields noise in the acceleration and angular velocity, which leads to error build-up in dead reckoning. As these two analyses are theoretical, they will be used as a reference in the future during the testing phase. The real data from the team's system will be compared to them for validation of system operation. Finally, case 3 showed that while some metrics may seem more enticing than others, testing conditions must be regarded to the highest degree. While LSTM-PSO seems to perform better than GRU-aided AKF on numbers alone, the testing conditions show that GRU-aided AKF is the clear winner.

12c. Testing Considerations

We have further narrowed down our testing scenarios from the concept generation phase into the preliminary design phase. Our team has decided to use an RC car to perform testing of the system, as it provides real-world applicability and straightforward testing operation to test the performance of our system. While we are still determining which RC car will be used, our system will be mounted on top of the RC car chassis using a 3-D designed and printed plate for mounting. This will allow any car to be used, providing modularity on the RC car. This will also allow the Kearfott van to be used during testing if desired, as well as other platforms.

13. Alpha Prototyping: Plan and Budget

13a. Hardware Prototyping Plan

To demonstrate the key functionality of our hardware model, we integrated all components on a testing platform. The platform included the NVIDIA Jetson Orin Nano, MTi-3 IMU, u-blox EVK-X20P GNSS Receiver, and Crucial P1 SSD for storage. There was considerable setup involved to interface the Jetson Nano with the sensors, including installing drivers and setting up communication protocols between the components. This required the team to spend a non-trivial amount of time integrating all components successfully, which delayed the evaluation portion of the alpha prototyping phase.

Once all components were integrated, the team 3D printed a temporary testing platform made of PLA with 50% infill to collect small amounts of data from the IMU and GNSS receiver. While this platform is not practical for rigorous testing, it is a great initial design iteration as all components are mounted rigidly and easily accessible for tuning. This platform enables easy data collection for short trials and will help the team ensure the sensors are performing as expected. The physical prototype platform is shown below in Figure 13.1.

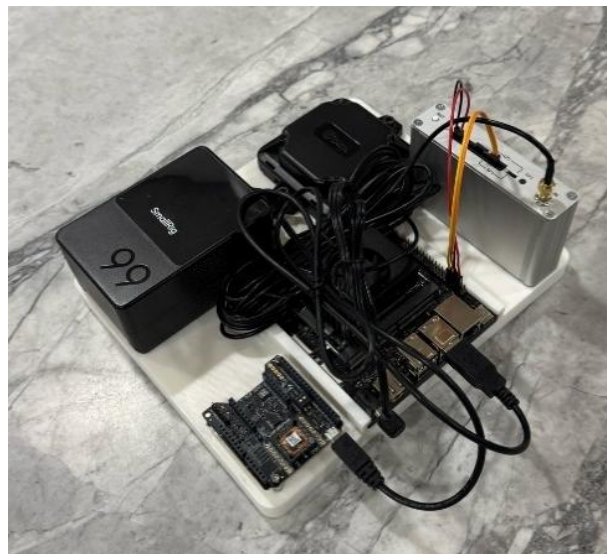


Figure 13.1: Alpha Prototype Platform (isometric view)

Furthermore, a wiring diagram was created to demonstrate communication between the sensors and the Jetson. The wiring diagram below in Figure 13.2 showcases the connections between all components used in the alpha prototype. The Jetson is central to all connections. The GPS receiver (ZED-X20P) is connected via 4 wires, with two for power and two for data transfer. The IMU (MTi-3) is connected via USB to the Jetson and Micro-USB to the IMU, and the Jetson is powered from a 12 V DC-DC barrel jack cable from our battery pack.

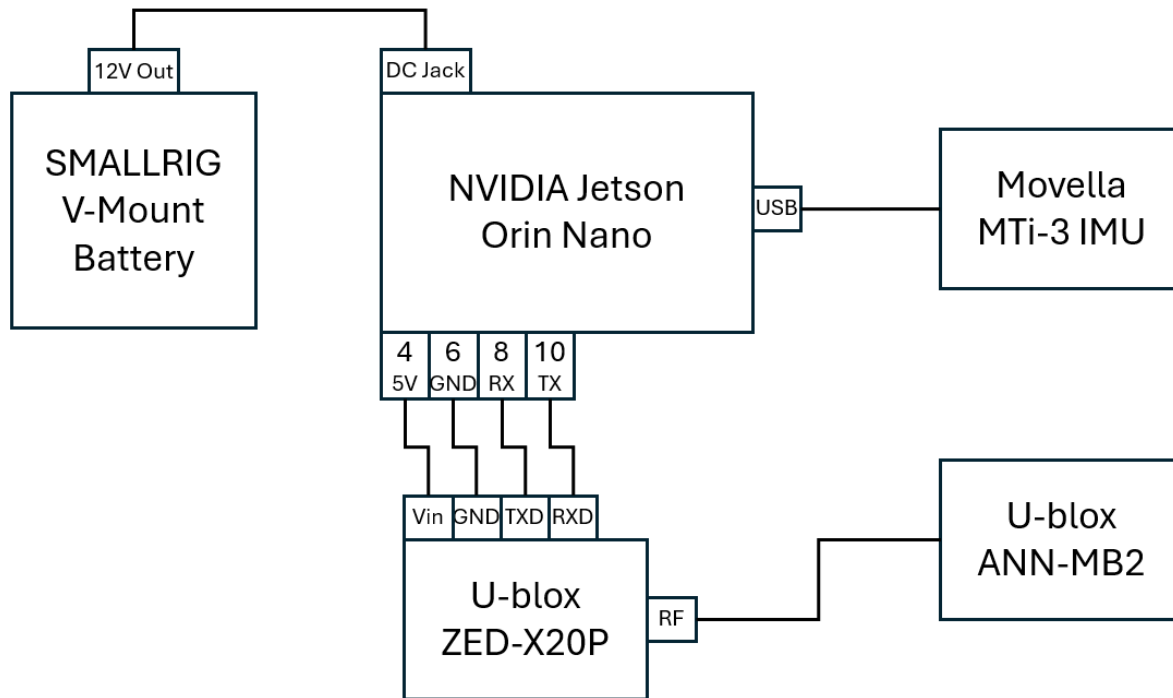


Figure 13.2: Wiring Diagram of Alpha Prototype

To aid the hardware prototyping process, signal filtering and processing were explored at the alpha prototype stage. In addition to generating simulated IMU data for the development of the AI drift correction improvement models, Kalman filtering was explored for fusing IMU and GPS data together, and a digital filter was explored as a vibration isolation system for the IMU data.

The IMU data simulation was improved and refined to better reflect the real-world performance of the IMU and GPS sensors. This includes adjusting the update rate of the IMU to 50 Hz, and the GPS receiver to 5 Hz, which better reflects the real-world performance of both sensors. In addition, new trajectories were generated, including a square path that repeats 10 times. This generated approximately 5,000 seconds (just under 1.5 hours) of data to train machine learning models with. Also of significance, a straight-line path was generated with approximately 70 seconds of data, which allows the software team to accurately assess the performance of the model on a low-complexity vehicle trajectory. A snapshot of the data format is shown below in Figure 13.3 to reflect these changes.

Time	acc_x	acc_y	acc_z	gyro_x	gyro_y	gyro_z	lat	long	alt
0	-0.00691	0.001424	-9.81065	0.003865	-0.00305	0.001668	40.74312	-74.027	52.89838
0.02	-0.0252	0.002061	-9.81049	0.01482	0.012173	0.003587	NaN	NaN	NaN
0.04	-0.03583	-0.00068	-9.808	0.000825	-0.02107	0.001386	NaN	NaN	NaN
0.06	-0.04984	-0.00216	-9.812	-0.01438	0.012203	0.003599	NaN	NaN	NaN
0.08	-0.05871	0.000515	-9.80618	0.015817	-0.0223	-0.00077	NaN	NaN	NaN
0.1	-0.06458	-0.00021	-9.81101	0.027779	-0.00187	-0.0086	NaN	NaN	NaN
0.12	-0.08366	0.001987	-9.80784	0.006868	0.01138	-0.00142	NaN	NaN	NaN
0.14	-0.09458	0.000135	-9.80397	0.008996	0.020909	0.001248	NaN	NaN	NaN
0.16	-0.10627	-0.00521	-9.81415	0.017965	0.002772	-0.00266	NaN	NaN	NaN
0.18	-0.11694	-0.00469	-9.81105	-0.00445	-0.01767	-0.0167	NaN	NaN	NaN
0.2	-0.13202	-0.00182	-9.80568	-0.04045	-0.00907	0.012423	40.74312	-74.027	52.66489
0.22	-0.14384	0.002558	-9.80756	0.012465	0.022776	-0.00676	NaN	NaN	NaN
0.24	-0.16078	0.005661	-9.80575	-0.01407	0.009788	-0.0206	NaN	NaN	NaN
0.26	-0.16827	0.000907	-9.80624	0.000522	-0.01028	-0.00785	NaN	NaN	NaN
0.28	-0.18165	0.000574	-9.80905	-0.01342	0.022864	0.016951	NaN	NaN	NaN
0.3	-0.19198	0.003888	-9.81506	-0.00932	-0.00564	-0.02064	NaN	NaN	NaN
0.32	-0.19886	0.001909	-9.81169	-0.01242	-0.01049	0.01455	NaN	NaN	NaN
0.34	-0.21704	-0.00236	-9.80843	-0.01673	-0.00332	-0.0061	NaN	NaN	NaN
0.36	-0.22758	-0.00266	-9.81212	-0.00634	-0.0057	-0.001174	NaN	NaN	NaN

Figure 13.3: Updated Trajectory Simulation Script Data Output

Kalman filtering was explored using the simulated IMU data in MATLAB. Mathworks provides inertial sensor filtering resources [23], which implement various types of Kalman filters on IMU data. For our system, we chose to use a non-holonomic INS filter, which takes sensor inputs from a gyroscope, accelerometer, and GPS position data (when available) and fuses them into a state estimation. The non-holonomic filter is appropriate for our system because we anticipate deploying the prototype and future iterations on robots that are constrained to non-holonomic movements, at least for the beta prototype stage forthcoming.

When implementing the Kalman filter in MATLAB, the straight line trajectory was used as the main performance metric, specifically analyzing the drift of the predicted trajectory to the ground truth obtained by the GPS. The Kalman filter provided decent results when the GPS was active, but was not adequate when the GPS was inactive. However, the filter only provided acceptable results when it was told to trust the GPS significantly more than the IMU, which attempts to “lock” the predicted trajectory onto the GPS trajectory when available. Two simulated implementations of the Kalman filter for no GPS-denial and partial GPS-denial cases are shown in Figure 13.4 and Figure 13.5 below.

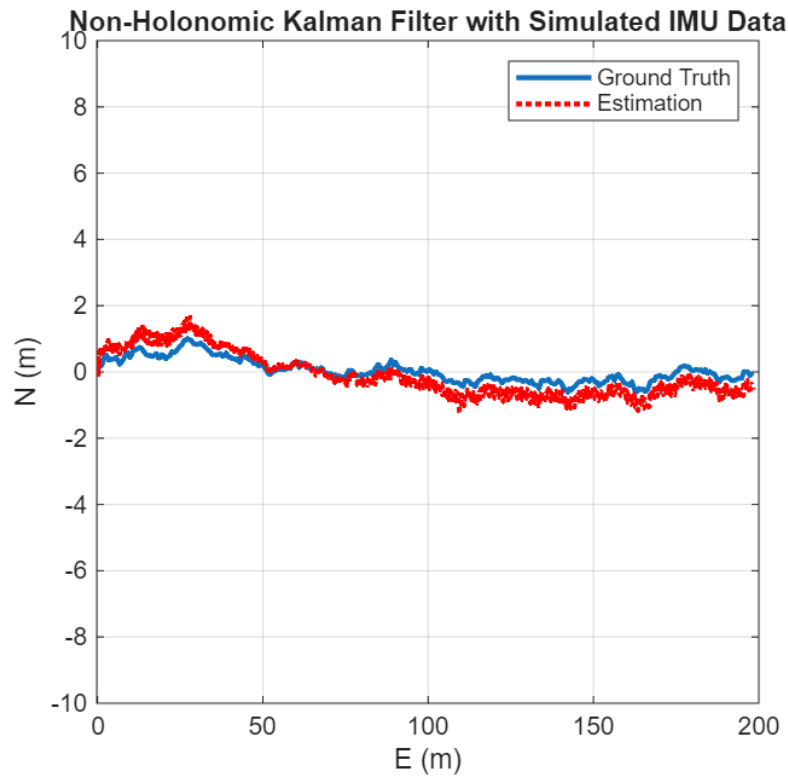


Figure 13.4: MATLAB Kalman Filter Implemented on Simulated Data (No GPS Denial)

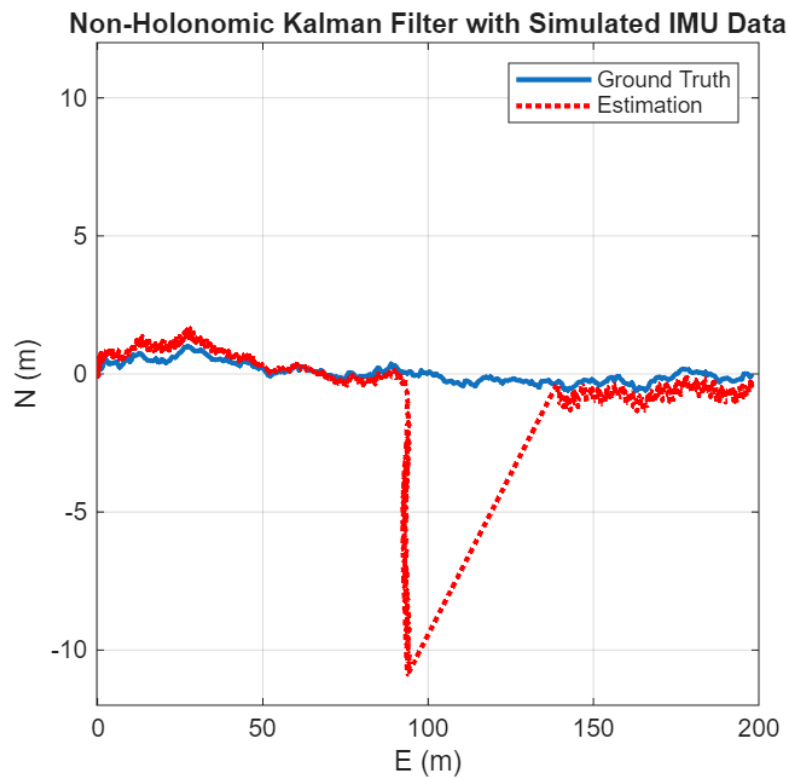


Figure 13.5: MATLAB Kalman Filter Implemented on Simulated Data (Partial GPS Denial)

The Kalman filter was effective in adhering to the trajectory while the GPS was present. However, once there is GPS denial, the Kalman filter's estimation goes off-track and does not recover until the GPS returns. The performance of the Kalman filter will be further investigated while working towards the beta prototype, and potentially implemented in the design before the AI drift correction takes place.

The application and integration of digital filters were also explored at the alpha prototype stage. Specifically, accelerometers are prone to noise via vibrations that can degrade the integrity of the system. While we are unsure of the specific vibration and their frequencies that will be experienced during the prototype testing plan, the important motion of the system should occur at low frequencies, such as a ground vehicle accelerating or decelerating. Thus, a low-pass digital filter was explored to eliminate vibration noise in the IMU data. A 4th-order Butterworth low-pass filter was implemented in Python to explore the results of digital filtering of the IMU signal. The cutoff frequency of this filter was parametrically analyzed at multiple frequencies, including 2, 5, and 10 Hz. A noisy IMU signal was recorded by taking the MTi-3 IMU and moving it dynamically in a circular motion, then placing it on a table and mechanically exciting it with vibrations, through impulse taps on the surface the IMU was sitting, while holding the IMU stationary. Figure 13.6 shows the raw IMU data in this scenario. Figure 13.7 - Figure 13.9 show the raw IMU data as well as the filtered data with cutoff frequencies 2, 5, and 10 Hz. The results show that much of the vibrations captured by the IMU are filtered out, while the dynamic portion of the signal is passed through the filter. In the 5 and 10 Hz cutoff frequencies, most of the dynamic portion of the signal is passed through. In the 2 and 5 Hz cutoff frequencies, most of the static vibration part of the signal is attenuated. However, the 10 Hz cutoff frequency fails to attenuate a sizable amount of vibration motion, and the 2 Hz cutoff frequency attenuates a sizable amount of the dynamic motion. Therefore, to ensure the integrity of the data, 5 Hz cutoff frequency gives data most faithful to the testing scenario. With respect to the beta prototype, we will strongly consider pre-processing accelerometer data using this low-pass filter when integrating all subsystems together.

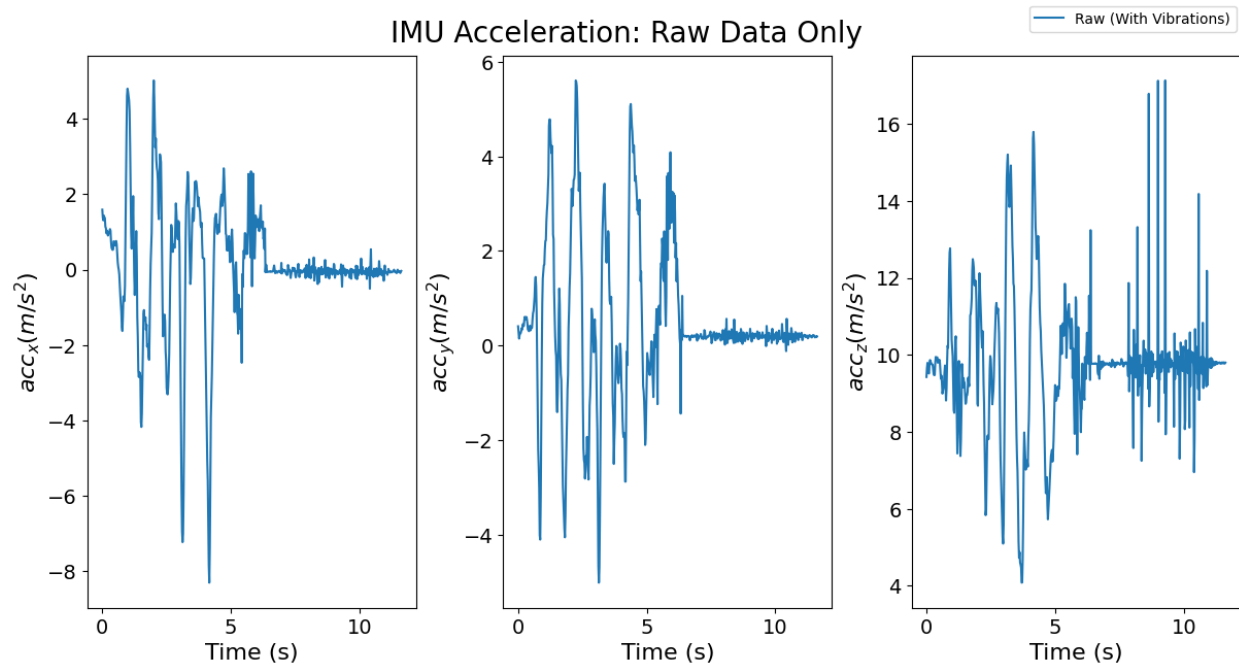


Figure 13.6: Raw IMU Acceleration Data in X, Y, and Z Axes

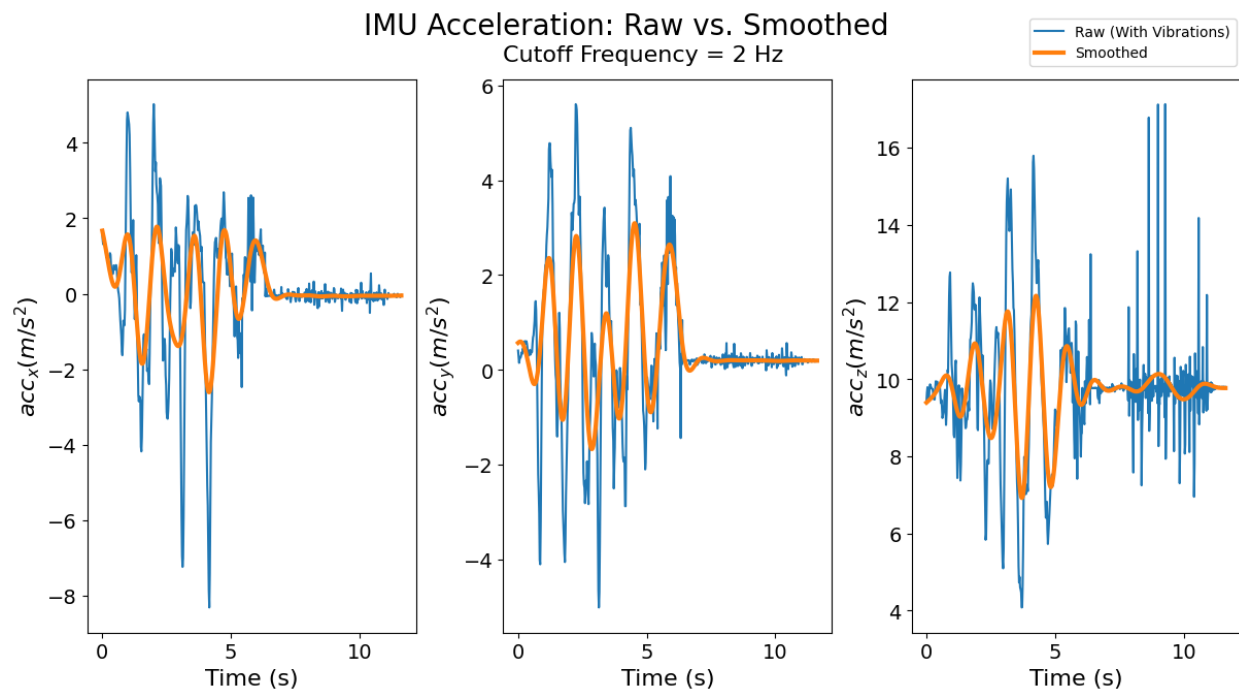


Figure 13.7: IMU Dynamic and Static Motion with Cutoff Frequency 2 Hz

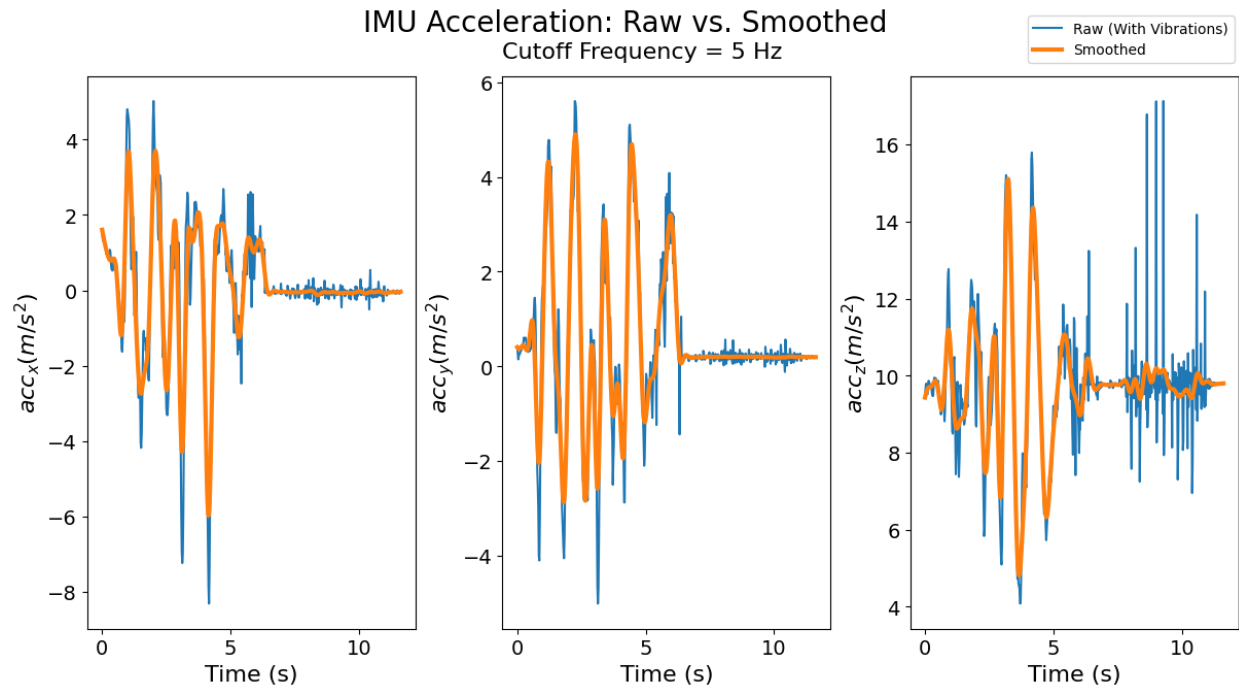


Figure 13.8: IMU Dynamic and Static Motion with Cutoff Frequency 5 Hz

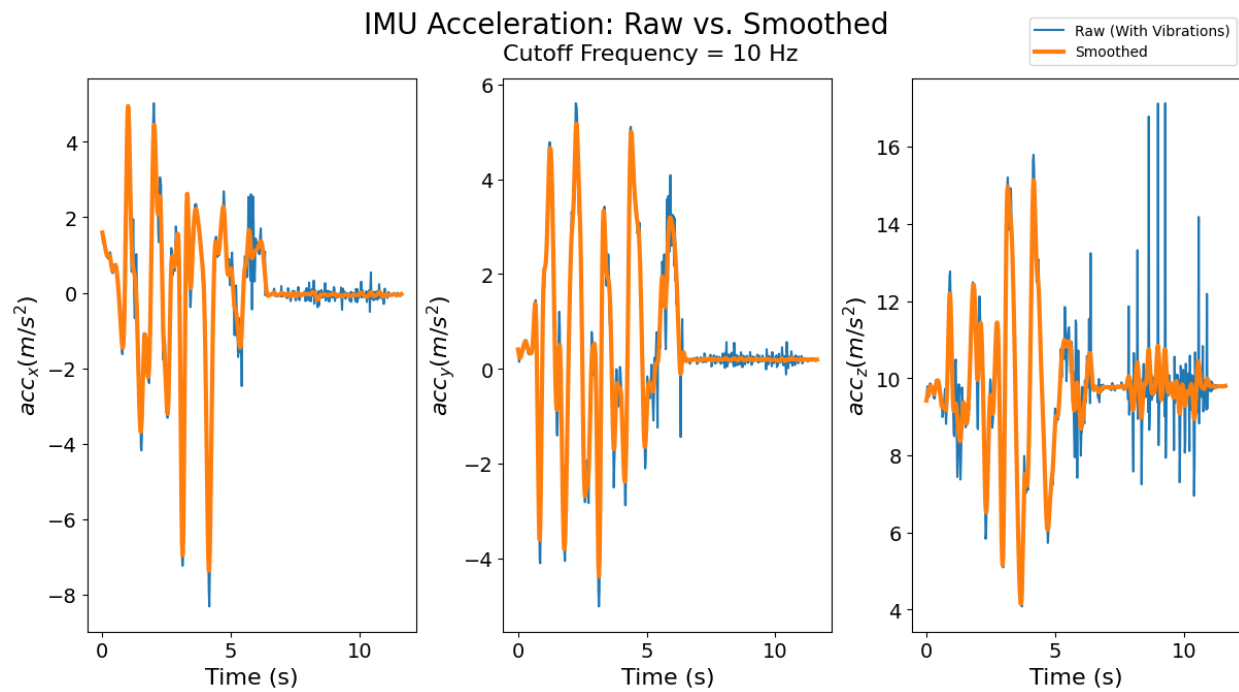


Figure 13.9: IMU Dynamic and Static Motion with Cutoff Frequency 10 Hz

13b. Software Prototyping Plan

13b.i. AI Models

To finalize the selection of our main AI model, there are two different approaches that were considered based on two different papers. These two models include: a Long Short-Term Memory (LSTM) model and a Gated Recurrent Unit (GRU) model.

In the study by Tang et al. [22], they propose a GRU+AKF model with 12 input channels and the GRU architecture as follows.

Inputs:

- **3-axis accelerometer** (specific force): f_x, f_y, f_z from the IMU
- **3-axis gyroscope** (angular velocity): $\omega_x, \omega_y, \omega_z$ from the IMU
- **3-axis velocity** (INS-estimated): v_N, v_E, v_D computed by the INS mechanization
- **3-axis attitude** (INS-estimated): roll, pitch, yaw from the INS

GRU Architecture:

- GRU layer with 128 hidden neurons
- Fully connected output layer with sigmoid activation
- **Time step** (sequence length): 4 (last 4 epochs of input data)
- **Output**: 2 values, position variation in North and East ($\Delta N, \Delta E$)

Adaptive Kalman Filter:

The navigation filter is implemented as a loosely coupled INS/GNSS error-state Kalman filter with a 15-dimensional state vector, where the measurement noise covariance is adapted online using innovation-based adaptive estimation (Sage–Husa IAE). The state vector comprises:

- **3D position error ($\delta \mathbf{r}$)**: North, East, Down
- **3D velocity error ($\delta \mathbf{v}$)**: North, East, Down
- **3D attitude misalignment (ϕ)**: roll, pitch, yaw errors
- **3D gyroscope bias ($\mathbf{b}_g$)**: x, y, z
- **3D accelerometer bias ($\mathbf{b}_a$)**: x, y, z

We chose to follow this GRU + AKF solution because it mitigates the primary issue of a loosely coupled INS/GNSS system when GNSS signals are blocked. In this case, the Kalman filter is no longer provided with position measurements, and the INS operates in an open-loop fashion, with the position error diverging rapidly. The GRU offers a new measurement source by predicting the horizontal position increments based on the motion that has occurred, which can be accumulated to form a pseudo-GNSS position that can be used to continue the measurement updates

during the outage. This helps to mitigate the drift that would otherwise occur when operating in an open-loop fashion compared to INS-only operation.

This solution also ensures that the navigation system remains physically grounded. In this case, the INS mechanization and Kalman filter continue to implement the state propagation and error-state estimation, with the GRU operating as an aiding system rather than an estimator. The addition of the INS-estimated velocity and attitude as inputs further reduces the complexity of the learning problem by requiring the network to learn the position variation based on a pre-existing estimate of the motion state rather than having to learn the motion dynamics from the raw acceleration and angular rate measurements alone.

Lastly, the adaptive Kalman filter is added to the system because the quality of the pseudo-GNSS measurements changes over time. In this case, small errors in the GRU-predicted increments will build up as the outage progresses, making the pseudo-position measurements less reliable as time progresses. The use of innovation-based adaptive estimation (Sage-Husa IAE) allows the filter to adapt the noise covariance in real-time, allowing the filter to decrease the weight given to the pseudo-measurements as the errors in these measurements build up. This makes the system more stable and amenable to longer GNSS outages.

The model will be trained in a scenario where GNSS is always available. The raw IMU and GNSS position data are loaded from CSV files. INS mechanization and a 9-state error-state Kalman filter are applied over the entire trajectory so that at every time step, the filter updates the INS with the current GNSS position. The corrected velocity and attitude, along with the raw accelerometer and gyroscope data, form a 12-channel input. Sliding windows of these 12-channel inputs are constructed of fixed size of four-time steps, and the corresponding target at every step is the GNSS position difference in the local East-North-Up reference frame.

The inputs are divided into training and validation sets, and a mean and standard deviation scaler is trained on the training set only. The GRU is trained to minimize mean squared error on the position deltas, with early stopping on validation loss and saving the best model. Testing is done by simulating a GNSS outage on a different trajectory. The same INS and error-state Kalman filter are applied in a forward pass on the test CSV file. When GNSS is available, the filter continues to update the INS; when simulating an outage, the INS is run open-loop while the trained GRU predicts the position deltas based on the current 12-channel window. These predicted deltas are integrated to form a pseudo-position measurement that is fused by an adaptive Kalman filter with Sage-Husa innovation-based adaptive estimation for the measurement noise, so that confidence in the GRU predictions decreases as prediction error accumulates. Position accuracy is measured as the horizontal root mean squared error of the fused position solution against the reference GNSS trajectory over the entire run and can be reported separately over the outage period.

In the study by Cao et al. [21], a PSO-LSTM model is proposed for GNSS/INS navigation that learns GNSS position increments from recent IMU and SINS-derived signals and uses those predictions to bridge GNSS-denied periods. The PSO-LSTM model has 10 input channels and the architecture and configurations are as follows.

Inputs:

- Specific force from IMU: f_{ib}^b
- Angular velocity from IMU: ω_{ib}^b
- SINS velocity: V_{SINS}
- SINS yaw angle: γ_{SINS}

Output:

- GNSS position increment between adjacent times: DP_{GNSS}
- During GNSS rejection, predicted increments are accumulated to form a pseudo-GNSS position used in the integrated navigation solution

Network architecture:

- 2 LSTM layers
- LSTM output goes through a fully connected layer using Leaky ReLU activation before the final output

Training and PSO Settings:

- Time step (sequence length): **8**
- Epochs: **100**
- Initial learning rate: **0.005**
- Learning rate drop factor: **0.2**
- Gradient threshold: **1**
- Hidden units search range: **30-80**
- Batch size search range: **30-80**
- PSO population size: **10**
- PSO iterations: **15**
- Learning factor c_1 : **2**
- Learning factor c_2 : **2**
- Inertia weight ω : **0.5**

We chose to follow this PSO-LSTM approach because it directly targets the main GNSS-denied failure mode. When GNSS is rejected, the navigation solution must rely on inertial propagation and the position estimate drifts. The paper's key idea is to train an LSTM to predict the GNSS

position increment between adjacent timestamps from recent IMU and SINS-derived signals. Then these predicted increments will be accumulated to form a pseudo-GNSS position, which helps keep the solution from drifting as quickly as inertial only navigation.

This method is also practical because it predicts step-by-step position changes instead of absolute position. This keeps the learning problem focused on short-term motion. The inputs also include both raw IMU signals and simple motion context, which helps the network relate sensor readings to movement. To make the model more stable and reduce trial-and-error tuning, the paper denoises the IMU data using wavelet-thresholding and uses PSO to pick hyperparameters.

13b.ii. Hardware-Software Integration

To properly integrate the IMU, GNSS receiver, and Jetson Nano, the hardware-software integration phase of prototyping included a few major steps:

1. Connect each sensor to the Jetson Nano individually (on wall power)
2. Verify sensor readings individually
3. Log sensor readings individually to csv files
4. Log sensor readings together to csv files
5. Repeat steps 1-4 while operating on battery power

By completing each of these steps, the alpha prototype will demonstrate that the Jetson is able to collect data from each sensor and log it in an appropriate fashion. While AI drift correction utilizing the physical prototype will not be present at the alpha prototype stage, it will lay the groundwork for the beta prototype to have drift correction present, whether processed in real-time on the Jetson, or post-processed offline.

First, each sensor was connected to the Jetson Nano, and we confirmed that they were able to be accessed on the Jetson. The GPS receiver was connected via USB, but unfortunately stopped working after some testing. Therefore, we adjusted it to use the GPIO pins on the Jetson and the TX/RX pins on the u-blox GPS receiver, as shown in the wiring diagram in the hardware prototyping plan (Section 13a). Then, serial communication was used to read data from the GPS receiver, with a baud rate configured at 115200, and an update rate on the GPS receiver of 10 Hz. These were configured on Windows using the u-blox u-center2 application and updating the GPS receiver's board configuration. Next, to connect the MTi-3 IMU, ROS2 was used to install the IMU drivers from Xsens [24] and connect the IMU. A script was also created and ran upon startup to initialize the IMU on all Jetson startups and start sending the output to a ROS2 topic. Then, we were able to see all the data our IMU could provide, including raw acceleration and angular velocity data, as well as other outputs such as the free acceleration, quaternion, and Euler angles. These ROS2 topics provided the IMU data at 100 Hz and used a baud rate of 921600. Now that each sensor was connected to the Jetson, we verified the outputs of their sensor, ensuring they seemed accurate by viewing them in real-time. To visualize the IMU data in real-time,

we used plotjuggler inside of ROS2 to view the accelerations and Euler angles. All seemed accurate, so we moved on to the next part of prototyping.

At the next phase of the alpha prototype, we were concerned with logging the data in an acceptable format for analysis and usage for AI Model training and development. We started by logging IMU and GPS data (individually) to CSVs using a Python script. This is shown below in Figure 13.10 and Figure 13.11. The IMU data contains the ROS timestamp, acceleration, and angular velocity in all three axes, and a quaternion to represent the current orientation of the IMU. The GPS data contains latitude, longitude, system time, and UTC time received from the satellite, as well as the quality of the signal based on the number of satellites contacted.

timestamp_ros	accel_x	accel_y	accel_z	gyro_x	gyro_y	gyro_z	orient_x	orient_y	orient_z	orient_w
0.030504	-8.658978	2.363010	-3.936519	0.000775	-0.002740	-0.000108	0.000000	0.000000	0.000000	1.000000
0.040504	-8.661504	2.356749	-3.927523	0.000475	-0.003366	0.000275	0.292709	0.784319	-0.049580	0.544708
0.050504	-8.660896	2.366418	-3.923431	0.000582	-0.003012	-0.000348	0.292720	0.784314	-0.049588	0.544709
0.060504	-8.673939	2.353374	-3.931326	0.000457	-0.002723	0.000118	0.292718	0.784316	-0.049588	0.544707
0.070504	-8.663358	2.354060	-3.933471	0.000261	-0.002205	0.000308	0.292725	0.784316	-0.049585	0.544703
0.080504	-8.666018	2.356748	-3.932513	0.000150	-0.002431	-0.000368	0.292727	0.784318	-0.049585	0.544700
0.090504	-8.661111	2.348061	-3.946355	0.000213	-0.002431	-0.000300	0.292727	0.784318	-0.049588	0.544698
0.100504	-8.666079	2.352202	-3.946616	0.000585	-0.002824	-0.000009	0.292732	0.784317	-0.049591	0.544697
0.110504	-8.658197	2.358763	-3.934014	0.000461	-0.002606	-0.000083	0.292739	0.784315	-0.049594	0.544697
0.120504	-8.661614	2.355830	-3.930789	-0.000136	-0.003208	-0.000353	0.292739	0.784314	-0.049595	0.544697
0.130504	-8.660454	2.355058	-3.927398	0.000996	-0.002602	-0.000218	0.292742	0.784313	-0.049598	0.544698
0.140504	-8.666595	2.359461	-3.939907	0.000474	-0.002081	0.000139	0.292751	0.784315	-0.049602	0.544690
0.150504	-8.662577	2.362345	-3.942459	0.000145	-0.003010	-0.000573	0.292752	0.784315	-0.049601	0.544689
0.160504	-8.657829	2.355171	-3.943485	-0.000064	-0.002946	-0.000298	0.292750	0.784312	-0.049604	0.544693

Figure 13.10: Collected IMU Data

timestamp_system	gps_time	latitude	longitude	quality
6:55:26.5 PM	11:55:26.5 PM	40.74456	-74.0387	1
6:55:26.6 PM	11:55:26.6 PM	40.74456	-74.0387	1
6:55:26.7 PM	11:55:26.7 PM	40.74456	-74.0387	1
6:55:26.8 PM	11:55:26.8 PM	40.74456	-74.0387	1
6:55:26.9 PM	11:55:26.9 PM	40.74456	-74.0387	1
6:55:27.0 PM	11:55:27.0 PM	40.74456	-74.0387	1
6:55:27.1 PM	11:55:27.1 PM	40.74456	-74.0387	1
6:55:27.2 PM	11:55:27.2 PM	40.74456	-74.0387	1
6:55:27.3 PM	11:55:27.3 PM	40.74456	-74.0387	1
6:55:27.4 PM	11:55:27.4 PM	40.74456	-74.0387	1
6:55:27.5 PM	11:55:27.5 PM	40.74456	-74.0387	1
6:55:27.6 PM	11:55:27.6 PM	40.74456	-74.0387	1

Figure 13.11: Collected GPS Data

Both of these individual logging scripts worked on wall power and battery power. The next and final step was to combine the two into a single CSV file for logging. The challenge at this stage was synchronizing the timestamps between ROS time and the system/UTC time. However, the IMU includes a channel for the UTC time, and this was used to synchronize the two data streams

and log them together. Despite this, since the IMU and GPS receiver are being initialized to collect data at different times (IMU during Jetson startup, GPS receiver during serial connection in the logging script), the times were not perfectly in sync at this stage. As a temporary workaround, the IMU time was used, and the GPS time was attached to the closest IMU reading to the time received by the GPS receiver. This should cause minimal issues with the integrity of the data, as the GPS data should only be inaccurate by +/- 5 ms. The data was once again collected using Python, and the output data (combined GPS and IMU) is shown below in Figure 13.12. In the beta prototype, the ideal state is to further refine this time synchronization issue by logging the GPS by itself with the GPS time, then linearly interpolating the IMU between the nearest two points to get an IMU “measurement” that lines up exactly with the GPS measurement received. Using ROS for timestamping will be explored as it allows for straightforward time synchronization.

time_sys	acc_x	acc_y	acc_z	gyro_x	gyro_y	gyro_z	orient_x	orient_y	orient_z	orient_w	lat	long	quality
08:50:14.737 PM	-8.44016	2.44034	-4.37026	-0.00124	-0.00036	0.00194	0.66193	0.53436	-0.30306	0.42949	40.7445	-74.0388	1
08:50:14.738 PM	-8.43791	2.44376	-4.36847	-0.00062	-0.00066	0.00156	0.66193	0.53435	-0.30306	0.4295	NaN	NaN	
08:50:14.739 PM	-8.43312	2.44854	-4.36649	-0.00037	-0.00063	0.00095	0.66195	0.53433	-0.30308	0.42948	NaN	NaN	
08:50:14.779 PM	-8.4372	2.44505	-4.36508	-0.00031	-0.00024	0.00049	0.66194	0.53435	-0.30307	0.42948	NaN	NaN	
08:50:14.780 PM	-8.43616	2.44594	-4.36333	-0.00047	-0.00064	0.00038	0.66193	0.53436	-0.30307	0.42948	NaN	NaN	
08:50:14.781 PM	-8.43693	2.44528	-4.36583	-0.00022	-0.00075	0.00084	0.66194	0.53434	-0.30308	0.42948	NaN	NaN	
08:50:14.782 PM	-8.43781	2.44687	-4.36674	-0.00104	-0.00021	0.00109	0.66193	0.53435	-0.30308	0.42948	NaN	NaN	
08:50:14.821 PM	-8.43526	2.45135	-4.36757	-0.00126	-0.00085	0.00116	0.66193	0.53435	-0.30307	0.42949	NaN	NaN	
08:50:14.822 PM	-8.43473	2.44275	-4.36201	-0.00108	#####	0.00076	0.66193	0.53436	-0.30307	0.42949	NaN	NaN	
08:50:14.824 PM	-8.43509	2.45168	-4.35691	-0.00117	-0.00038	0.0009	0.66195	0.53433	-0.30309	0.42947	NaN	NaN	
08:50:14.825 PM	-8.44444	2.44288	-4.3654	-0.00028	-0.0005	0.00054	0.66192	0.53437	-0.30307	0.42949	40.7445	-74.0388	1
08:50:14.865 PM	-8.43214	2.44561	-4.36056	-0.00059	-0.0009	0.00094	0.66194	0.53434	-0.30309	0.42947	NaN	NaN	
08:50:14.866 PM	-8.43565	2.44675	-4.36976	-0.00089	-0.00067	0.00092	0.66194	0.53435	-0.30309	0.42948	NaN	NaN	
08:50:14.867 PM	-8.44069	2.45287	-4.36707	-0.00103	-0.00093	0.0011	0.66194	0.53434	-0.30309	0.42948	NaN	NaN	
08:50:14.868 PM	-8.43882	2.4514	-4.37102	-0.00086	-0.00067	0.00147	0.66193	0.53435	-0.30308	0.42949	NaN	NaN	
08:50:14.907 PM	-8.43585	2.44514	-4.36616	-0.00116	-0.00076	0.00137	0.66194	0.53434	-0.30308	0.42948	NaN	NaN	
08:50:14.909 PM	-8.43604	2.44421	-4.35625	-0.00052	-0.00054	0.00096	0.66195	0.53432	-0.3031	0.42948	NaN	NaN	

Figure 13.12: Combined IMU and GPS Data From Jetson

13c. Prototyping Budget

The alpha prototype phase incurred expenses for the team to build it. The team primarily spent money on a compute unit (NVIDIA Jetson Orin Nano) and sensors (IMU and GNSS receiver) to collect data. This was necessary for a meaningful alpha prototype to collect real-world data before moving forward to the beta prototype. Additionally, the team purchased various hardware, such as screws, to fabricate a physical prototype with the components physically mounted. A full bill of materials is shown below in Table 13.1. The total cost for the prototype was \$ 1368.20.

Table 13.1: Alpha Prototype Bill of Materials

Item NO.	QTY.	Item Name	Vendor	Total Price
1	1	MTI-3-5A-DK Xsens Movella IMU Developer Kit	Digikey	\$ 469.00
2	1	GNSS Receiver Board: u-blox EVAL BOARD FOR ZED-X20P	Mouser	\$ 311.25
3	1	NVIDIA Jetson Orin Nano Super Developer Kit	Amazon	\$ 249.00

4	1	Battery: SMALLRIG V Mount VB99 Battery	Amazon	\$ 199.99
5	1	Crucial P310 1TB SSD, PCIe Gen4 NVMe M.2 2280	Amazon	\$ 72.28
6	1	Dual Band M.2 Wireless Card	Amazon	\$ 16.88
7	1	USB 2.0 Cable A Male, Right Angle to Micro B Male, Right Angle 3.3' (1.00m) Shielded	Digikey	\$ 11.08
8	1	Barrel Plug Cable	Digikey	\$ 3.89
9	1	3D Printing Testing Platform	CUSTOM MADE	N/A
10	3	Helical Insert Assortment with Tool, M3 x 0.5 Thread Size	McMaster-Carr	\$ 13.86
		Male-Female Threaded Hex Standoff	McMaster-Carr	\$ 6.60
11	3	18-8 Stainless Steel, 4.500 mm Hex, 10 mm Long, M3 x 0.50 mm Thread		
		18-8 Stainless Steel Cheese Head Slotted Screw	McMaster-Carr	\$ 1.26
12	3	M3 x 0.5 mm Thread Size, 4 mm Long, 4 mm Head Diameter		
13	4	ELEGOO Multicolored Dupont Wire Female to Female Breadboard Jumper Ribbon Cables	Amazon	\$ 00.92
		Alloy Steel Low-Profile Socket Head Screw	McMaster-Carr	\$ 1.93
14	2	Hex Drive, Black Oxide, 4-40 Thread Size, 1/2" Long		
15	2	Helical Inserts with Tools, M5 x 0.8 mm Right-Hand, 5mm Long	McMaster-Carr	\$ 8.52
16	2	Black-Oxide Alloy Steel Socket Head Screw	McMaster-Carr	\$ 00.33
		M5 x 0.8 mm Thread Size, 12 mm Long		
17	1	Black-Oxide Alloy Steel Socket Head Screw	McMaster-Carr	\$ 00.29
		1/4"-20 Thread Size, 3/8" Long		
18	1	Black-Oxide Alloy Steel Socket Head Screw	McMaster-Carr	\$ 1.12
		3/8"-16 Thread Size, 3/8" Long		

14. Testing and Evaluation: Plan and Results

14a. Hardware Testing Plan and Results

The hardware testing and evaluation have been an iterative process, primarily focused on ensuring calibration and data integrity of the onboard sensors. Before passing data to filtering methods and machine learning-based correction, the data collected must be meaningful and calibrated accordingly. This process involved examining and refining the system layout to address any inhibiting factors in data collection and testing it on a few elementary controlled motions and functions. Specifically, the team improved and prepared the existing integrated platform for testing and investigated functionality based on the following categories: translational motion, rotational motion, and GNSS location retrieval. With the completion of these tests, the team anticipates an improved platform setup with meaningful data collection from the IMU and GNSS receiver, ready to proceed to advanced system testing and deployment with more complex trajectories.

First, the ML model can only correct as well as the data it receives; therefore, it is essential to develop a robust platform setup to compensate for the errors of interest: the sensor noise. The first step taken to improve the integrity of the data is reducing the lever-arm, the three-dimensional offset distance between the center of the IMU and the phase center of the GNSS antenna. By minimizing this offset, the acceleration data from the IMU can be as close to matching the changes in velocity that the GNSS records. It should be noted that, despite efforts to minimize this offset, hardware constraints make it challenging, and a non-negligible lever-arm will result. Therefore, the team is investigating pre-processing methods before feeding the data to the model that will translate the GNSS data to the IMU center to represent the same point in space. Additionally, the velocity at the IMU center can be calculated by subtracting the GNSS velocity component due to the offset. Moreover, the sensors must be mounted rigidly, orthogonally, and aligned with the platform frame to minimize noise, prevent gravity from affecting the horizontal accelerations, and maintain correct attitude, respectively. The physical proximity of these components and security is showcased in Figure 14.1 below.

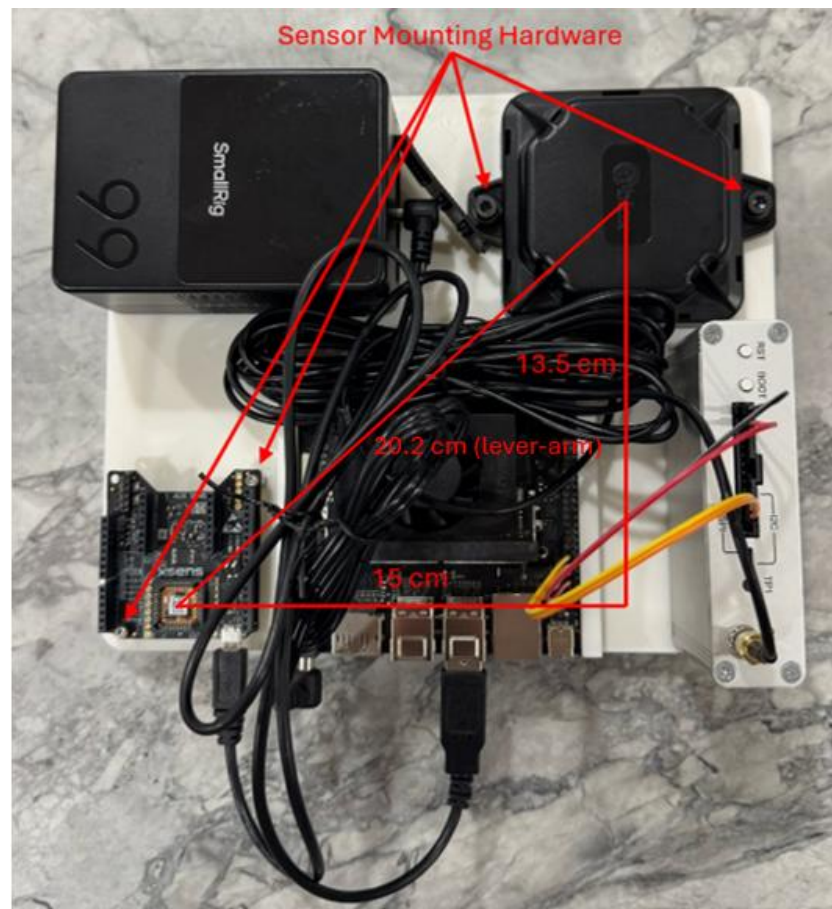


Figure 14.1: Alpha Prototype - IMU & GNSS Relative Alignment and Mounting (top-view)

The mounting hardware includes HELICOIL threaded inserts as well as screws that help rigidly mount the sensors, as well as standoffs for the IMU, so the GPIO pins aren't in contact with the

testing platform itself. As demonstrated, assuming a negligible z-axis offset, the planar lever-arm is approximately 20.2cm, which will need to be accounted for in pre-processing. Additionally, the power consumption of the entire integrated package is around 8.8 Watts while testing with a resting power consumption of 8 Watts.

Finally, the last calibration effort made by the team was to ensure correct time synchronization between the IMU and GNSS. With the IMU and GNSS synchronized, this will ensure the IMU data is in accordance with the GNSS data over time, enabling meaningful drift correction. Please refer to the previous section: Hardware-Software Integration for more information on the team's efforts to achieve proper time synchronization with the team's workflow.

With the platform setup complete, the team is ready to test the sensor's data collection mechanisms and overall hardware functionality in practice. To test the IMU, both translational and rotational motion patterns were investigated. In the translational motion case, the platform is placed on a rolling stool that starts from rest and is pushed approximately along the IMU's x-axis and then brought to rest. The team anticipates a positive acceleration that transitions to a negative acceleration along the x-axis, acceleration due to gravity along the z-axis, and no change in y-axis acceleration or angular velocity. A picture illustrating the testing setup is shown below in Figure 14.2.



Figure 14.2: Translational Motion Case Testing Setup (platform on rolling stool)

For the rotational motion case, the platform was taken off the stool and simply rotated a full rotation about the z-, y-, and x-axes (yaw, pitch, roll). The team anticipates no acceleration, besides

gravity, and a change in the yaw, pitch, and then roll. The raw IMU data for the translational and rotational motions are shown below in Figure 14.3 and Figure 14.4, respectively.

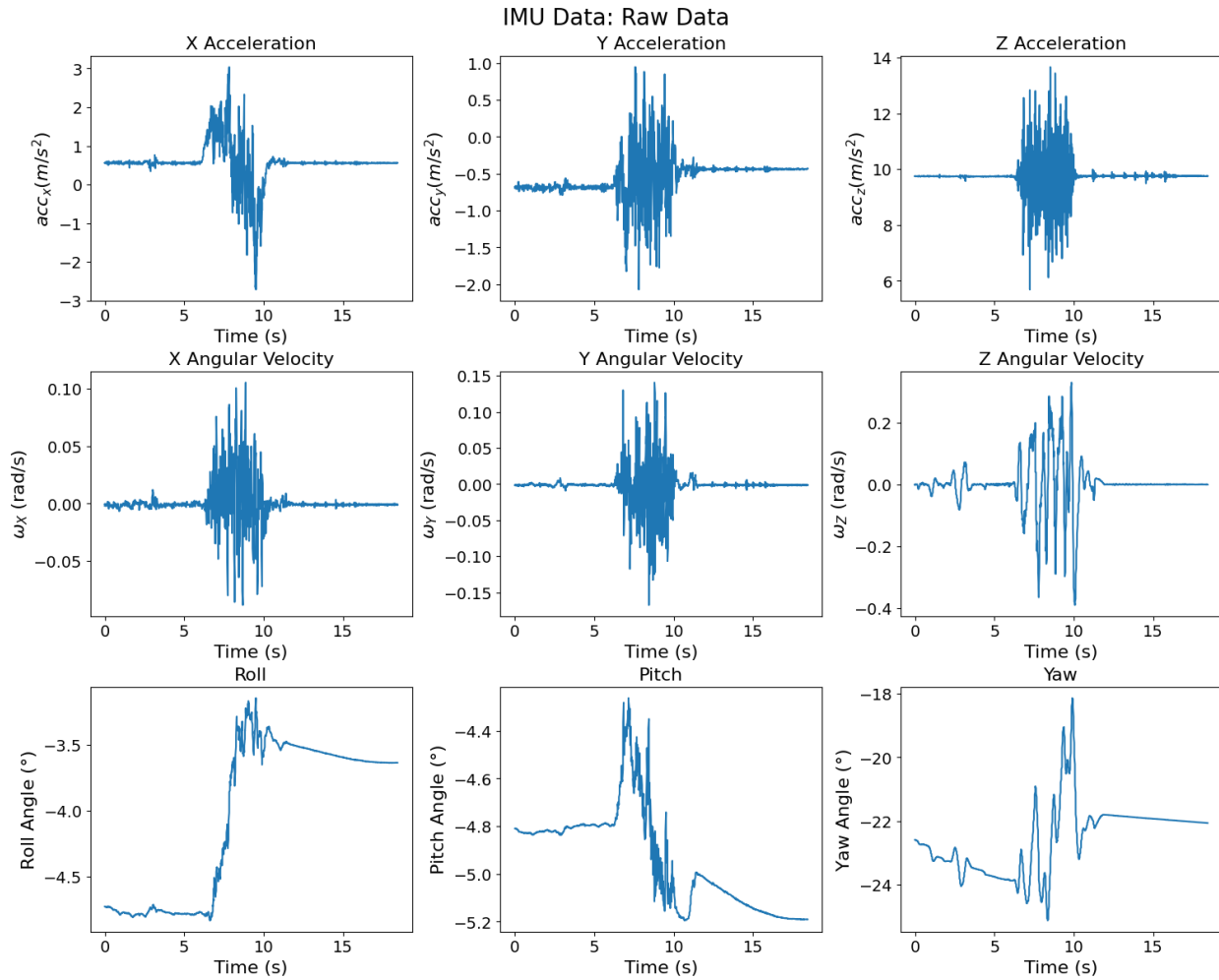


Figure 14.3: Raw IMU Data for Translational Motion Case (along x-axis)

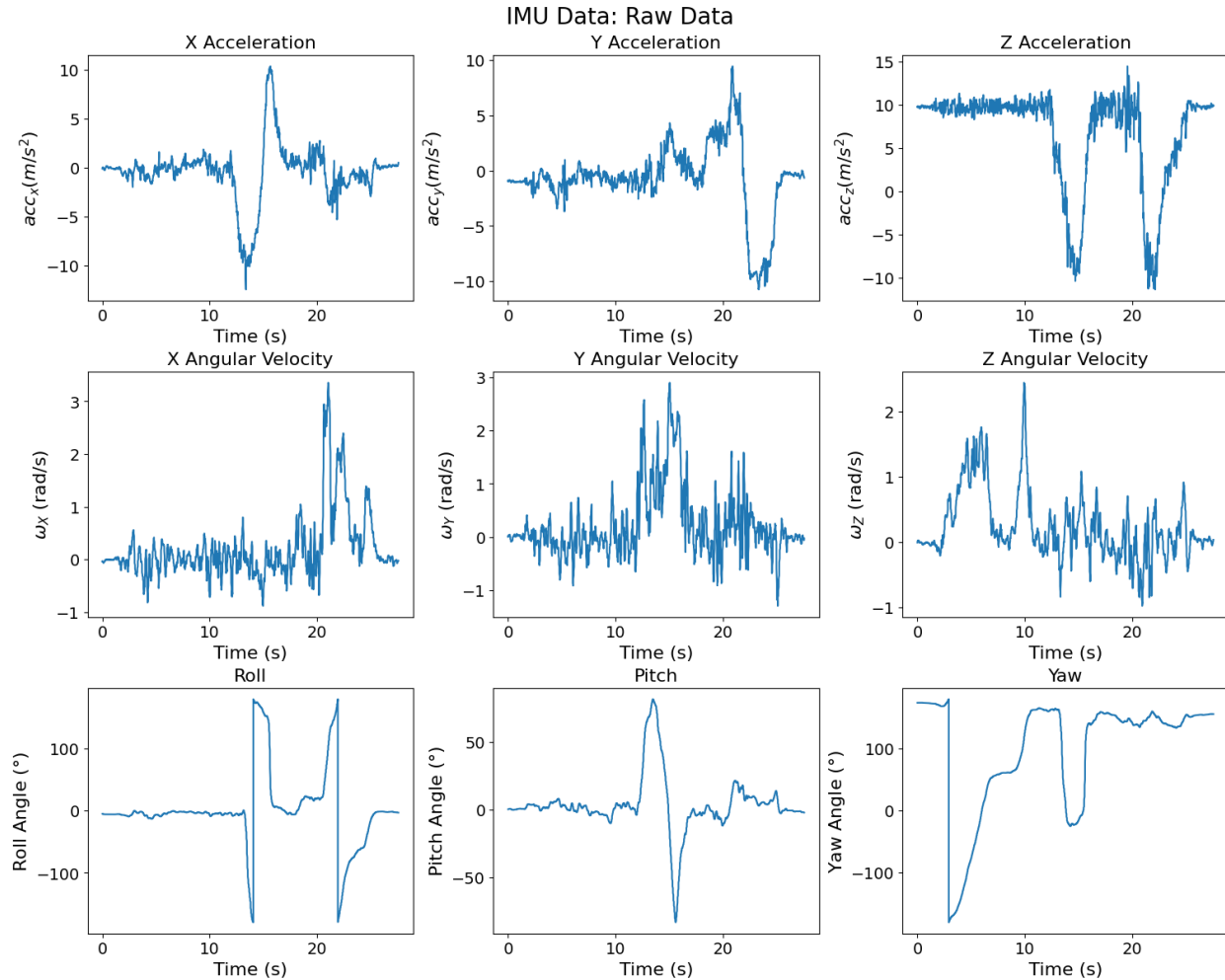


Figure 14.4: Raw IMU Data for Rotational Motion Case (yaw, pitch, roll)

As both cases produce the anticipated results, the system is functioning properly. However, the amount of noise present in these plots should be noted, especially in the z-axis acceleration in the translational motion case. This noise is likely due to the abrupt vibrations in the stool when rolling. To combat this, a 5th-order Butterworth low-pass filter with a cutoff frequency of 5Hz was used for the acceleration data from the translational motion case. The smoothed data is demonstrated in Figure 14.5 below. Note how well the filter reduces the vibrational noise in the z-axis acceleration.

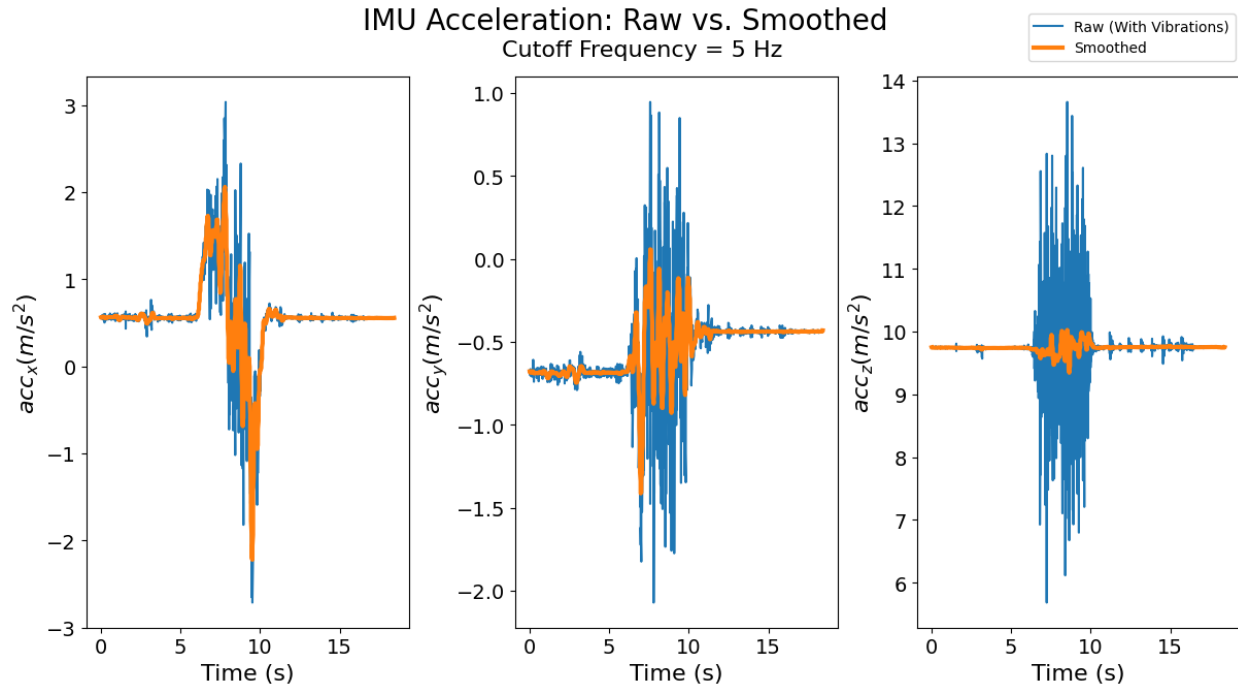


Figure 14.5: Raw vs. Smooth IMU Acceleration Data with Cutoff Frequency 5 Hz from Translational Motion Case

Finally, the GNSS functionality was validated by ensuring accurate position readings. This was tested by simply taking the platform outside to enable clear transmission. As demonstrated previously in Figure 13.11, the system consistently retrieves a GPS fix of highly rated quality. The constant longitude and latitude values align with the behavior of the platform when collecting, as it remained stationary. Additionally, the integrity of the coordinate readings was verified by comparing the coordinates to the actual location, and there were only minor discrepancies.

Now, the team has a functioning platform with hardware that retrieves meaningful data. This setup will continue to improve and evolve as the team begins more complex testing and evaluation. At this stage, the team successfully validated the functionality of the two driving sensors in the system: the IMU and GNSS, while operating the Jetson Nano, running on the battery power supply. This progress aids the transition from testing with simulated data to collecting real, verifiable test data.

For further testing, the team plans to utilize an RC car to mount the platform on, making the necessary adjustments to the platform design to ensure the security of components. The RC car will be used for various trajectories to collect a broad spectrum of data. The team plans to perform significant testing in open areas, away from buildings and other obstructions, for a strong GPS signal, and thus a more reliable ground truth. The GNSS receiver also offers an RTK network that can be utilized for more accuracy, which the team plans to incorporate. Most of the testing will be done locally, making it easily verifiable. With this plan, the team will collect more data on controlled trajectories, which will enhance the correction capability of the ML model.

14b. Software Testing Plan and Results

The software testing and evaluation includes two main sections: the AI models for localization improvement and the integration of the sensors and the Jetson Nano. Both will be discussed for the alpha prototyping phase.

14b.i. AI Models

There are three AI model implementations that were deployed to improve inertial navigation accuracy during GNSS-denied operation. Each model was designed to reduce drift accumulation and maintain position stability when satellite signals are unavailable, and model development and evaluations were conducted using simulated and controlled motion datasets designed to replicate realistic vehicle dynamics while enabling repeatable testing and quantitative performance evaluation.

The first approach under development focused on using a recurrent neural network-based drift correction model optimized with Particle Swarm Optimization (PSO). This model employs a Long-Short Term Memory (LSTM) architecture due to its ability to model temporal dependencies and long-term error accumulation in inertial sensor data. Unlike other neural networks, the LSTM network retains information across time steps, which enables the model to learn motion dynamics and compensate for drift introduced by double integration of accelerometer measurements.

Within the navigation pipeline, the PSO-LSTM model is designed to estimate incremental position corrections from recent IMU measurements, and during GNSS outages, these estimates act as pseudo measurements that constrain the state estimation process, reducing drift growth and maintaining trajectory stability until GNSS signals are restored. This approach aligns with recent GNSS/INS integration research, demonstrating that a PSO-optimized LSTM network can significantly improve navigation accuracy under satellite rejection conditions [21].

Two versions of the PSO-LSTM model are being developed in parallel to evaluate the most accurate and deployment-feasible learning approach for GNSS denied inertial drift correction. These variants differ in network architecture and training configuration, allowing for comparative evaluation of prediction accuracy, convergence behavior, and computational load. Initial testing focuses on assessing drift reduction performance during simulated GNSS outages and verifying that inference latency remains within real-time constraints required for embedded deployment.

One PSO-LSTM variant formulates the learning task as incremental displacement regression, where the network predicts the change in position over the next timestep rather than absolute position. These predicted increments are integrated forward to reconstruct the trajectory during GNSS outages, allowing the model to focus on local motion dynamics and improving numerical stability during training. To improve robustness to sensor noise and spikes in IMU measurements, this implementation uses the Huber loss function during training. Huber loss behaves

quadratically for small errors and transitions to a linear penalty for larger deviations, which reduces sensitivity to outliers while maintaining stable convergence. Model performance is also monitored using mean absolute error to provide an interpretable measure of drift correction accuracy. The architecture is intentionally light to support embedded deployment and real-time inference requirements, and consists of an input normalization layer, a single LSTM layer, and a dense output layer producing three-axis displacement corrections. Particle Swarm Optimization (PSO) is used to identify optimal hyperparameters – including hidden unit size, learning rate, and batch size, by minimizing validation loss during short training runs, ensuring a balance between prediction accuracy and computational efficiency.

During testing with simulated straight-line trajectories, the model maintained close agreement with the ground truth trajectory prior to GNSS denial, and once the simulated outage began, horizontal position error increased gradually rather than diverging rapidly, reaching approximately 2 meters by the end of the outage interval. The predicted trajectory remained stable and closely aligned with the true path, indicating that the network successfully learned short-term motion dynamics and limited drift growth. The gradual error increase is expected due to the absence of external position corrections and reflects residual inertial bias and noise accumulation. These results demonstrate that this PSO-LSTM model effectively constrains drift during GNSS outages while maintaining computational simplicity suitable for real-time embedded operation.

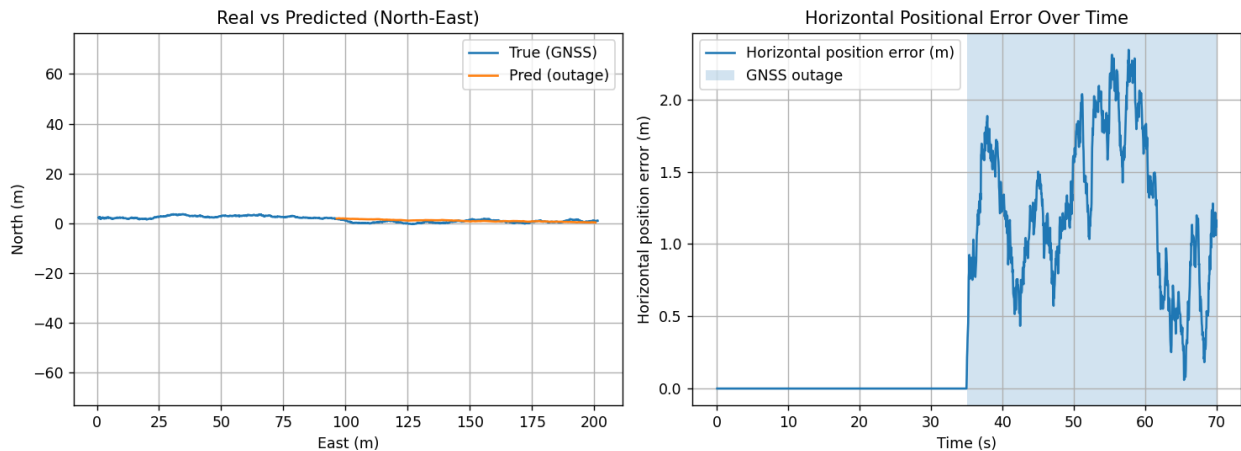


Figure 14.6: *LSTM Straight-line Path Real vs Predicted Data and Horizontal Positional Error*

Another PSO-LSTM variant was implemented to test a slightly different network setup for GNSS-denied navigation. GNSS latitude/longitude/altitude are first converted into ENU coordinates (East, North, Up in meters). The model is trained to predict the next-step ENU displacement (ΔE , ΔN , ΔU) using a short history window of sensor data with a Sequence Length = 8. During a simulated GNSS outage, the algorithm starts from the last valid GNSS position before the outage and repeatedly add the predicted displacements to generate a pseudo-GNSS trajectory. The inputs are 11 features and consist of accelerations (α_x , α_y , α_z), gyroscope rates (ω_x , ω_y , ω_z), INS velocities (v_x , v_y , v_z), yaw, and speed. Inputs are standardized using a scaler fit on the

training data and saved with the model, so inference uses the same normalization. The network is a two-layer LSTM. The model is trained with Adam and mean square error (MSE), and results are reported using RMSE on validation. PSO is used to choose hyperparameters by minimizing validation RMSE. PSO searches hidden units, batch size, and learning rate on a log scale from 10^{-4} to 10^{-1} , using 10 particles and 15 iterations. Performance is evaluated by forcing GNSS outages of different lengths and measuring ENU position and velocity.

Testing of the models is performed using multiple simulated trajectories as previously mentioned, with the two main trajectories being a straight-line path with approximately 70 seconds of data, and a square path that repeats 10 times with approximately 5000 seconds of data. The best results were obtained with the second variation of the PSO-LSTM model. For this model, an outage of 40 seconds was simulated for the straight-line path after 30 seconds of GNSS data, and for the squared path, an outage of 3600 seconds was simulated after 60 seconds of GNSS data.

The results obtained from each simulation were plotted on a Real vs Predicted and a Horizontal Positional Plot, as seen below in *Figure 14-6* and *Figure 14-7*. For the straight-line path North-East plot (*Figure 14-6*), we can see that the Predicted data maintained a close line to the Real data. We can confirm this by looking at the Horizontal Positional Error over Time plot on the right, computed as $\sqrt{(E_{pred} - E_{true})^2 + (N_{pred} - N_{true})^2}$. The error increases gradually throughout the outage, reaching approximately 1.8 m by the end of the window. This was expected from the short straight-line path with a short denial period, since there isn't much time for small step-by-step errors to accumulate.

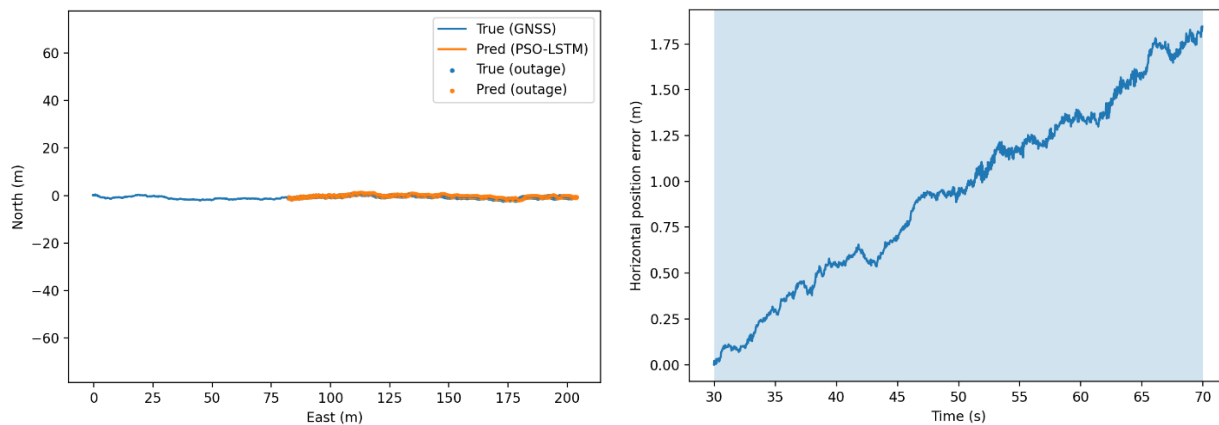


Figure 14-6: LSTM Straight-line Path Real vs Predicted Data and Horizontal Positional Error

However, for the square path, which was a longer path and had a longer simulated outage, we can see below in *Figure 14-7* the North-East plot that linearly drifts with each fully completed square route where GNSS denial simulation occurred. The horizontal Positional Error over Time plot gives us a better view of this linear error accumulation, where we can observe the error line increasing in almost a linear manner, reaching approximately 140 meters of error by the end of the outage.

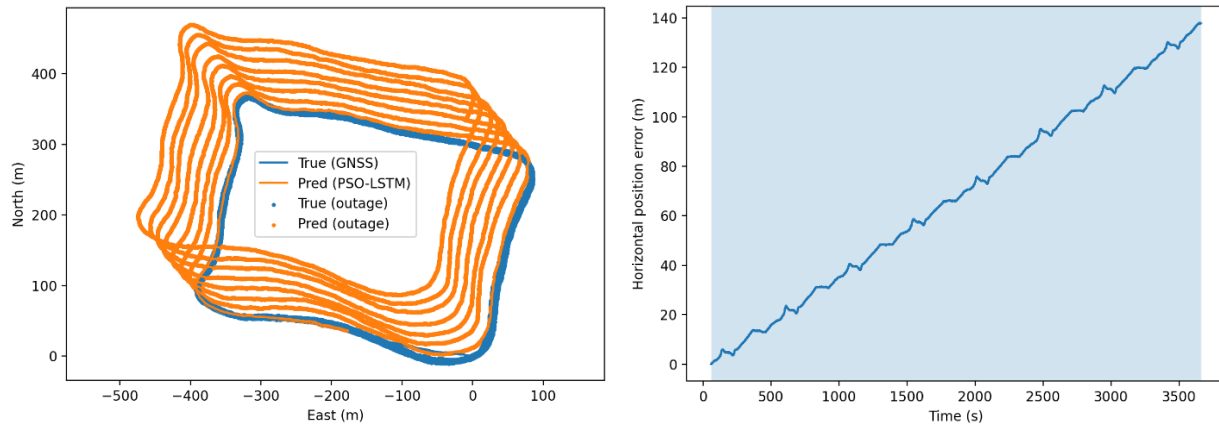


Figure 14-7: LSTM Square Path Real vs Predicted Data and Horizontal Positional Error

Overall, the PSO-LSTM behaves well for short outages but degrades over longer denial periods, mainly because this variant predicts incremental displacements that are integrated forward in an open-loop manner when GNSS is unavailable. Over long outages, even a small systematic bias in the predicted increments accumulates over thousands of timesteps, which is consistent with the near-linear growth in horizontal error seen in *Figure 14-7*. For the next steps, we plan to test alternative loss functions to improve robustness, particularly in the presence of occasional large errors. Specifically, we will experiment with replacing the current MSE loss with the mean absolute error (MAE), since MAE is less sensitive to outliers and may reduce the influence of large deviations during training, leading to more stable predictions during extended GNSS-denied outages.

The GRU-aided adaptive Kalman filter (GRU+AKF) was evaluated on the same two motion profiles used for the PSO-LSTM tests: a short straight line with a forced GNSS outage after an initial period, and a longer square trajectory with an extended outage window. The GRU model provides pseudo-GNSS updates during the outage, and the AKF is intended to down-weight those updates when their uncertainty grows.

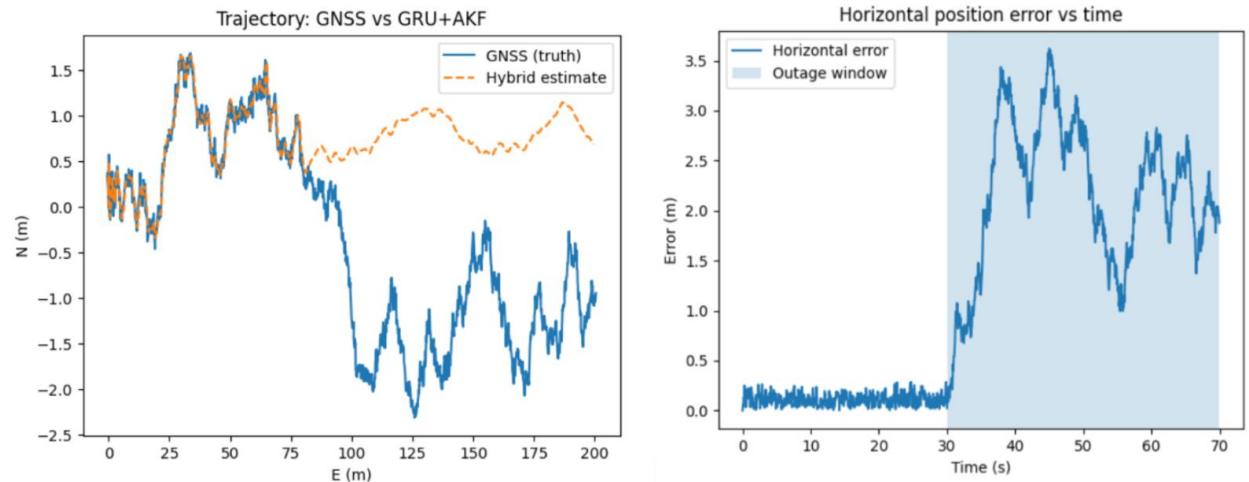


Figure 14-8: GRU Straight-line Path Real vs Predicted Data and Horizontal Positional Error

The first test conducted on the GRU+AKF model was the straight-line test. As seen in figure 14-8, prior to the outage, the hybrid estimate tracks the GNSS truth well, which is a sign that the filter is healthy when the true GNSS data is available. However, once the outage starts, the hybrid track diverges from the ground truth and becomes much smoother and biased, causing the horizontal position error to grow from less than .5m to 2-3.5 m. The oscillating nature of the error growth shows that the pseudo-updates of the GRU model are not consistently aligned with the ground truth, meaning the AKF keeps intermittently mitigating the inertial drift but then is thrown off track by the biased learned corrections. When using a recurrent model that is modeling the short-term dynamics, these small systematic errors in its predictions get accumulated in the filter and cause the trajectory to slowly drift away from the truth.

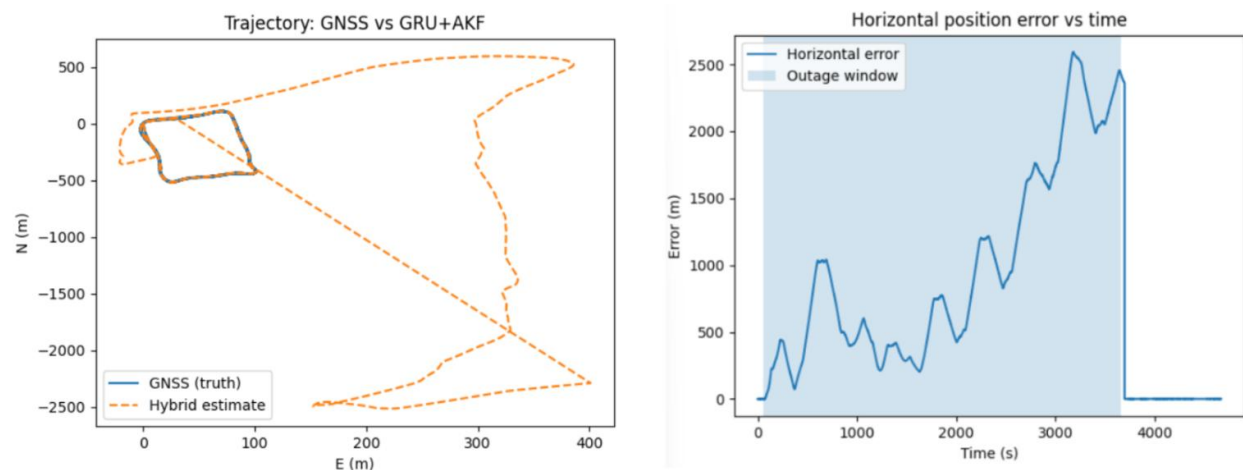


Figure 14-9: GRU Square Path Real vs Predicted Data and Horizontal Positional Error

The second test conducted on the GRU+AKF model was the trajectory of a square ten times. As seen in Figure 14-9, during the outage, the hybrid estimates deviate much more severely and fail to preserve the square trajectory. The trajectory plot shows the estimate cutting across the square instead of tracking it. This, again, indicates compounding bias in the learned pseudo-GNSS updates over time, as previously mentioned. The horizontal error plot shows a very large spike in error during the GNSS outage, which means the model still needs further refinement.

Overall, the GRU + AKF approach behaved as expected when GNSS was available, but it did not hold up as well once GNSS measurements were removed. The straight-line test shows the hybrid estimate staying close to the truth before the outage, then gradually drifting during the denial window. In the square trajectory test, the system diverges much more aggressively, and the predicted trajectory is unable to preserve the shape of the trajectory, indicating that small biases in the learned pseudo-measurements compound quickly over long outage durations. A solution for this will be further explored for the Beta Prototype.

Overall, these results show a practical limitation. This method is highly dependent on how well the GRU generalizes beyond the exact motion patterns it was trained on, and on the filter's ability to appropriately down-weight the learned updates when confidence is low. The reference paper achieves strong performance using substantially more extensive and representative training data and evaluates shorter outage windows (on the order of minutes), whereas our training was constrained to short ~70s datasets, and a single ~5000s square trajectory. We also tested much longer denial periods compared to the paper's 120s and 180s periods [22]. Under these conditions, even mmodel bias is enough to accumulate into large position errors. The main conclusion is that our GRU+AKF implementation is stable with GNSS present but not yet robust for extended GNSS-denied operation, and closing that gap will likely require broader training data coverage, closer alignment to the paper's training and evaluation setup, and more conservative adaptive-noise tuning to avoid over-trusting the GRU during outages.

15. Beta Prototyping

The beta prototype demonstrates the integrated connection of both major subsystems: the hardware package and software model. At this stage, both subsystems are refined from the alpha prototype phase into a fully functional prototype. The inertial and ground truth data collected from the hardware package is now used to train the software models.

15a. Mechanical Design

The mechanical design of the beta prototype shows a complete and more refined version of the enclosure developed during the alpha prototyping phase. At this stage, portability has been transferred into a compact, fully enclosed housing that protects the internal hardware while also improving

the portability and integration with future test platforms. As shown in Figure 15.1, the enclosure uses a box style design with rounded corners, a removable top section, and integrated mounting feet that allow the prototype to be secured while testing. The overall form factor supports a more professional prototype while still allowing access to the IO and interfaces.



Figure 15.1: Beta Prototype

Internally, the beta prototype layout was arranged to package the battery, jetson, GNSS and GNSS receiver, and the IMU in a compact configuration. This arrangement helps reduce unnecessary lengths while also keeping the sensors mounted rigidly utilizing mounting hardware. As seen in Figure 15.2, the inside layout of the prototype can be seen with all components in their respective places. One key feature that is very helpful is the charging port being accessible from the outside for the enclosure which makes it easier to charge the system instead of taking the lid and battery out of the case. This feature can be seen in Figure 15.3.

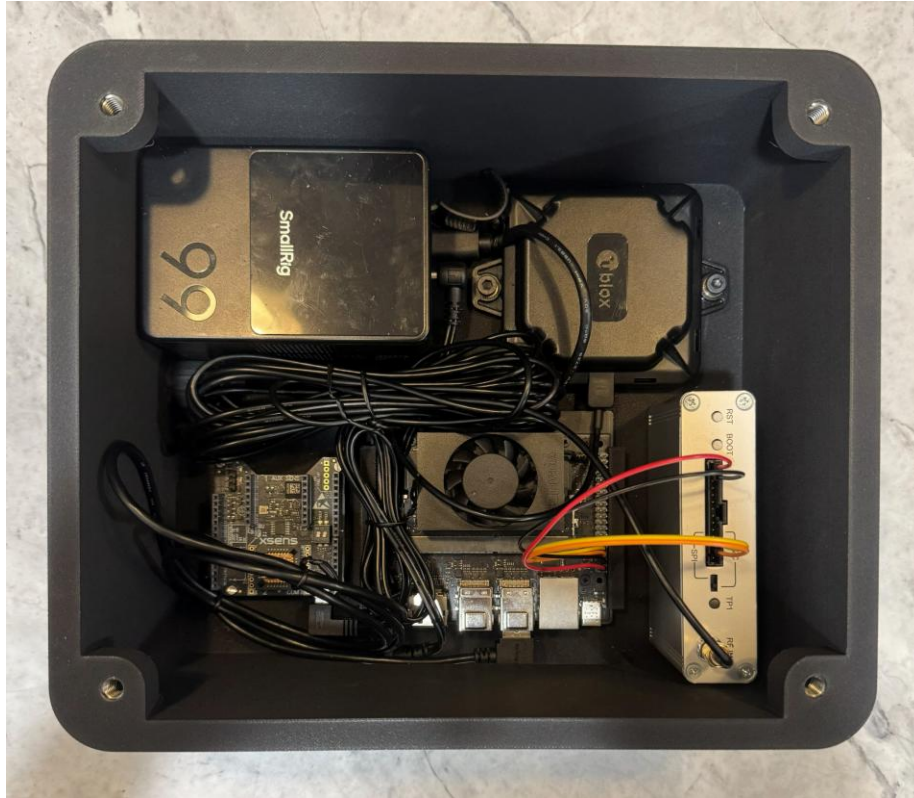


Figure 15.2: Inside Layout of Beta Prototype

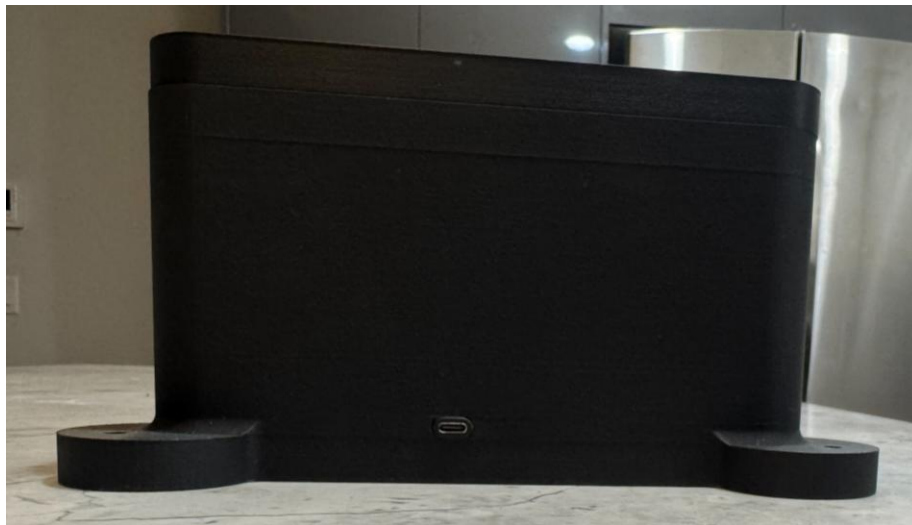


Figure 15.3: Side View of Beta Prototype

To improve both the packaging and functionality of the system, standoffs are being incorporated into the IMU and GNSS mounts to allow the wiring to be routed underneath each component. This helps create a cleaner inside layout, reduces wire clutter, and protects the connections from unnecessary strain during assembly and operation. In addition, the lid is being manufactured from polycarbonate rather than PLA-CF because testing showed that the PLA-CF lid was

interfering with GPS signal reception. By switching to polycarbonate, the design utilizes a durable protective cover while minimizing signal interference and improving GNSS performance.

At this stage, a total of \$3,430.14 has been used out of the \$10,000 allocated budget. This means the team has spent roughly one-third of the available funding while still maintaining a strong buffer for future purchases, testing needs, and any unexpected project expenses. Overall, the project remains in a respected financial position, with sufficient funds remaining to support the rest of the development process.

15b. Electronics Design

The electrical design of our prototype has remained mostly unchanged from the alpha prototype. The NVIDIA Jetson Orin Nano is at the center of our prototype, with all connections going to it in some fashion. The battery is connected via a 12 V DC-DC cable. The Movella MTi-3 IMU is connected via USB (UART) to the Jetson and used for reading the sensor data. The u-blox EVK-X20P GNSS receiver is connected via UART using the RX/TX pins on the Jetson and GNSS receiver. For the beta prototype, all components are mounted inside a box, and wires are twist-tied together to be secured. This provides a compact wiring solution for the beta prototype, which is a change from the alpha prototype. Additionally, right-angle cables are used for the IMU USB cable to provide a more compact solution.

15c. Software Design

15c.i. Machine Learning Models

During the process of beta prototyping, one implementation of the machine learning model, as previously discussed, was a Particle Swarm Optimization tuned Long Short-Term Memory Model (PSO-LSTM), and it was designed to predict short-term vehicle displacement using IMU data during GNSS-denied conditions. The overall objective of this model was to learn the relationship between inertial sensor behavior and resulting position changes, allowing continued navigation when GNSS measurements were unavailable. The dataset used for training conditions consists of mixed IMU and GPS measurements recorded – IMU data was collected at a high rate, and GPS measurement occurred less frequently and served as the ground truth for position. To align these two data sources, the preprocessing pipeline was structured to group all IMU measurement between consecutive GPS timestamps into a single interval, and for each interval, statistical features were collected (including mean, standard deviation and range) to represent the motion that occurred between two GPS updates. The model was trained to predict the displacement vector (delta x, y and z) between consecutive GPS positions, and a sliding window approach was used where a sequence of recent IMU derived feature vectors was provided as input to the LSTM and the model predicted the next displacement – PSO was used to tune key hyperparameters such as hidden layer size, and learning rate which improved model performance without having to follow a process of manual trial and error.

An example of this model's performance is shown in Figure 15.4, highlighting the predicted trajectory compared to the ground truth, as well as the horizontal position error during a simulated GNSS outage. The error CDF shows performance during the outage period, with the median horizontal error of about 396 meters and a 75th percentile error of about 537 meters, indicating that while short term predictions are reasonable accurate, error increases progressively over longer durations because of accumulated drift. The dataset used for the model, producing these results was approximately 16.4 minutes of total runtime, with a simulated GNSS outage of about 8.2 minutes. As shown in the figure, the model can follow the general shape of the trajectory immediately after GNSS loss, demonstrating that meaningful motion patterns are learned from the IMU data, however over long outage durations the error accumulates due to drift.



Figure 15.4: PSO-LSTM (1) Trajectory prediction and horizontal error during GNSS outage using experimental IMU/GPS dataset

During preprocessing, missing IMU values were addressed by exploring imputation, and specifically using time-based linear interpolation, which filled small gaps in the IMU signal by estimating values based on surrounding timestamps, allowing short interruptions in sensor data to be recovered [25]. The interpolation process was, however, constrained to only fill short durations of gaps, ensuring that large missing segments were not being artificially reconstructed. While this approach did improve dataset continuity and enabled stable model training, it introduced an important limitation – linear interpolation assumed smooth motion between two points and therefore estimates values that are not physically measured. As discussed in meeting with Kearfott, this can introduce issues particularly in a dynamic motion scenario where true behavior may not follow a linear trend. As such, while interpolating improved short term data quality, it can introduce bias and reduce the physical realism of the training data.

The primary limitation of the current approach is that it modified the original sensor signal by introducing estimated values, while eliminating one approach to address missing IMU values. To address this limitation, future work will replace interpolation-based imputation with a filter-based approach using a Kalman Filter. Instead of estimating missing values through interpolation, the Kalman Filter will provide physically grounded estimates of system state including

position, velocity, and orientation and in this approach, will perform sensor fusion and noise reduction based on system dynamics rather than filling gaps mathematically.

Another AI model implementation of the PSO-LSTM was developed in parallel. This model had several key differences in preprocessing, feature representation, and loss function. The datasets used for this model consisted of real data gathered by a car, where IMU and GPS rows were interleaved in a single file. This long-format data was converted into a unified wide-format table at 100 Hz by merging IMU and GPS measurements using temporal nearest-match alignment during preprocessing. Wavelet-based denoising was applied to all six IMU channels using a Daubechies-4 (db4) decomposition at four levels with an improved threshold function. This reduces high-frequency noise while preserving the underlying motion dynamics without fabricating missing sensor values. GPS coordinates were converted to a local East-North-Up (ENU) frame, so the model operates in meters. Velocity components and heading angle were derived from position data to provide the model with a sense of speed and direction. Any remaining missing values were filled using linear interpolation with forward and backward fill, and all features and targets were standardized to zero mean and unit variance.

The model input consisted of fifteen features per timestep: accelerometer (3), gyroscope (3), orientation quaternion (4), derived velocity (3), yaw (1), and satellite count (1). These were fed as sliding window sequences of eight timesteps. The targeted output was the per-step displacement in East, North, and UP. A two-layer LSTM captured temporal patterns from each sequence, followed by a LeakyReLU and linear head that output the three position deltas. This model was trained using Mean Squared Error (MAE) rather than Mean Squared Error, making it less sensitive to large outliers. Particle Swarm Optimization simultaneously tuned the hidden layer size, batch size, and learning rate using an adaptive inertia weight to balance exploration and fine-tuning. During a simulated GPS outage, the model predicts deltas step by step, accumulating them into a continuous trajectory without GPS.

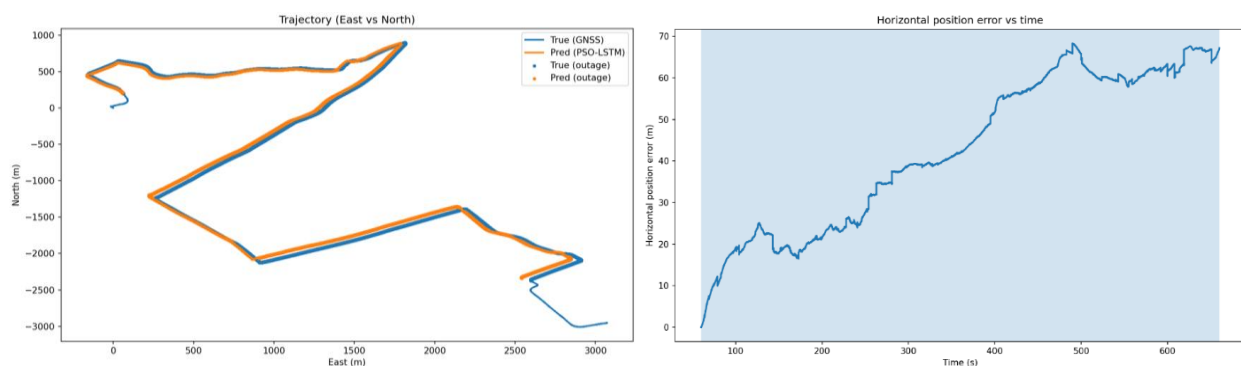


Figure 15.5: PSO-LSTM (2) Trajectory prediction and horizontal error during GNSS outage with first real vehicle path

An example of this model's performance is shown in *Figure 15.5*. The dataset was approximately 16.4 minutes of total runtime, with a simulated GNSS outage of 10 minutes occurring after 60

seconds. The trajectory plot shows the predicted path, in orange, closely following the true GNSS trajectory. The horizontal error plot shows that the error increases and hits its peak at approximately 70 meters after approximately 500 seconds of outage.

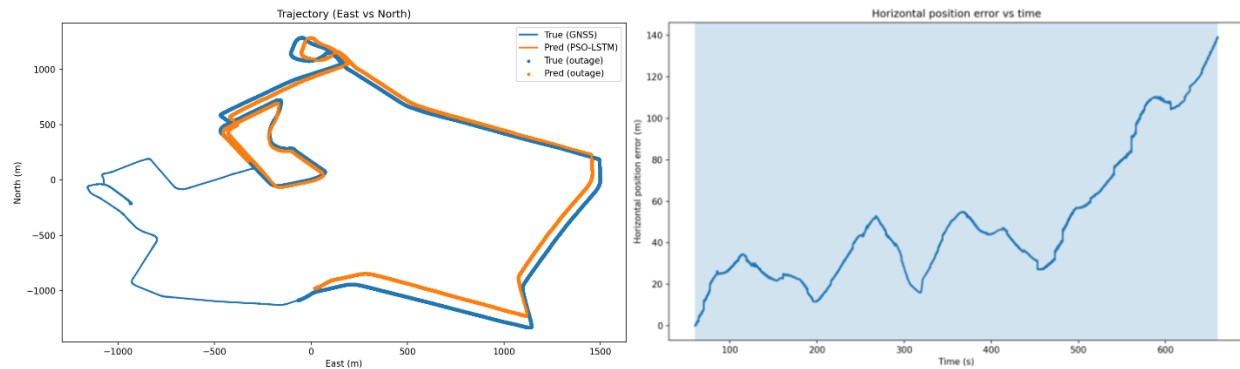


Figure 15.6: PSO-LSTM (2) Trajectory prediction and horizontal error during GNSS outage with second real vehicle path

Another example of this model's performance is shown in *Figure 15.6*. This dataset was shorter at approximately 13.5 minutes long, but a more difficult path with a lot of turns and repeated paths. However, we notice in the trajectory plot that the predicted path follows the true GNSS trajectory closely. Looking at the horizontal error plot, we can see that more errors were accumulated by the end of this path than the previous one above, with the max error being approximately 140 meters at the end of the outage window. As with the first PSO-LSTM model, error accumulates over time due to drift for both paths, however the combination of wavelet-denoised raw IMU features, a richer fifteen-feature input, and MAE-based training contributes to slower drift accumulation under GNSS-denied conditions.

In an attempt to find a different approach, a GRU plus Adaptive Kalman Filter model was also developed for GNSS denied navigation. The purpose of this model was to combine a learned temporal model with filter based estimation so that the system could continue producing navigation updates during GNSS outages. In this implementation, however, interpolation was still used during preprocessing to handle missing values and maintain a continuous input sequence for training and inference.

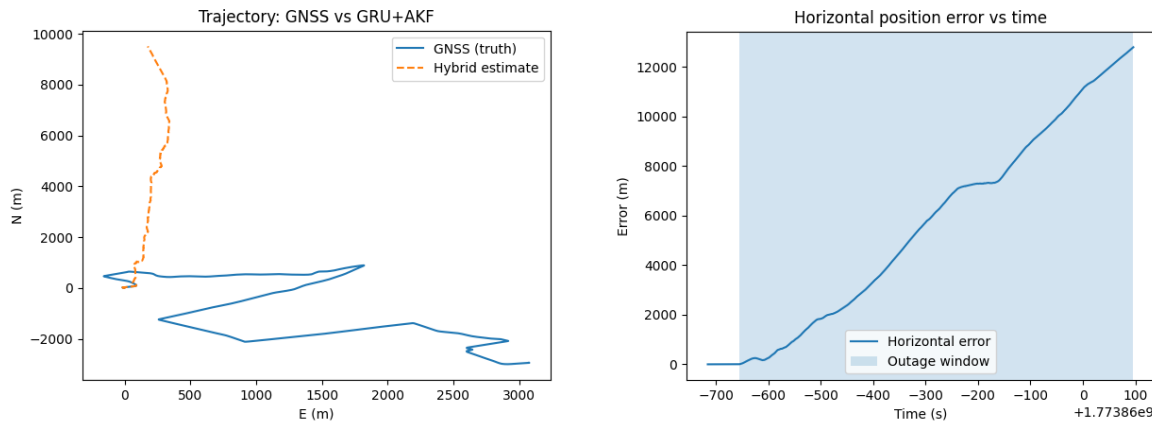


Figure 15.7: GRU+AKF Trajectory prediction and horizontal error during GNSS outage with first real vehicle path

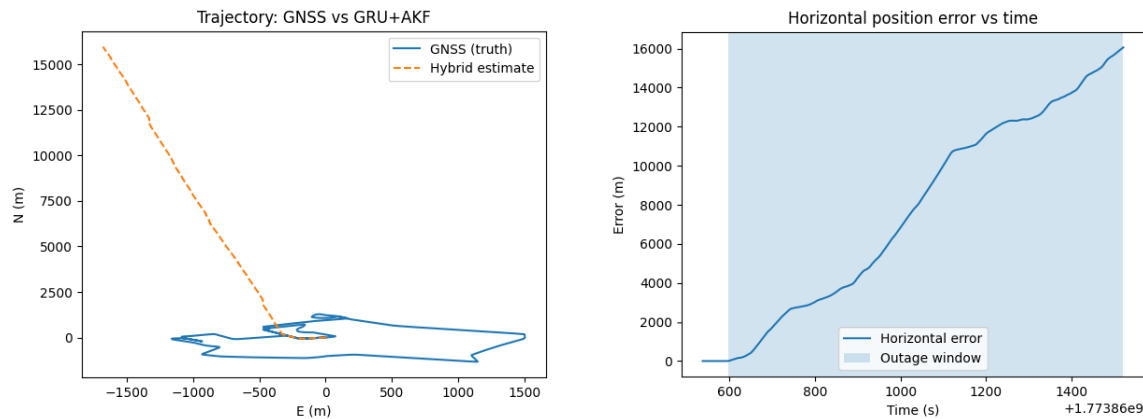


Figure 15.8: GRU+AKF Trajectory prediction and horizontal error during GNSS outage with second real vehicle path

Initial results show that the hybrid pipeline remained reasonable while GNSS was available, but once the outage began, the estimate diverged quickly from the ground truth trajectory. In the trajectory plots, the hybrid path drifted away from the true vehicle path instead of following the turns and overall structure of the route. The horizontal position error plots show the same pattern, with error growing from near zero before the outage to several kilometers during the denied window in the representative runs. This indicates that the current GRU plus AKF model is not yet controlling accumulated error well enough during extended GNSS outages.

One possible reason for this weak performance is the preprocessing itself, and especially the way interpolation was implemented. Although interpolation helped keep the dataset continuous, it may also have smoothed over real motion changes or introduced unrealistic intermediate values that were never actually measured by the sensors. If that happened, then the GRU may have been trained on motion patterns that were cleaner or more regular than the real vehicle dynamics. Once those learned outputs were fed into the adaptive Kalman filter during a long outage, even

small errors in timing, scale, or direction could accumulate quickly and produce the large drift seen in the results. In that sense, the interpolation step may not have just filled gaps in the data. It may also have reduced the physical realism of the signals the model was trying to learn from.

Compared with the PSO LSTM results earlier, the GRU plus AKF model is not nearly as robust. The PSO LSTM runs on the real vehicle paths showed substantially lower error over similar out-of-age periods, while the current hybrid model drifted much more aggressively. Because of that, the main takeaway from this implementation is that there's most likely an important limitation in the current preprocessing and integration strategy.

Future work will focus on improving how missing data is handled before training the GRU plus AKF model, verifying that the learned outputs are consistent with the filter state formulation, and checking whether the interpolation stage is distorting the motion patterns enough to hurt model learning. A more physically grounded preprocessing pipeline may be necessary before the GRU plus AKF architecture can be evaluated fairly against the stronger PSO LSTM baselines.

15c.ii. Hardware-Software Integration

The major improvements of the hardware-software interface involved standardizing the sensor's data streams to use the same time variables, debugging sensor latency issues, and ensuring uniform data collection according to the hardware specification. Additionally, focus was placed on the robustness of our logging platforms to system disruptions (such as network and device connections). At the beta prototype stage, the hardware-software integration mostly focused on preparing the sensor data for usage in the AI models onboard the prototype. Rather than implementing the AI drift correction model directly on the Jetson at this stage, the team decided to log the data in a standardized fashion and use that data for training/testing of the model. This allows for the most flexibility in gathering data as well as using it for model training post-collection, while also laying the groundwork required for real-time integration.

To facilitate the standard gathering of the data, troubleshooting occurred throughout the beta prototyping phase to integrate sensors with the Jetson and the ROS environment. Our Movella MTi-3 IMU has an open-source ROS 2 driver [26], [27] that is used to publish the data into various ROS topics. Then, this data is collected via a Python script to log the data in a CSV file.

Throughout this process, the IMU had some discrepancies and inconsistencies in the logging process, such as not collecting data at a clean 100 Hz logging rate. To address this, the baud rate was increased on the IMU itself (to 230400), as well as the port on the Jetson (/dev/ttyUSB1) set to low-latency mode. After making these changes, the data was able to be collected from various ROS topics and output to a CSV file for analysis. The channels collected and a brief description are as follows:

1. /imu/data: The raw data in the IMU reference frame. Inside this message is the linear acceleration (accelerometer), angular velocity (gyroscope), and various other readings.

2. `/filter/quaternion`: The orientation quaternion given by the onboard filter of the IMU.
3. `/filter/free_acceleration`: The acceleration after the onboard filter removes gravity from the raw acceleration measurements in an Earth reference frame.

Also inside these topics are the ROS timestamps (in seconds and nanoseconds), which are used to ensure the data is from the same moment in time before combining them in the data output. This is done by using the `TimeSynchronizer` function in the `message_filters` package in ROS.

To capture the GNSS data, the `NavSatFix` message type was used first to publish the GNSS readings into a ROS topic. However, we wanted to use the number of satellites as a quality or strength-of-signal metric in our machine learning model, which the message type does not support. To remedy this, a custom message time in ROS was made, called `gps_custom_data`, and is used to create the message type. The information contained in this message type is the timestamp, longitude, latitude, altitude, and number of satellites. The Python logging script subscribes to this new GNSS topic and records the data. The GNSS data is recorded on a separate line from the IMU, as the timestamps do not perfectly align with each other.

Figure 15.4 shows an example of data we collected at the beta prototype phase, demonstrating the form of the collected data. The IMU collects data at exactly 100 Hz, and the GPS collects data at exactly 10 Hz, demonstrating that the data collection works as intended in its functionality. Because the GNSS receiver and IMU are asynchronous in their timing nature (GNSS uses Unix time, IMU uses ROS system time), the system maps the IMU time to Unix time. When the script is started, it captures the time difference between the IMU and current Unix time, and applies a constant bias to all IMU time values. This assumes that the timing difference between the two sensors starts and ends at the same place, and there is no lag throughout the logging process as the sensor data is received. Following this, the hardware is also ready for AI model on-board integration, as this data is simultaneously being published to ROS topics, which the model can subscribe to as the data input. The beta prototype demonstrates a significant improvement in the sampling rate, as the time between IMU samples are a uniform 0.01 seconds, corresponding to the 100 Hz sample rate.

time_sensor	sensor_type	acc_x	acc_y	acc_z	gyro_x	gyro_y	gyro_z	orient_x	orient_y	orient_z	orient_w	lat	long	alt	num_sats
1773861969.6293	GPS											40.84536	-74.6418	212.1	12
1773861969.6761	IMU	-0.06437	0.048739	9.805924	0.004868	0.003948	-0.00021	0.035456	-0.05107	-0.94031	-0.33458	NaN	NaN	NaN	
1773861969.6861	IMU	-0.01885	0.052615	9.657996	0.005543	0.000715	-0.00052	0.03538	-0.05105	-0.94032	-0.33459	NaN	NaN	NaN	
1773861969.6961	IMU	-0.0472	0.072124	9.739002	0.004093	0.008872	-0.00012	0.035361	-0.05103	-0.94032	-0.33459	NaN	NaN	NaN	
1773861969.7061	IMU	-0.07693	0.063586	9.868585	0.002896	0.012263	-0.00014	0.035417	-0.05102	-0.94032	-0.33459	NaN	NaN	NaN	
1773861969.7161	IMU	-0.08115	0.05296	9.843227	0.00486	0.006379	0.000446	0.03542	-0.051	-0.94032	-0.33459	NaN	NaN	NaN	
1773861969.7261	IMU	-0.02338	0.052753	9.673497	0.005436	0.000445	-0.00066	0.035345	-0.05098	-0.94032	-0.33459	NaN	NaN	NaN	
1773861969.7361	IMU	-0.01041	0.069772	9.751781	0.004701	0.006922	-0.00062	0.035304	-0.05096	-0.94032	-0.3346	NaN	NaN	NaN	
1773861969.7461	IMU	-0.05053	0.054146	9.865185	0.001787	0.01232	-0.00015	0.035353	-0.05095	-0.94032	-0.3346	NaN	NaN	NaN	
1773861969.7561	IMU	-0.06408	0.035169	9.869345	0.003206	0.007801	0.000608	0.035377	-0.05092	-0.94032	-0.33459	NaN	NaN	NaN	
1773861969.7661	IMU	-0.01514	0.051082	9.712963	0.004882	0.001469	-0.00035	0.035315	-0.0509	-0.94032	-0.33459	NaN	NaN	NaN	
1773861969.7320	GPS											40.84536	-74.6418	212.1	12
1773861969.7761	IMU	-0.02031	0.084354	9.719755	0.00439	0.005253	-0.00014	0.035263	-0.05087	-0.94032	-0.3346	NaN	NaN	NaN	
1773861969.7861	IMU	-0.04532	0.083367	9.807743	0.002363	0.011474	3.93E-05	0.035294	-0.05086	-0.94032	-0.3346	NaN	NaN	NaN	
1773861969.7961	IMU	-0.08198	0.043916	9.837486	0.002855	0.008219	0.000996	0.035323	-0.05084	-0.94033	-0.33459	NaN	NaN	NaN	

Figure 15.5: Beta Prototype Sample Data Generated from Logging Script

The logging script (in Python) is shown in Appendix D.2 for reference. Additionally, the IMU data simulator in MATLAB has been further refined to match this data type, with separate rows for GPS and IMU sensor types, as well as adding the quaternion for each IMU estimate. While the simulator is mostly being used for straight-line and repetitive scenarios currently, it will be further refined as needed to increase the training data available for the software machine learning models. The updated data simulator is included in Appendix D.1.

In addition to the logging script, a system script was written to start the navigation drivers needed on the Jetson Nano. This includes connecting the IMU and GNSS receiver, starting the IMU ROS driver, and starting the Python scripts that publish the GNSS data to a ROS topic. The GNSS-ROS publishing script is included in Appendix D.2. This script publishes the GNSS data into two ROS topics: `/gps/fix` and `/gps/custom_data`. There is more information below in the `/gps/custom_data` topic, as it is custom to our project, while the `/gps/fix` topic adheres to the standard NavSatFix message type. The contents of the driver startup script are below. This file is stored in the `/etc/systemd/system` directory on the Jetson. Notably, it starts the ROS drivers in localhost-only mode. This prevents hiccups in the topic visibility (and subsequently, the logging capability of our prototype) when the Jetson connects and disconnects from Wi-Fi. It also references a separate script, `start_navigation_drivers.sh`, which is located in `/home/navfusion`, and is also included below. The second script starts the IMU driver and the GPS publish to ROS script.

Contents of the `navigation_sensors_drivers_startup.service` file

```
[Unit]
Description=Start IMU and GPS ROS2 Nodes
After=network.target

[Service]
ExecStartPre=/usr/bin/setserial /dev/ttyUSB0 low_latency
Type=simple
User=navfusion
ExecStart=/home/navfusion/start_navigation_drivers.sh
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
```

Contents of the `start_navigation_drivers.sh` script

```
#!/bin/bash
# Source environments
source /opt/ros/humble/setup.bash
source ~/ros2_ws/install/setup.bash

# Networking settings
export ROS_DOMAIN_ID=0
export ROS_LOCALHOST_ONLY=1
```

```
# Start GPS script
python3
~/ros2_ws/src/Kearfott_SrDsn_AINav/shared_utils/gpsRead_py/gpsReadToRos.py &

# Start IMU driver
ros2 run xsens_mti_ros2_driver xsens_mti_node --ros-args -p enable_deviceCon-
fig:=true
```

Additionally, a custom ROS2 library was created to allow for obtaining the number of satellites to which the GPS is connected. This will be used in the ML models as a “confidence” metric for the GPS signal, or how much it can be trusted in its state estimation. To create the library, the outputs had to be defined as well as their data types. This is stored in the ROS environment in */home/navfusion/ros2_ws/install/src/custom_gps_msgs*. Below is the definition of the information contained in the topic message.

Contents of the message description file:

```
std_msgs/Header header
float64 latitude
float64 longitude
float64 altitude
int8 quality
int16 satellites
```

The *robot_localization* library was explored to understand its application into our project, which would have implemented a Kalman filter directly into ROS, subscribing to the IMU and GPS data topics and outputting a state estimate of the robot. However, due to the previously mentioned time synchronization roadblocks and ROS environment issues, this was unable to be fully implemented at the beta prototype stage. Despite this, the Kalman filter continues to be an integral part of the machine learning models being used, and this delay does not set us back in implementing it for use in the models.

Finally, the last part of the hardware-software integration for the beta prototype is activating and deactivating the logging script discussed above. To do this, the PM2 process manager continues to be used to quickly start and stop the logging script on the Jetson. This ensures the script continues to run in the background after, for example, an SSH terminal is disconnected because of losing Wi-Fi connectivity. To start the process, the only command that must be typed is “pm2 start 0”, as the logging script is stored in memory upon system boot. Then, when logging is complete, the command “pm2 stop 0” can be used to stop the logging script. The script is designed to handle any abrupt stops in the logging effectively by “flushing” the csv file after a certain number of rows are logged (about 10-12 rows). This reduces the strain on the Jetson’s CPU, as well as ensures data security for the data logging process.

15d. System-Level Design

The system-level design has seen minor changes since the alpha prototype stage. These changes primarily involve clarifying the data flow for the IMU and GNSS readings, focusing only on the necessary steps to prepare the data for the model. In this way, the functionality of the beta prototype can be most easily demonstrated, avoiding some of the supplementary tools that add complexity, such as digital filters and Kalman filters, for now. First, the IMU and GNSS collect raw data and utilize integrated ROS 2 drivers and the NavSatFix message type, respectively, to publish the data into ROS topics. Then, all the data from the ROS topics is time-synchronized and collected by a Python script for data logging. The data logging script outputs a CSV file in a suitable format that is ready for the ML model. Alternatively, for live implementation, the ROS topics can be subscribed to directly from the ML model. Detailed data flow from raw data to prepared data for the ML model is showcased in the diagram below in Figure 15.6.

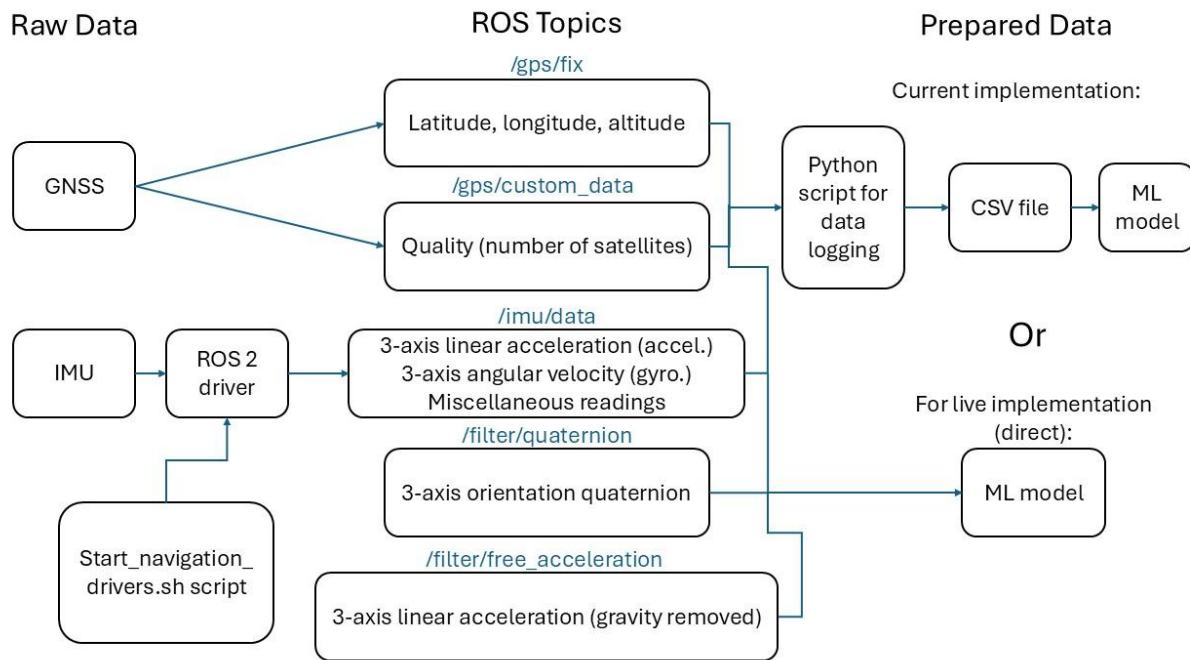


Figure 15.6: Data Flowchart (Raw Data to Prepared Data)

Overall, this diagram outlines the data flow process starting with raw data from the GNSS and IMU and the necessary steps taken to get it in a prepared format for the ML model to utilize effectively. To yield the best results from the ML model, supplemental tools can be integrated into this process, serving as additional pre-processing steps to provide more preferable data to the model. One notable supplemental step would be to implement a low-pass filter to reduce the vibrational noise in the system, as was experimented with previously. Furthermore, a non-holonomic Kalman filter can be used to improve the estimated position before providing the data to the model. The final step, not shown here, is to output the solution from the ML model to various ROS topics that are then ready for utilization.

16. Fabrication, Assembly, and Manufacturing Processes

The beta prototype enclosure and lid were fabricated using Fused Deposition Modeling (FDM) 3D printing process. This manufacturing method was selected because it allowed the team to rapidly produce a custom enclosure, make design revision, and create geometry that would be more difficult to manufacture using traditional manufacturing techniques. The enclosure was designed in SolidWorks, exported as an STL file, and prepared for printing using the Bambu Labs Software. PLA-CF was selected as the enclosure material because it provided the best combination of printability, accuracy, cost, and rigidity.

For fabrication, the enclosure was printed on the Bambu H2C due to the print bed size being accounting for our enclosure size. The specific print settings used were .2mm layer height, 215 °C nozzle temperature, 60 °C heated bed temperature, and 15% percent infill. After printing, the parts were post-processed by removing supports. Threaded inserts and machine screws were then used to assemble the enclosure.

During assembly, the internal components were mounted and arranged to create a compact and secure package. Standoffs and fasteners were used where needed to support the IMU and screws were used to mount the rest of the hardware. Wiring was routed and secured tother to reduce clutter and prevent cable movement during testing, and right-angle cables were utilized where possible to make the internal packaging more compact. This overall fabrication and assembly process results in a fully functional prototype enclosure that is durable and easy to use as well as suitable for system testing.

17. High-Performance Testing and Evaluation

High-performance testing and evaluation of the beta prototype focused on determining whether the mechanical package was suitable for more advanced system testing and deployment. From a mechanical standpoint, the enclosure was evaluated based on its ability to securely contain the hardware and prevent unwanted movement of the internal components. These evaluation goals are consistent with the earlier design of physical rigidity, ease of integration, and manufacturability.

The beta prototype showed clear improvement over the alpha prototype by moving the electronics into a fully enclosed and protected housing and organizing the internal layout into a more compact system. The enclosed package and secured cable routing reduced the likelihood of wiring disturbances during motion, while the mounting hardware improved sensor stability. In addition, compact packaging makes the system easier to mount for various testing platforms. These improvements indicate that the beta prototype is mechanically suited for vehicle-based testing.

To test the beta prototype in a high-performance manner, it was placed in a car and subjected to the motions associated with driving on roads. This includes faster turns, higher speeds, and various road conditions, including bumps and potholes. The prototype was securely placed in a car for testing, and data was logged for various trajectories. The results demonstrate that the GPS

particularly works well and is accurate for the route driven. This is crucial, as the ML models are trained on the GPS data as the ground truth, and must be accurate for valid model performance.

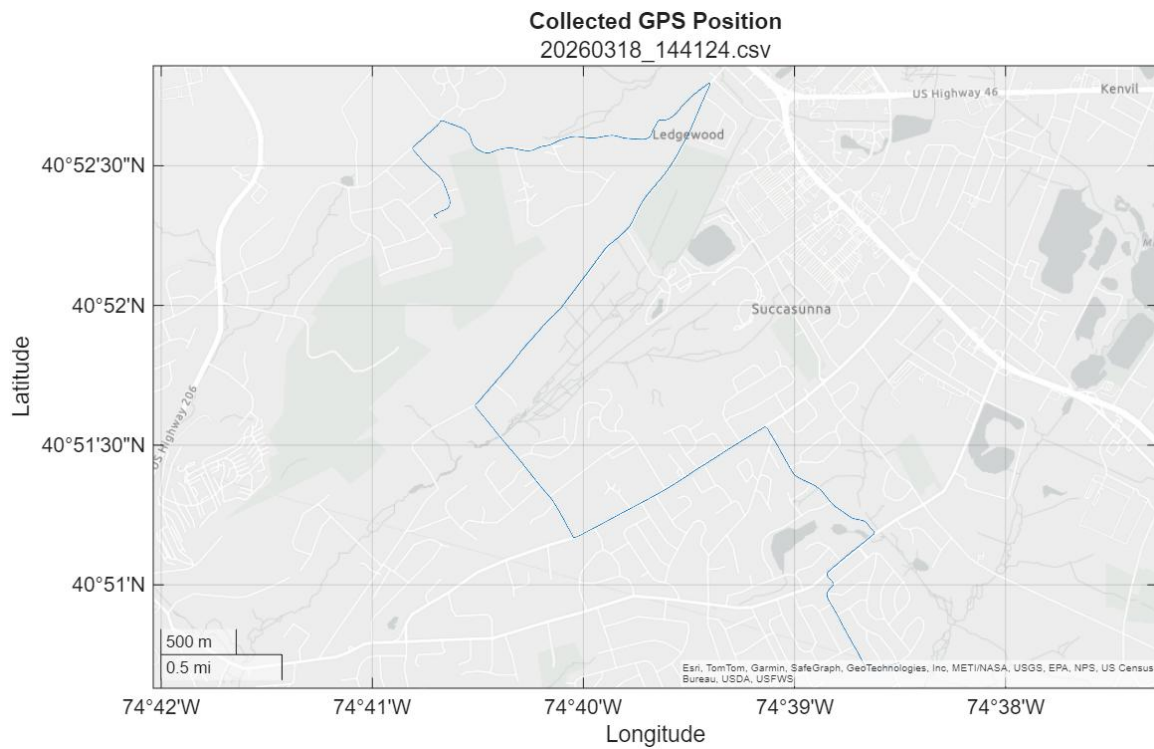


Figure 17.1: High-Performance Test Trajectory 1: GPS Readings

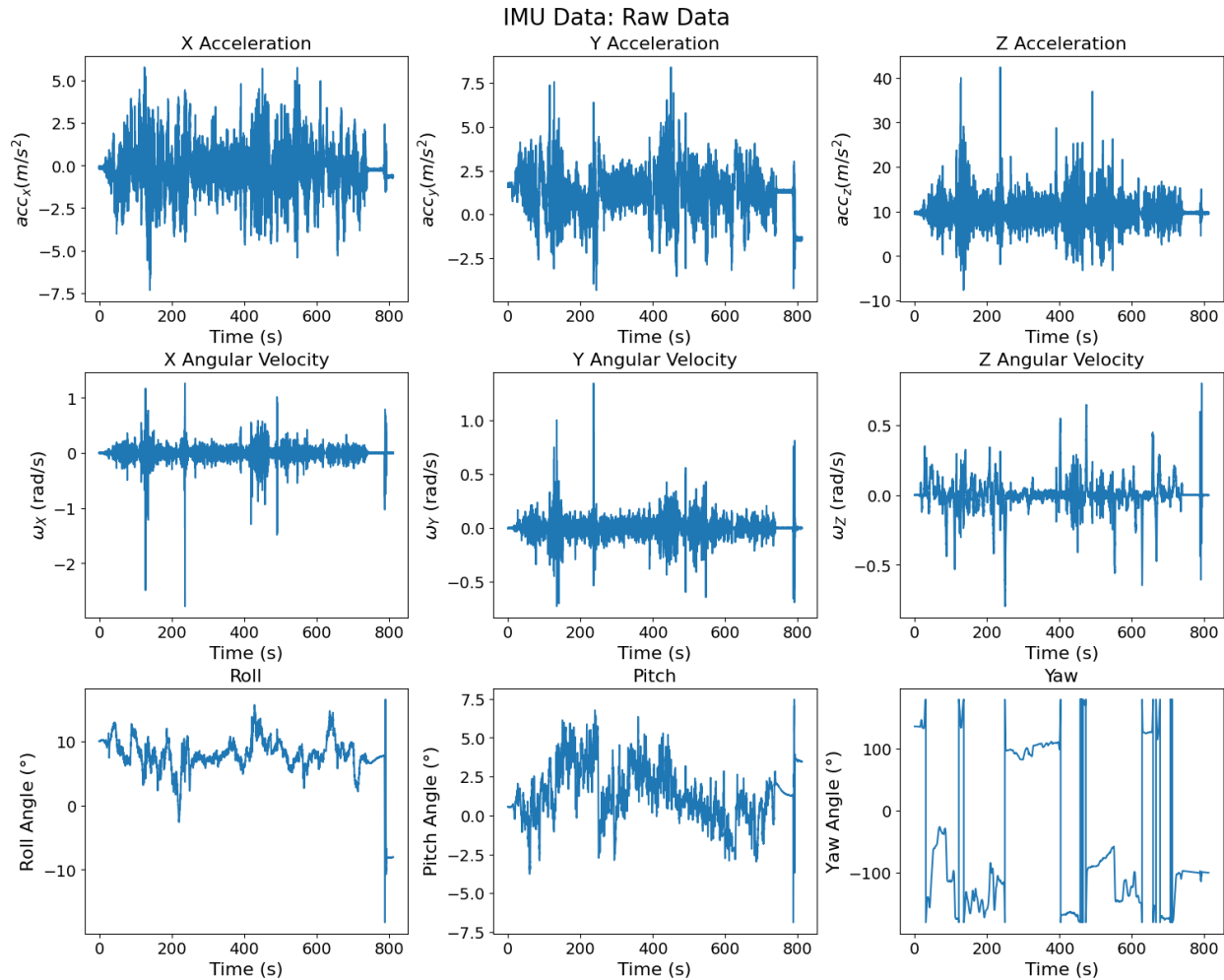


Figure 17.2: High-Performance Test Trajectory 1: IMU Readings

Figure 17.2 demonstrates data consistent with motion from a car. Particularly, the Euler angles demonstrate that the data is acceptable. The roll and pitch angles change relatively little in the overall range they are allowed, while the yaw undergoes major changes, particularly when the route involves a sharp turn. Despite the consistent measurements, the acceleration collected during the run experienced many spikes. To address this, a low-pass filter was applied to the measured data, shown in Figure 17.3. This resulted in a significantly reduced amount of noise, particularly in the Z acceleration, likely due to vibrations experienced in the car, as well as the effect of potholes that cause momentary spikes, particularly in the Z acceleration. When implementing the low-pass filter in real-time machine learning operation, a window can be used to perform the Fast Fourier Transform (FFT) on the incoming sensor data, called a Short-Time Fourier Transform (STFT). Additionally, the Daubechies 4 decomposition (db4) could be used as a substitute for the Fourier Transform, which could be more effective for real-time performance.

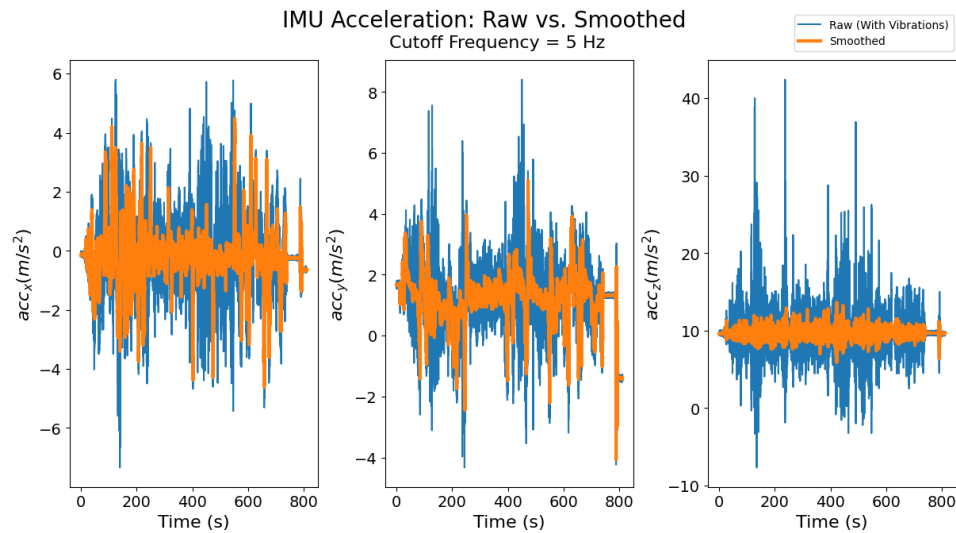


Figure 17.3: High-Performance Test Trajectory 1: Filtered IMU Acceleration

Figures 17.4-17.6 show the same plots of GPS position, IMU readings, as well as filtered IMU readings for a second test trajectory in a car. This testing involved more turns, high speeds, and repetitive types of motions. The results are similar to those of the first trajectory with respect to the IMU readings and filtered IMU readings.

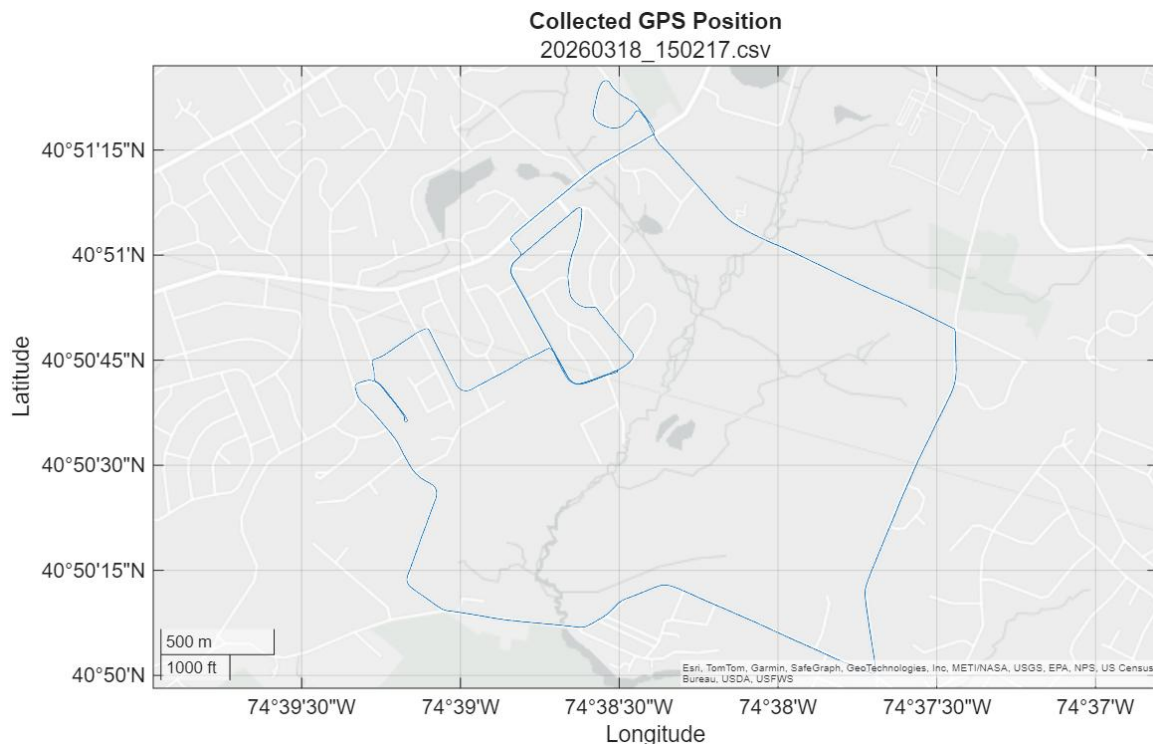


Figure 17.4: High-Performance Test Trajectory 2: GPS Readings

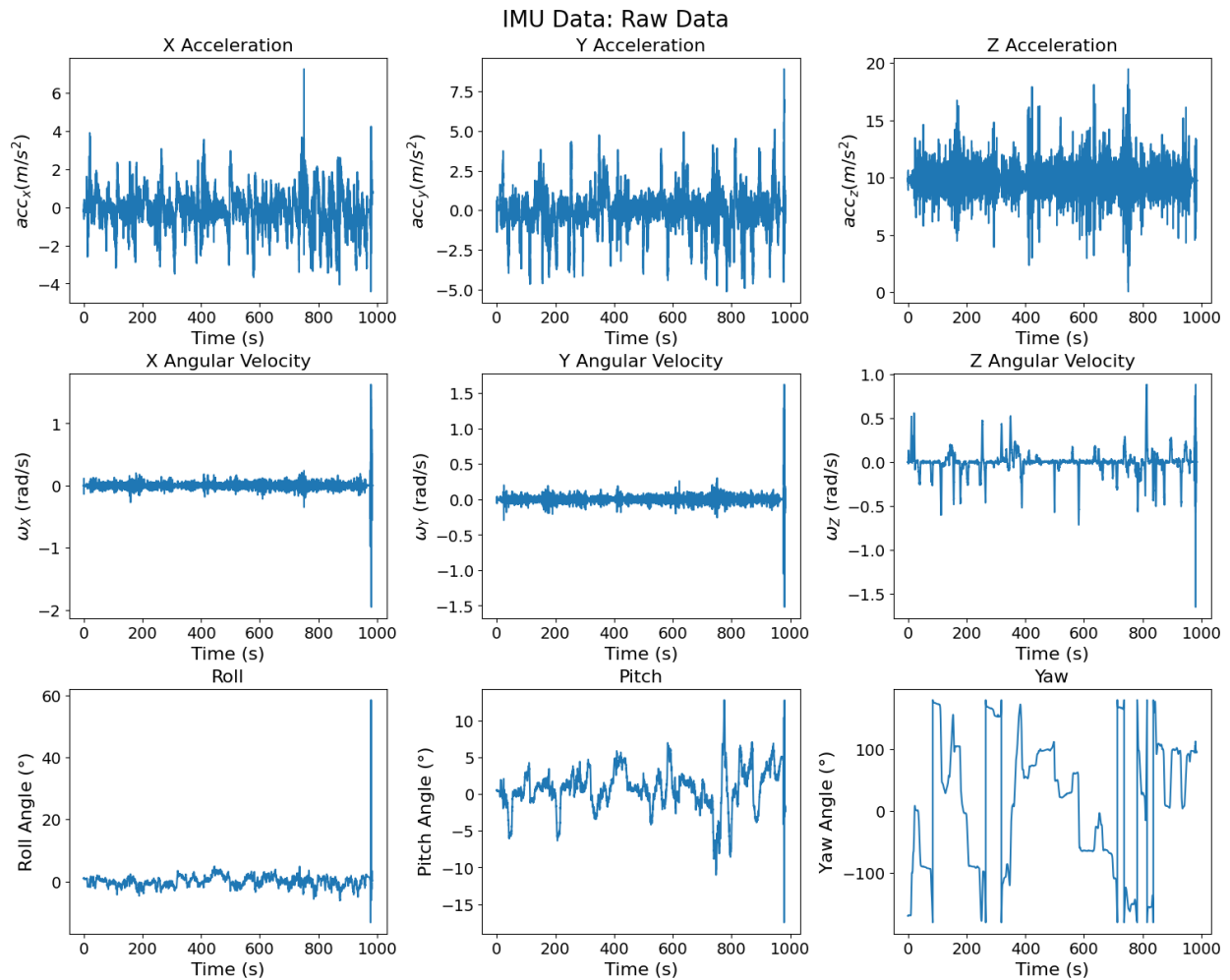


Figure 17.5: High-Performance Test Trajectory 2: IMU Readings

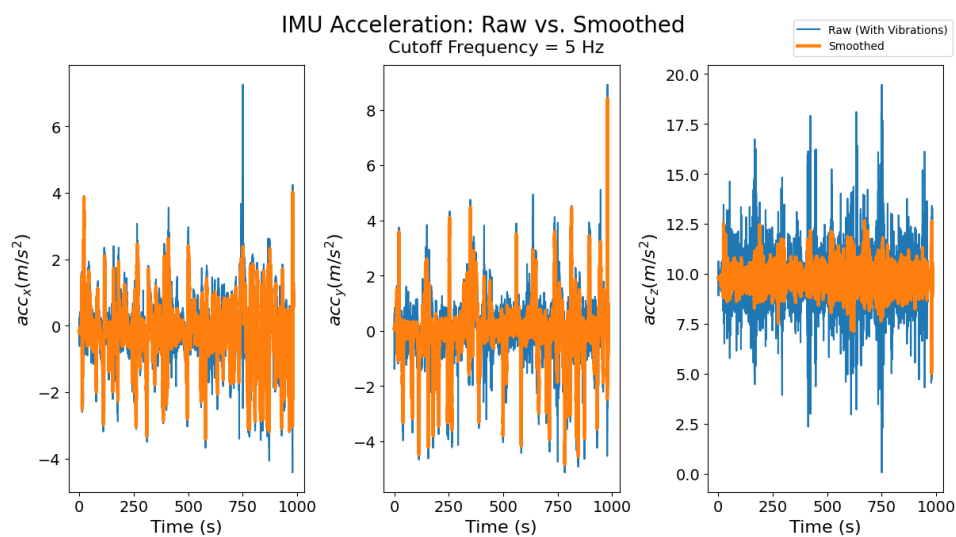


Figure 17.6: High-Performance Test Trajectory 2: Filtered IMU Acceleration

Figures 17.7-17.9 show the same plots of GPS position, IMU readings, as well as filtered IMU readings for a third test trajectory in a car, which is the same trajectory as the first, except in the opposite direction. The results are once again similar to those of the first trajectory with respect to the IMU readings and filtered IMU readings.

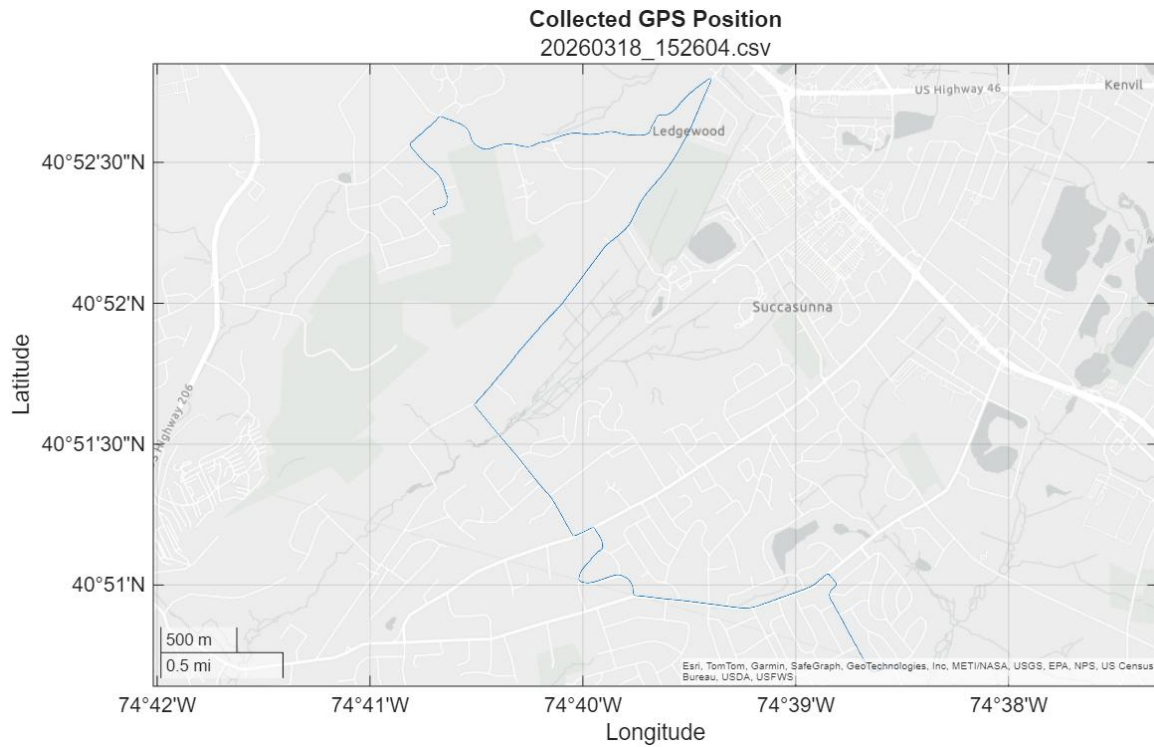


Figure 17.7: High-Performance Test Trajectory 3: GPS Readings

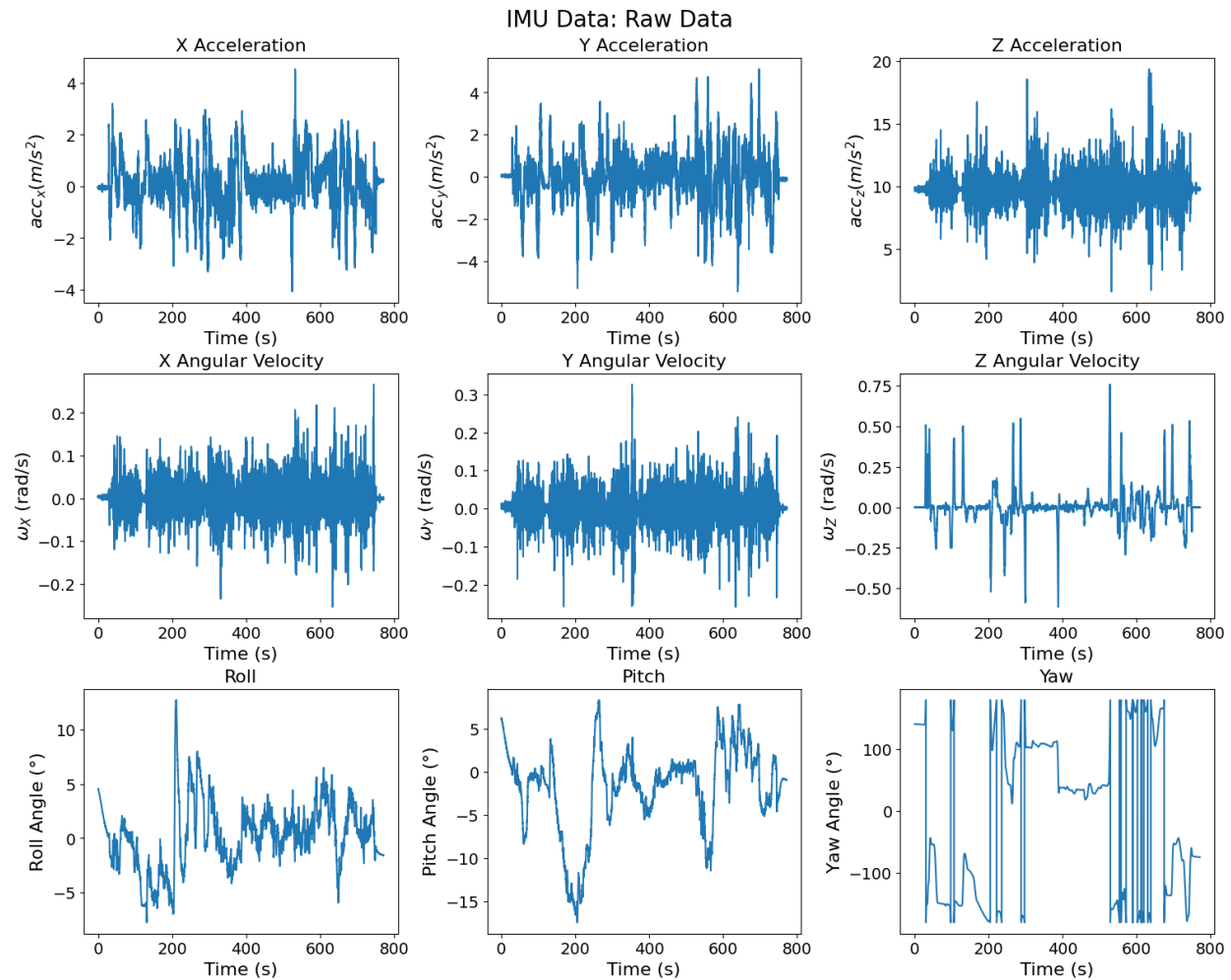


Figure 17.8: High-Performance Test Trajectory 3: IMU Readings

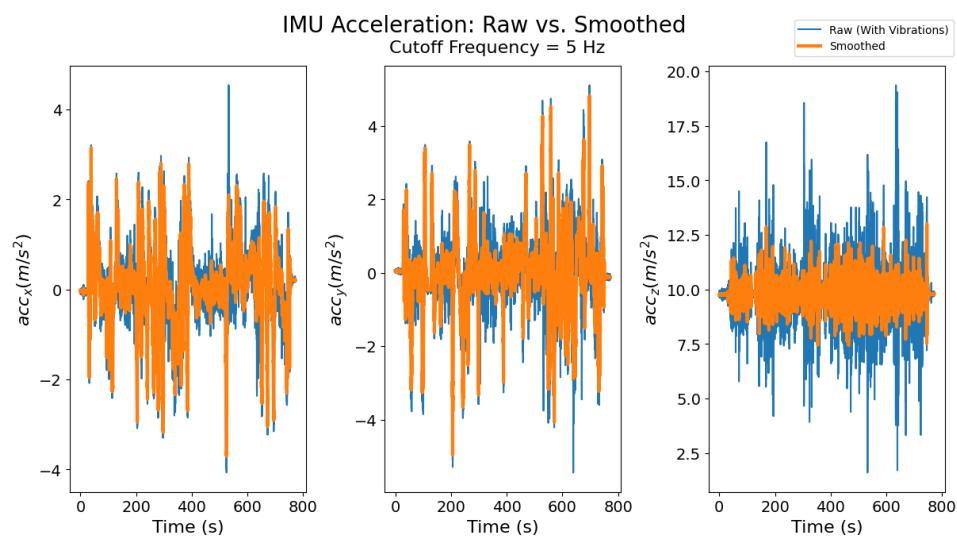


Figure 17.9: High-Performance Test Trajectory 3: Filtered IMU Acceleration

These test trajectories gave a good benchmark of the physical hardware package performance. The package was stable during all operations, and all sensors and parts remained rigidly attached to the frame during testing. Additionally, the results of the ML models when trained on this trajectory indicated positive results.

The second PSO-LSTM model demonstrated strong results when evaluated on two real vehicle trajectories, as discussed in *Section 15c.i*. In both path cases, the predicted trajectory closely followed the shape of the ground truth, indicating that the model learned meaningful motion patterns from the IMU data.

However, it is important to note that these results may be partially attributable to overfitting. Because the model was trained and tested on the same trajectory, the model may have learned patterns specific to that particular path rather than generalizing to arbitrary driving behavior. In a real-world deployment, the vehicle would encounter roads and maneuvers never seen during training, and the model's performance could degrade under those unseen conditions.

To address this limitation, future work we will train the model on one complete path and test it on a separate path of similar complexity, ensuring the model must generalize rather than memorize a single trajectory. The model will also be revised to include heading from magnetometer data as both input and output, and the training target will be replaced with a Kalman Filter blended solution rather than raw GPS positions. Two variants will be developed: an IMU-only model, where GPS position and heading are held fixed at their initial values, so the model is forced to rely on entirely on inertial data, and an IMU/GPS fusion model, where GPS and magnetometer readings are provided at their native update rates without interpolation which will allow the model to learn to fuse these measurements with the IMU signal. Both variants will be evaluated on the unseen test path against the Kalman Filter ground truth, providing a direct comparison between IMU-only and fused navigation performance.

18. Final Design Specification

Section 3: Entrepreneurship and Business Development

- 19. Business Objectives and Risks
- 20. Competitive Intelligence: Market Analysis
- 21. Lean Canvas Business Model
- 22. Financial Analysis
- 23. Intellectual Property
- 24. Innovation Expo (Pitch)

Section 4: Appendices

- A. Team organization chart
- B. Project Gantt chart
- C. Prototyping and Testing Budget
- D. Design Supplementary Materials
- D.1. Technical Analysis MATLAB Scripts

Technical Analysis Case 1: Stationary IMU MATLAB Script

```
%% IMU Drift Simulation (Stationary Short-Term)
clc; clear; close all;

% Simulation parameters
dt = 0.01;           % time step (s)
T = 60;              % total duration (s)
t = 0:dt:T;

% True motion (stationary)
true_ang_vel = zeros(3, numel(t)); % rad/s
true_accel = repmat([0; 0; 9.81], 1, numel(t)); % stationary, gravity only

%% IMU error parameters (MTi-3 representative)
% NOTE: these values are taken from the datasheet. Scale factor error for
% both gyro and accel are zero for now (assume negligible)
% Conversions:
% - deg/hr -> rad/s (divide by 3600 then deg2rad)
```

```

% - deg/s/sqrt(Hz) -> rad/s/sqrt(Hz) (deg2rad)
% - micro-g -> m/s^2 (micro * 1e-6 * 9.81)

% Gyroscope
gyro_bias = deg2rad([0.001; -0.0005; 0.0008]); % small initial biases (rad/s)
% in-run bias stability (datasheet): 6 deg/hr -> converted value:
gyro_bias_inst_rad_s = deg2rad(6/3600); % ~2.9089e-05 rad/s
% For short-term sim: represent bias instability as random-walk increment scale
gyro_bias_rw = gyro_bias_inst_rad_s; % rad/s per sqrt(sec) (approximation)
gyro_scale = diag([0.0, 0.0, 0.0]); % assume negligible
gyro_noise_density = deg2rad(0.003); % rad/s/sqrt(Hz) (~5.236e-05)

% Accelerometer
accel_bias = [0.0003924; -0.0002; 0.0003]; % small initial biases (m/s^2)
% in-run bias stability (datasheet): 40 micro-g -> converted
accel_bias_inst_ms2 = 40e-6 * 9.81; % ~3.9240e-4 m/s^2
% Represent bias instability as random-walk increment scale
accel_bias_rw = accel_bias_inst_ms2; % m/s^2 per sqrt(sec) (approximation)
accel_scale = diag([0.0, 0.0, 0.0]); % assume negligible
accel_noise_density = 70e-6 * 9.81; % m/s^2/sqrt(Hz) (~6.866999e-4)

%% Generate IMU measurements with errors
rng(1);
N = numel(t);
gyro_meas = zeros(3, N);
accel_meas = zeros(3, N);

% Initialize time-varying biases
bg = gyro_bias;
ba = accel_bias;

for k = 1:N
    % Gyro: true + scale + bias + white noise
    gyro_meas(:,k) = (eye(3)+gyro_scale)*true_ang_vel(:,k) + ...
        bg + gyro_noise_density*randn(3,1);
    bg = bg + gyro_bias_rw*randn(3,1)*sqrt(dt); % bias random walk

    % Accel: true + scale + bias + white noise
    accel_meas(:,k) = (eye(3)+accel_scale)*true_accel(:,k) + ...
        ba + accel_noise_density*randn(3,1);
    ba = ba + accel_bias_rw*randn(3,1)*sqrt(dt); % bias random walk
end

%% Integrate to get orientation, velocity, and position
quat = quaternion(1, 0, 0, 0); % Identity quaternion (stationary)
vel = [0;0;0];
pos = [0;0;0];

vel_hist = zeros(3,N);
pos_hist = zeros(3,N);

for k = 1:N
    % Integrate angular velocity

```

```

    omega = gyro_meas(:,k);
    dq = quaternion(omega' * dt, 'rotvec');
    quat = quatmultiply(quat, dq);
    quat = normalize(quat);

    % Rotate accel to global frame and remove gravity
    acc_body = accel_meas(:,k);
    acc_global = rotateframe(quat, acc_body)' - [0;0;9.81];

    % Integrate acceleration
    vel = vel + acc_global * dt;
    pos = pos + vel * dt;

    vel_hist(:,k) = vel;
    pos_hist(:,k) = pos;
end

%% Visualization
figure;
subplot(2,1,1);
plot(t, pos_hist); grid on;
title('Position Drift Over Time (Stationary IMU)');
xlabel('Time (s)'); ylabel('Position Error (m)');
legend('X', 'Y', 'Z');

subplot(2,1,2);
plot(t, vecnorm(pos_hist)); grid on;
title('Total Position Drift Magnitude');
xlabel('Time (s)'); ylabel('|Drift| (m)');

```

Technical Analysis Case 2: Dynamic IMU MATLAB Script

Create a driving scenario with one vehicle. Define waypoints to have the vehicle move forward and make a turn. To simulate GPS readings, specify the reference location in geodetic coordinates. Generate the vehicle trajectory with the smoothTrajectory object function. Plot the waypoints.

```

lla0 = [40.7431 -74.0270 50];
s = drivingScenario('GeoReference',lla0);
s.SampleTime = 1/100; % IMU sample rate (100 Hz)
v = vehicle(s);
waypoints_onex = [-600 0 0;
    -747.154 27.133 0;
    -865.5 54 0;
    -984 80.5 0;
    -985.235 81.798 0;
    -975 120 0;
    -952 220 0;
    -930 320 0;
    -920.080 363.190 0;
    -900 355 0;

```

```

-810 339 0;
-700 313 0;
-550 285 0;
-524.91 272.202 0;
-530 220 0;
-577.1267 49.8103 0;
-598 1 0;];

waypoints = repmat(waypoints_onex, 2, 1);
speed = 3*ones(1, size(waypoints,1));

%waypoints = waypoints_onex;
%waypoints = repmat(waypoints, 10, 1);
%speed = [0 3 3];
%speed = 3*ones(1, size(waypoints,1));
smoothTrajectory(v,waypoints,speed);

dispPlots = false;
outCSV = false;

if dispPlots == true
    figure
    plot(waypoints(:,1),waypoints(:,2),'-o')
    xlabel('X (m)')
    ylabel('Y (m)')
    title('Vehicle Position Waypoints')
end

```

Create Sensors

Create the IMU, GPS, and wheel encoder sensors. Specify the offset location and angles of the IMU and GPS. These can be edited to change the mounting position and orientation of the sensors on the vehicle.

```

mountingLocationIMU = [1 2 3];
mountingAnglesIMU = [0 0 0];
% Convert orientation offset from Euler angles to quaternion.
orientVeh2IMU = quaternion(mountingAnglesIMU,'eulerd','ZYX','frame');
% ReferenceFrame must be specified as ENU.
imu = imuSensor('SampleRate',1/s.SampleTime,'ReferenceFrame','ENU');

```



```
% Specifications from Movella Xsens MTi-3 IMU datasheet
imu.Gyroscope.BiasInstability = deg2rad(6)/3600; % convert to rad/s
imu.Gyroscope.NoiseDensity = 0.003;
imu.Accelerometer.BiasInstability = 40e-6*9.81; % convert to m/s^2
imu.Accelerometer.NoiseDensity = 70e-6*9.81; % convert to m/s^2/sqrt(Hz)
```

```
mountingLocationGPS = [1 2 3];
mountingAnglesGPS = [0 0 0];
% Convert orientation offset from Euler angles to quaternion.
orientVeh2GPS = quaternion(mountingAnglesGPS,'eulerd','ZYX','frame');
% The GeoReference property in drivingScenario is equivalent to
% the ReferenceLocation property in gpsSensor.
% ReferenceFrame must be specified as ENU.
gps = gpsSensor('ReferenceLocation',lla0,'ReferenceFrame','ENU');
gps.SampleRate = 10; % 10 Hz matches our GPS
```

Run Simulation

Run the simulation and log the generated sensor readings.

```
% IMU readings.
accel = [];
gyro = [];
quats = [];
% GPS readings.
lla = [];
gpsVel = [];
% Define the rate of the GPS compared to the simulation rate.
simSamplesPerGPS = (1/s.SampleTime)/gps.SampleRate;
idx = 0;
while advance(s)
    groundTruth = state(v);

    % Unpack the ground truth struct by converting the orientations from
    % Euler angles to quaternions and converting angular velocities from
    % degrees per second to radians per second.
    posVeh = groundTruth.Position;
    orientVeh = quaternion(flipplr(groundTruth.Orientation), 'eulerd', 'ZYX',
'frame');
    velVeh = groundTruth.Velocity;
    accVeh = groundTruth.Acceleration;
    angvelVeh = deg2rad(groundTruth.AngularVelocity);

    % Convert motion quantities from vehicle frame to IMU frame.
    [posIMU,orientIMU,velIMU,accIMU,angvelIMU] = transformMotion( ...
        mountingLocationIMU,orientVeh2IMU, ...
```

```

        posVeh,orientVeh,velVeh,accVeh,angvelVeh);
[accel(end+1,:), gyro(end+1,:)] = imu(accIMU,angvelIMU,orientIMU);
[quats(end+1,:)] = compact(orientIMU);

% Only generate a new GPS sample when the simulation has advanced
% enough.
if (mod(idx, simSamplesPerGPS) == 0)
    % Convert motion quantities from vehicle frame to GPS frame.
    [posGPS,orientGPS,velGPS,accGPS,angvelGPS] = transformMotion(...
        mountingLocationGPS, orientVeh2GPS,...
        posVeh,orientVeh,velVeh,accVeh,angvelVeh);
    [lla(end+1,:), gpsVel(end+1,:)] = gps(posGPS,velGPS);
end
idx = idx + 1;
end

```

Visualize Results

Plot the generated sensor readings.

```

if dispPlots == true
    figure
    plot(-accel(:,1:2))
    ylabel('Acceleration (m/s^2)')
    title('Simulated Accelerometer on Test Trajectory')
    xt = get(gca, 'XTick');
    set(gca,'XTickLabel',xt/50)
    xlabel('Time (s)')
    legend('a_x', 'a_y')

    figure
    plot(gyro(:,3))
    ylabel('Angular Velocity (rad/s)')
    title('Simulated Gyroscope on Test Trajectory')
    xt = get(gca, 'XTick');
    set(gca,'XTickLabel',xt/50)
    xlabel('Time (s)')
    legend('\omega_Z')

    figure
    geoplot(lla(:,1),lla(:,2))
    title('Simulated GPS Position on Test Trajectory')

    figure
    plot(gpsVel(:,,:))
    ylabel('GPS Velocity Simulation Readings (m/s)')
    title('GPS Velocity')
    set(gca,'XTickLabel',xt/50)
    xlabel('Time (s)')
    legend('V_x','V_y','V_z')

```

end

Copyright 2020 The MathWorks, Inc.

Process & Export simulated data to csv

% Create timestamps matrix

imu_timestamps=0:s.SampleTime:s.SampleTime*length(gyro);

imu_timestamps(end) = [];

gps_timestamps=0:1/gps.SampleRate:length(gpsVel)/gps.SampleRate;

gps_timestamps(end) = [];

% Convert lla values into XYZ values based on origin of lla0

[sim_x, sim_y, sim_z] = latlon2local(lla(:,1),lla(:,2),lla(:,3),lla0);

% Combine timestamps and IMU data (with quaternion)

imuDataCombined = [imu_timestamps', accel, gyro, quats];

dataHeaders = {'Time', 'acc_x', 'acc_y', 'acc_z', 'gyro_x', 'gyro_y', 'gyro_z',
'quat_w', 'quat_x', 'quat_y', 'quat_z'};

dataTbl = array2table(imuDataCombined, 'VariableNames', dataHeaders);

dataTbl.Time = round(dataTbl.Time,2);

% Combine timestamps and GPS data

gpsDataCombined = [gps_timestamps', lla, sim_x, sim_y, sim_z];

gpsDataHeaders = {'Time', 'lat', 'long', 'alt', 'sim_x', 'sim_y', 'sim_z'};

gpsDataTbl = array2table(gpsDataCombined, 'VariableNames', gpsDataHeaders);

gpsDataTbl.Time = round(gpsDataTbl.Time,2);

dataTbl.sensor_type = repmat("IMU", height(dataTbl), 1);

gpsDataTbl(:,["acc_x" "acc_y" "acc_z" "gyro_x" "gyro_y" "gyro_z" "quat_w"
"quat_x" "quat_y" "quat_z"]) = array2table(NaN(height(gpsDataTbl), 10));

dataTbl(:,["lat" "long" "alt" "sim_x" "sim_y" "sim_z"]) = array2table(NaN(height(dataTbl), 6));

gpsDataTbl.sensor_type = repmat("GPS", height(gpsDataTbl), 1);

% Join two tables

%allDataTbl = outerjoin(dataTbl, gpsDataTbl, "Keys", "Time", "MergeKeys",true);

allDataTbl = vertcat(dataTbl, gpsDataTbl);

finalTable = sortrows(allDataTbl, 'Time');

```
% Write to CSV
if outCSV == true
    outFileName = 'simData10XSquare_03112026.csv';
    targetDirectory = '../..Data Backup/DataBackup_0217_2108/synthetic';
    %writetable(dataTbl, 'simDataLong.csv');
    fullTextPath = fullfile(targetDirectory, outFileName);
    writetable(finalTable, fullTextPath);
    fprintf('Output file %s generated', fullTextPath)
end
Output file ..\..\Data Backup\DataBackup_0217_2108\synthetic\simData10XSquare_03112026.csv
generated
```

Visualize XYZ

```
if dispPlots == true
    figure
    subplot(1,2,1)
    plot(imu_timestamps, [sim_x sim_y sim_z])
    ylabel('GPS Velocity Simulation Readings (m/s)')
    title('GPS Velocity')
    xlabel('Time (s)')
    legend('V_x', 'V_y')
    subplot(1,2,2)
    plot(sim_x, sim_y)
    axis equal
end
```

D.2. Jetson Nano Beta Prototype Scripts and Programs

Some scripts and programs used in the beta prototype are included in this section for reference.

CSV Logging Script (*allSensorLoggerMk3.py*)

```
#!/usr/bin/env python3
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Imu, NavSatFix
from custom_gps_msgs.msg import GpsData
from geometry_msgs.msg import Vector3Stamped
import message_filters
import csv
import os
import threading
from datetime import datetime

LOG_DIR = '/home/navfusion/ros2_ws/src/Kearfott_SrDsn_AINav/data/raw'
```

```
class InterleavedSparseLogger(Node):
    def __init__(self):
        super().__init__('interleaved_sparse_logger')

        if not os.path.exists(LOG_DIR):
            os.makedirs(LOG_DIR)

        timestamp = datetime.now().strftime('%Y%m%d_%H%M%S')
        self.filename = os.path.join(LOG_DIR, f'allSensor_data_{timestamp}.csv')

        self.csv_file = open(self.filename, mode='w', newline='')
        self.csv_writer = csv.writer(self.csv_file)

        self.csv_writer.writerow([
            'time_sensor', 'sensor_type',
            'acc_x', 'acc_y', 'acc_z', 'free_acc_x', 'free_acc_y',
            'free_acc_z', 'gyro_x', 'gyro_y', 'gyro_z',
            'orient_x', 'orient_y', 'orient_z', 'orient_w',
            'lat', 'long', 'alt', 'num_sats'
        ])

        self.lock = threading.Lock()

        # Add this variable to track the time difference
        self.imu_epoch_offset = None

        # Standard subscriber for the asynchronous GPS data
        #self.gps_sub = self.create_subscription(NavSatFix, '/gps/fix',
self.gps_callback, 10)
        self.gps_sub = self.create_subscription(GpsData, '/gps/custom_data',
self.gps_callback, 10)

        # Message Filter setup for the strictly timed IMU data
        self.imu_sub = message_filters.Subscriber(self, Imu, '/imu/data')
        self.filter_sub = message_filters.Subscriber(self, Vector3Stamped,
'/filter/free_acceleration')

        # ExactTime synchronizer requires the timestamps to match perfectly
        self.ts = message_filters.TimeSynchronizer([self.imu_sub, self.filter_sub], queue_size=10)
        self.ts.registerCallback(self.sync_imu_callback)

        self.get_logger().info(f"Message Filter Logger started. File:
{self.filename}")

    def sync_imu_callback(self, imu_msg, filter_msg):
        with self.lock:
```

```

        sec = imu_msg.header.stamp.sec
        nanosec = imu_msg.header.stamp.nanosec
        sensor_time = sec + (nanosec / 1e9)

        if self.imu_epoch_offset is None:
            # Get the current Jetson system time (Unix Epoch)
            sys_time = self.get_clock().now().nanoseconds / 1e9
            self.imu_epoch_offset = sys_time - sensor_time
            self.get_logger().info(f"Calculated IMU-to-Epoch offset:
{self.imu_epoch_offset} seconds")

        adjusted_time = sensor_time + self.imu_epoch_offset
        t = f"{adjusted_time:.6f}"

        row = [
            t, 'IMU',
            imu_msg.linear_acceleration.x, imu_msg.linear_acceleration.y,
imu_msg.linear_acceleration.z,
            filter_msg.vector.x, filter_msg.vector.y, filter_msg.vec-
tor.z,
            imu_msg.angular_velocity.x, imu_msg.angular_velocity.y,
imu_msg.angular_velocity.z,
            imu_msg.orientation.x, imu_msg.orientation.y, imu_msg.orien-
tation.z, imu_msg.orientation.w,
            'NaN', 'NaN', 'NaN', ''
        ]

        self.csv_writer.writerow(row)

def gps_callback(self, msg):
    """Writes the separate GPS row using hardware timestamps."""
    with self.lock:
        sec = msg.header.stamp.sec
        nanosec = msg.header.stamp.nanosec
        sensor_time = sec + (nanosec / 1e9)
        t = f"{sensor_time:.6f}"

        row = [
            t, 'GPS',
            '', '', '', '', '', '', '', '', '',
            '', '', '', '',
            msg.latitude, msg.longitude, msg.altitude, msg.satellites
        ]

        self.csv_writer.writerow(row)
        self.csv_file.flush()

def destroy_node(self):
    self.csv_file.close()

```

```

        super().destroy_node()

def main(args=None):
    rclpy.init(args=args)
    node = InterleavedSparseLogger()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()

```

GNSS ROS Publishing Script (*gpsReadToRos.py*)

```

#!/usr/bin/env python3
import rclpy
from rclpy.node import Node
from rclpy.time import Time
import datetime
from sensor_msgs.msg import NavSatFix, NavSatStatus
from custom_gps_msgs.msg import GpsData
import serial
from pyubx2 import UBXReader, NMEA_PROTOCOL, UBX_PROTOCOL

class GpsPublisher(Node):
    def __init__(self):
        super().__init__('gps_publisher')

        self.declare_parameter('port', '/dev/ttyTHS1')
        self.declare_parameter('baud', 115200)

        port = self.get_parameter('port').value
        baud = self.get_parameter('baud').value

        self.publisher_ = self.create_publisher(GpsData, '/gps/custom_data',
10)
        self.std_publisher = self.create_publisher(NavSatFix, '/gps/fix', 10)

        try:
            self.ser = serial.Serial(port, baud, timeout=0.1)
            self.ubr = UBXReader(self.ser, protfilter=NMEA_PROTOCOL |
UBX_PROTOCOL)
            self.get_logger().info(f"Connected to {port} at {baud}")
        except Exception as e:
            self.get_logger().error(f"Failed to open serial: {e}")

```

```

        return

    self.timer = self.create_timer(0.05, self.timer_callback)

def get_synced_gps_time(self, utc_val):
    """
    Converts NMEA UTC (HHMMSS.ss) to a ROS Time message.
    """
    try:
        now = datetime.datetime.now(datetime.timezone.utc)
        gps_time_str = str(utc_val) # e.g., "17:24:05"
        combined = datetime.datetime.combine(now.date(),
                                              datetime.time.fromisoformat(gps_time_str))

        timestamp = combined.timestamp()
        seconds = int(timestamp)
        nanoseconds = int((timestamp - seconds) * 1e9)

        return Time(seconds=seconds, nanoseconds=nanoseconds).to_msg()
    except Exception:
        # Use system time if GPS time is missing (no fix)
        return self.get_clock().now().to_msg()

def timer_callback(self):
    try:
        while self.ser.in_waiting > 0:
            (raw_data, parsed_data) = self.ubr.read() # Non-blocking read
            if parsed_data is None:
                continue

            if hasattr(parsed_data, 'msgID') and parsed_data.msgID ==
'GGA':

                std_msg = NavSatFix()
                msg = GpsData()
                currentTime = self.get_clock().now().to_msg()
                msg.header.stamp = currentTime
                std_msg.header.stamp = currentTime
                msg.header.frame_id = "gps_link"
                std_msg.header.frame_id = "gps_link"

                if parsed_data.quality > 0:
                    # Populate standard message fields
                    std_msg.status.status = NavSatStatus.STATUS_FIX
                    #std_msg.quality = int(parsed_data.quality)
                    std_msg.latitude = float(parsed_data.lat)
                    std_msg.longitude = float(parsed_data.lon)
                    std_msg.altitude = float(parsed_data.alt)

                    # Populate custom message fields

```



```
        msg.quality = int(parsed_data.quality)
        msg.latitude = float(parsed_data.lat)
        msg.longitude = float(parsed_data.lon)
        msg.altitude = float(parsed_data.alt)
        msg.satellites = int(parsed_data.numSV)
    else:
        msg.quality = int(parsed_data.quality)
        std_msg.status.status = NavSatStatus.STATUS_NO_FIX

    # Publish both messages simultaneously
    self.publisher_.publish(msg)
    self.std_publisher.publish(std_msg)
    break

except Exception as e:
    # Clears the 'sequence size exceeds buffer' error
    #self.ser.reset_input_buffer()
    pass
    #self.get_logger().warn(f"Buffer reset after error: {e}")

def main(args=None):
    rclpy.init(args=args)
    node = GpsPublisher()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()
```

E. References and Bibliography

- [1] W. Lei, C. Zhang, Z. Jin, and J. Chen, “A review on magnetic-assisted localization for mobile robots,” *Measurement*, vol. 246, p. 116643, Mar. 2025, doi: 10.1016/j.measurement.2025.116643.
- [2] M. B. Alatise and G. P. Hancke, “A Review on Challenges of Autonomous Mobile Robot and Sensor Fusion Methods,” *IEEE Access*, vol. 8, pp. 39830–39846, 2020, doi: 10.1109/ACCESS.2020.2975643.
- [3] K. Borodacz, C. Szczepański, and S. Popowski, “Review and selection of commercially available IMU for a short time inertial navigation,” *Aircr. Eng. Aerosp. Technol.*, vol. 94, no. 1, pp. 45–59, Jan. 2022, doi: 10.1108/AEAT-12-2020-0308.
- [4] P. Grates, “Positioning and Navigation Using IMUs and Low-Cost Sensors,” *IEEE Open J. Instrum. Meas.*, vol. 3, pp. 1–13, 2024, doi: 10.1109/OJIM.2024.3477574.
- [5] G. Cook, *Mobile Robots: Navigation, Control and Sensing, Surface Robots and AUVs*, Second edition. Newark: Wiley, IEEE Press, 2020.
- [6] N. Gyagenda, J. V. Hatilima, H. Roth, and V. Zhmud, “A review of GNSS-independent UAV navigation techniques,” *Robot. Auton. Syst.*, vol. 152, p. 104069, Jun. 2022, doi: 10.1016/j.robot.2022.104069.
- [7] “GPSJAM GPS/GNSS Interference Map.” Accessed: Oct. 03, 2025. [Online]. Available: <https://gpsjam.obliscence.com/>
- [8] C. Chen and X. Pan, “Deep Learning for Inertial Positioning: A Survey,” Mar. 21, 2024, *arXiv*: arXiv:2303.03757. doi: 10.48550/arXiv.2303.03757.
- [9] C. Sahu and S. Poddar, “ALiMU: an end-to-end mapping framework for advanced low-grade inertial measurement units,” *Measurement*, vol. 257, p. 118736, Jan. 2026, doi: 10.1016/j.measurement.2025.118736.
- [10] A. E. Mahdi, A. Azouz, A. E. Abdalla, and A. Abosekeen, “A Machine Learning Approach for an Improved Inertial Navigation System Solution,” *Sensors*, vol. 22, no. 4, p. 1687, Jan. 2022, doi: 10.3390/s22041687.
- [11] B. Patel, M. Raghunandan, and S. Simon, “Artificial Intelligence for Robust Navigation Solutions,” Kearfott Guidance and Navigation Corp, Sep. 16, 2025.
- [12] “IMU Specifications Explained · VectorNav.” Accessed: Oct. 06, 2025. [Online]. Available: <https://www.vectornav.com/resources/inertial-navigation-primer/specifications--and--error-budgets/specs-imuspecs>
- [13] “A Signal Point of Failure: Integrating BeiDou into U.S. Positioning, Navigation, and Timing Systems,” War on the Rocks. Accessed: Oct. 07, 2025. [Online]. Available: <https://warontherocks.com/2025/05/a-signal-point-of-failure-integrating-beidou-into-u-s-positioning-navigation-and-timing-systems/>
- [14] M. Tarek, S. Zahran, E. S. ODA, and S. F. Nafea, “Integrated INS/GNSS Navigation Systems: A Comprehensive Review of Filtering and AI-Based Fusion Techniques,” Jul. 2025, doi: 10.21608/sceee.2025.378670.1076.
- [15] M. Brossard, A. Barrau, and S. Bonnabel, “RINS-W: Robust Inertial Navigation System on Wheels,” in *2019 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, Macau, China: IEEE, Nov. 2019, pp. 2068–2075. doi: 10.1109/IROS40897.2019.8968593.
- [16] H. Chen, T. M. Taha, and V. P. Chodavarapu, “Towards Improved Inertial Navigation by Reducing Errors Using Deep Learning Methodology,” *Appl. Sci.*, vol. 12, no. 7, p. 3645, Apr. 2022, doi: 10.3390/app12073645.

- [17] “PointPerfect Flex,” u-blox. Accessed: Nov. 10, 2025. [Online]. Available: <https://www.u-blox.com/en/product/pointperfectflex>
- [18] “NVIDIA Jetson Orin™ Nano Super Developer Kit.” Accessed: Nov. 06, 2025. [Online]. Available: <https://www.sparkfun.com/nvidia-jetson-orin-nano-developer-kit.html>
- [19] “Amazon.com: SMALLRIG V Mount Battery VB99, 99Wh / 6800mAh V Lock Battery with 100W Output, 2H Full Charge, with USB-C, D-TAP, USB-A, Dual DC Port, OLED Screen, for Camera, Camcorder, Monitor, Video Light : Electronics.” Accessed: Nov. 06, 2025. [Online]. Available: https://www.amazon.com/SmallRig-Battery-VB99-Camcorder-Filmmaker/dp/B0B6Z9K42P/?_encoding=UTF8&pd_rd_w=FPGf7&content-id=amzn1.sym.4efc43db-939e-4a80-abaf-50c6a6b8c631%3Aamzn1.symc.5a16118f-86f0-44cd-8e3e-6c5f82df43d0&pf_rd_p=4efc43db-939e-4a80-abaf-50c6a6b8c631&pf_rd_r=3C7JV9QDCE8Q8Q02NDWE&pd_rd_wg=lpZlK&pd_rd_r=1c3803a2-c9a7-4b28-816b-bc57313eb586&ref_=pd_hp_d_atf_ci_mcx_mr_ca_hp_atf_d&th=1
- [20] “Simulate Inertial Sensor Readings from a Driving Scenario - MATLAB & Simulink.” Accessed: Nov. 10, 2025. [Online]. Available: <https://www.mathworks.com/help/nav/ug/simulate-intertial-sensor-readings-from-a-driving-scenario.html>
- [21] Y. Cao *et al.*, “An GNSS/INS Integrated Navigation Algorithm Based on PSO-LSTM in Satellite Rejection,” *Electronics*, vol. 12, no. 13, Jul. 2023, doi: 10.3390/electronics12132905.
- [22] Y. Tang, J. Jiang, J. Liu, P. Yan, Y. Tao, and J. Liu, “A GRU and AKF-Based Hybrid Algorithm for Improving INS/GNSS Navigation Accuracy during GNSS Outage,” *Remote Sens.*, vol. 14, no. 3, p. 752, Jan. 2022, doi: 10.3390/rs14030752.
- [23] “Choose Inertial Sensor Fusion Filter - MATLAB & Simulink.” Accessed: Feb. 10, 2026. [Online]. Available: <https://www.mathworks.com/help/fusion/ug/introduction-on-choosing-inertial-sensor-fusion-filters.html>
- [24] “Interfacing MTi devices with the NVIDIA Jetson,” XSens. Accessed: Feb. 19, 2026. [Online]. Available: https://base.xsens.com/s/article/Interfacing-MTi-devices-with-the-NVIDIA-Jetson-1605870420176?language=en_US
- [25] “7.4. Imputation of missing values,” scikit-learn. Accessed: Mar. 31, 2026. [Online]. Available: <https://scikit-learn/stable/modules/impute.html>
- [26] “xsenssupport/Xsens_MTi_ROS_Driver_and_Ntrip_Client at ros2,” GitHub. Accessed: Mar. 30, 2026. [Online]. Available: https://github.com/xsenssupport/Xsens_MTi_ROS_Driver_and_Ntrip_Client/tree/ros2
- [27] “Third Party Drivers for use with the MTi.” Accessed: Mar. 30, 2026. [Online]. Available: https://base.xsens.com/s/article/Third-Party-Drivers-for-use-with-the-MTi?language=en_US